



Università degli Studi di Napoli
FEDERICO II

A.A 2022-2023

LINGUAGGI DI PROGRAMMAZIONE I

INDICE

0 INTRODUZIONE	5
I INTRODUZIONE AI LINGUAGGI DI PROGRAMMAZIONE	6
1.1 Cos'è un linguaggio di programmazione	6
1.2 Le macchine astratte	7
1.3 La traduzione dei linguaggi di programmazione	8
1.3.1 Il processo di compilazione	8
1.4 Le proprietà dei linguaggi	10
1.4.1 Semplicità	10
1.4.2 Astrazione	10
1.4.3 Espressività	10
1.4.4 Ortogonalità	10
1.4.5 Portabilità	10
1.5 I paradigmi computazionali	10
2 IL PARADIGMA IMPERATIVO	12
2.1 Perché imperativo?	12
2.2 L'ambiente di esecuzione	13
2.3 Data Objects e bindings	17
2.3.1 I puntatori	18
2.3.2 Il legame di tipo	19
2.4 Blocchi di istruzione	20
2.4.1 Ambito del name binding	21
2.4.2 Ambito del location binding	22
2.4.3 Lo stack di attivazione	22
3 ASTRAZIONE PROCEDURALE	24
3.1 Le procedure come astrazioni	24
3.2 Ambiente di una procedura	25
3.2.1 Record di attivazione	25
3.2.2 Ambiente locale	26
3.2.3 Ambiente non locale	26
3.3 La parametrizzazione delle procedure	28
3.3.1 Modalità di associazione	28
3.3.2 Implementazione del passaggio dei parametri	28
3.3.3 Aliasing	29
3.3.4 Procedure come parametri di procedura	30
3.3.5 Macro	30
3.4 Il vettore degli ambienti non locali	32
3.4.1 Mantenimento del vettore degli ambienti non locali	32
3.4.2 Proprietà dell'implementazione mediante vettore dell'ambiente non locale	33
4 IL PARADIGMA OBJECT ORIENTED: IL LINGUAGGIO JAVA	34
4.1 Le fasi di un programma	34
4.1.1 Fase 1: Scrittura e modifica di un programma	34
4.1.2 Fase 2: Compilazione di un programma Java in bytecode	34
4.1.3 Fase 3: Caricamento di un programma in memoria	35
4.1.4 Fase 4: Verifica del bytecode	35
4.1.5 Fase 5: Esecuzione	35

4.2 La Java Runtime Environment	36
4.2.1 Il garbage collector	36
4.3 Tipi primitivi	37
4.3.1 I tipi interi	37
4.3.2 Tipi a virgola mobile	38
4.3.3 Casting dei tipi primitivi	38
4.4 Tipi reference: le classi	40
4.4.1 Il costruttore	41
4.4.2 I metodi	42
4.4.3 Il passaggio dei parametri	45
4.4.4 Lo shadowing: la parola chiave <code>this</code>	46
4.5 Ereditarietà e polimorfismo	47
4.5.1 La keyword <code>extends</code>	47
4.5.2 La keyword <code>super</code>	47
4.5.3 Il polimorfismo in Java	49
4.5.4 L'operatore <code>instanceof</code>	49
4.5.5 La keyword <code>super</code>	50
4.5.6 Overloading	51
4.5.7 Override	53
4.5.8 Casting di tipi reference	55
4.6 File Java, package e cartelle	57
4.6.1 File sorgenti	57
4.6.2 I package	57
4.6.3 Cartelle	58
4.7 Gli operatori	59
4.7.1 Operatori logici	59
4.7.2 Operatori bitwise	59
4.8 Enunciati Java	61
4.8.1 Enunciati branch	61
4.8.2 Enunciati loop	62
4.9 Gli array in Java	63
4.9.1 Dichiarazione di un array monodimensionale	63
4.9.2 Inizializzazione degli array	64
4.9.3 Array multidimensionali	65
4.9.4 La classe <code>Arrays</code>	65
4.9.5 Assegnazione	66
4.9.6 Copia di array: il metodo <code>arraycopy()</code>	66
4.10 La costruzione degli oggetti	66
4.11 La classe <code>Object</code>	68
4.11.1 Il metodo <code>equals</code>	68
4.11.2 La classe <code>String</code>	68
4.11.3 Le classi <code>Wrapper</code>	69
4.12 I modificatori	70
4.12.1 I modificatori di accesso	70
4.12.2 Il modificatore <code>final</code>	70
4.12.3 Il modificatore <code>abstract</code>	71
4.12.4 Il modificatore <code>static</code>	71
4.12.5 I modificatori <code>native</code> , <code>transient</code> , <code>synchronized</code> e <code>volatile</code>	72
4.13 Le classi astratte e interfacce	72
4.13.1 Le interfacce: la keyword <code>interface</code>	73
4.13.2 Vantaggi delle interfacce	74
4.14 Le eccezioni	74
4.14.1 Cosa sono le eccezioni?	75
4.14.2 Il costrutto <code>try...catch</code>	76
4.14.3 Sollevamento delle eccezioni	77
4.14.4 La cattura delle eccezioni	78
4.15 Le classi interne	78
4.15.1 Le classi membro	78
4.15.2 Classi membro statiche	80
4.15.3 Classi locali	81
4.15.4 Classi anonime	81
 5 IL PARADIGMA FUNZIONALE: IL LINGUAGGIO ML	 83
5.1 Il paradigma funzionale	83
5.2 Il sistema dei tipi	84

5.3 Dichiarazioni e scoping in ML	85
5.3.1 Dichiarazione di identificatori	85
5.3.2 Dichiarare una funzione	85
5.3.3 Blocchi e scope locale	86
5.3.4 Dichiarazioni locali	87
5.4 Tipi strutturati in ML	88
5.4.1 Prodotti cartesiani	88
5.4.2 Record	88
5.4.3 Sinonimi di tipo	89
5.4.4 Datatypes e costruttori	89
5.5 Pattern Matching	91
5.6 Liste	92
5.6.1 Principali operatori sulle liste in ML	93
5.7 Currying	93
5.8 Funzioni di ordine superiore	94
5.8.1 La funzione Filter	95
5.8.2 La funzione Map	95
5.8.3 La funzione Reduce	95
5.8.4 Funzioni anonime	96
5.8.5 Ancora sul currying	96
5.9 Polimorfismo parametrico	97
5.9.1 Tipi parametrici	97
5.10 Encapsulation e interfacce	98
5.10.1 Signatures	98
5.10.2 Structures	98
5.10.3 Functors	99
5.11 Eccezioni e integrazione con type checking	99
5.11.1 Eccezioni di default e meccanismo	99
5.11.2 Eccezioni personalizzate	100
5.11.3 La gestione delle eccezioni	100
6 IL PARADIGMA LOGICO: IL LINGUAGGIO PROLOG	101
6.1 Introduzione	101
6.2 Costrutti di base	101
6.2.1 Facts	102
6.2.2 Queries	102
6.2.3 Variabili logiche nelle query	102
6.2.4 Termini	103
6.2.5 Gli alberi	103
6.2.6 Le regole	105
6.3 Struttura di un programma Prolog	105
6.3.1 Query semplici	105
6.3.2 Sostituzione	106
6.3.3 Istanze	106
6.3.4 Vincoli sugli alberi	107
6.3.5 L'unificazione	108
6.3.6 Overloading e Wildcards	110
6.4 Liste	111
6.4.1 Predicati predefiniti sulle liste	111
6.5 Calcolo simbolico in Prolog	113
6.6 Programmazione nondefinitiva	114
6.6.1 Il backtracking	114
7 ESERCIZI E APPROFONDIMENTI	115
7.1 Il paradigma imperativo	115
7.1.1 Le funzioni mem ed env	115
7.2 Il paradigma funzionale	119
7.2.1 Lo stack di attivazione	119
7.3 Esercizi linguaggio Java	141
7.4 Esercizi linguaggio ML	172
7.5 Esercizi linguaggio Prolog	173

INTRODUZIONE

ESTRATTO

Il seguente documento rappresenta una raccolta di appunti, riorganizzata e migliorata nelle vesti grafiche, del corso di Linguaggi di Programmazione I del docente Piero Andrea Bonatti, per il corso di laurea in Informatica A.A 2022-2023, della Federico II di Napoli.

Si tiene a specificare che questo documento non ha lo scopo di **sostituire libri di testo**, note del professore o lezioni frontali, nonostante siano tutte fonti utilizzate per la stesura, ma quello di integrare al materiale più complesso una rilettura degli argomenti da un punto di vista meno tecnico e quindi più lento nel soffermarsi nei passaggi critici.

Si ringraziano  **Valentino Bocchetti** e  **Pasquale Miranda** ideatori e gestori del progetto *Unina Docs*,

Si ringraziano inoltre tutti i revisori che hanno contribuito al documento con consigli e proposte di modifica, invitando a dare feedback in caso di errori.

Si ringraziano in fine i lettori nella speranza che questo lavoro sia stato utile.

Redazione a cura di  **Giorgio Di Fusco**

Revisione testo a cura di  **Valentino Bocchetti**

INTRODUZIONE AI LINGUAGGI DI PROGRAMMAZIONE

1.1

COS'È UN LINGUAGGIO DI PROGRAMMAZIONE



Un **linguaggio di programmazione** è un insieme di regole per comunicare idee. Con i linguaggi naturali, come l'inglese o l'italiano, questa comunicazione tra esseri umani e può essere svolta sia in maniera orale che in maniera scritta. I linguaggi di programmazione differiscono da quelli naturali in quanto la comunicazione avviene tra un *essere umano* e un *calcolatore*. La seconda differenza principale è il *contenuto* della comunicazione: i **programmi**. I programmi sono un modo di esprimere la soluzione per un determinato problema. La terza differenza sta nel mezzo attraverso il quale avviene la comunicazione tra il programmatore e il calcolatore. Dato che un computer è solitamente il destinatario della comunicazione, i programmi devono essere rappresentati obbligatoriamente come stringhe di caratteri.

Linguaggio di programmazione

Un linguaggio di programmazione è un linguaggio che deve essere usato da una persona per esprimere un processo attraverso il quale un calcolatore può risolvere un problema

I quattro termini chiave presenti in questa definizione di linguaggio di programmazione sono i seguenti:

1. **Processore:** è la macchina che eseguirà il processo descritto dal programma; non si deve intendere come un singolo oggetto, ma come una *architettura di elaborazione*.
2. **Programma:** è l'espressione codificata di un processo.
3. **Persona:** il programmatore che desidera comunicare col calcolatore.
4. **Problema:** il sistema o ambiente che il processo deve modellare.

È uso comune intendere come linguaggi di programmazione solo quelli **computazionalmente completi**, cioè quelli che possono programmare qualsiasi *funzione calcolabile*. Questi vengono anche detti **general purpose** in quanto sono in grado di poter simulare qualunque *macchina di Turing*. Sono linguaggi completi solo quelli che riescono ad esprimere anche programmi di cui non è decidibile la terminazione.

Osservazione



Il linguaggio SQL puro non è un linguaggio completo poiché la terminazione dei programmi è sempre decidibile. Analogamente, il linguaggio HTML non è un linguaggio computazionalmente completo in quanto manca della capacità di eseguire calcoli o implementare logica condizionale.



Macchina astratta

Dato un linguaggio di programmazione L , una macchina astratta per L (in simboli M_L) è un qualsiasi insieme di strutture dati e algoritmi che permettono di memorizzare ed eseguire i programmi scritti in L .

Le componenti chiave di una macchina astratta sono:

1. La **memoria**: rappresenta lo spazio in cui vengono memorizzati i dati, le variabili, i programmi e altri elementi necessari per l'esecuzione del software. Questa memoria può essere suddivisa in diverse aree, come memoria volatile (RAM) e memoria non volatile (disco rigido), e può includere variabili, costanti, strutture dati e altro ancora.
2. Il **processore**: rappresenta l'elemento di calcolo che esegue le istruzioni del programma. Le istruzioni vengono lette dalla memoria e interpretate o eseguite dal processore. Il processore può includere registri, una ALU (unità aritmetica e logica), un'unità di controllo e altre componenti che gestiscono il flusso di esecuzione del programma.

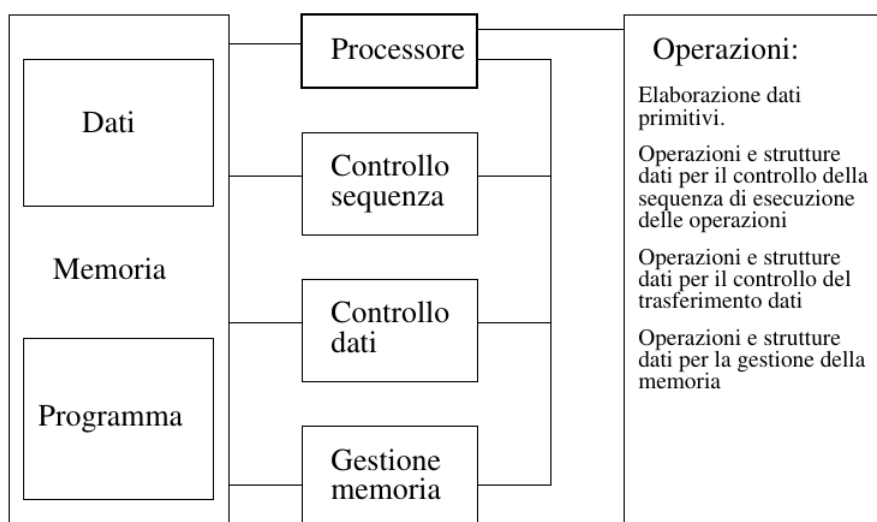


Figura 1.1: Struttura di una macchina astratta

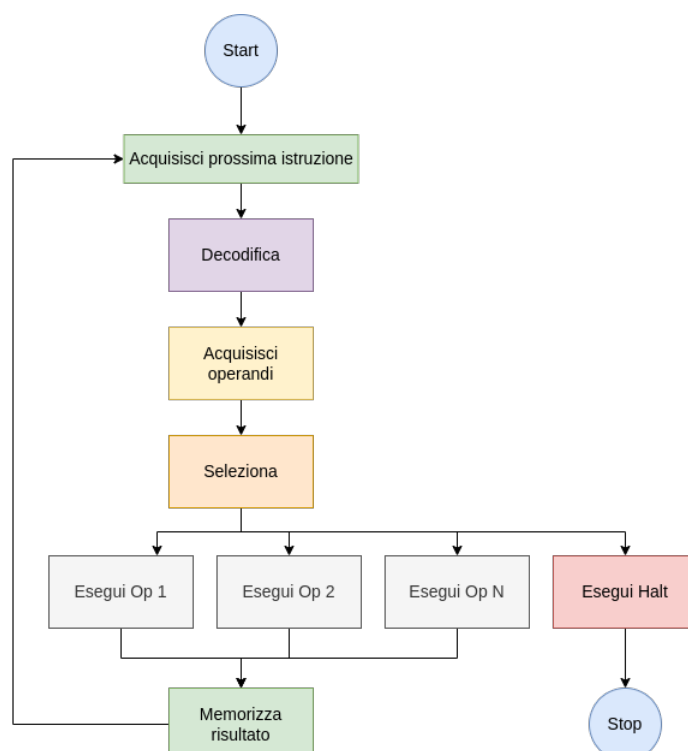


Figura 1.2: Ciclo di esecuzione all'interno di un processore di una macchina astratta

Una macchina astratta può essere implementata in tre modi principali: hardware, software o firmware. L'implementazione hardware coinvolge la creazione di un dispositivo fisico dedicato per eseguire programmi specifici. L'implementazione software comporta la scrittura di un programma o interprete che simula il comportamento della macchina astratta su un computer generale. L'implementazione firmware è una via intermedia, in cui il software è incorporato in un dispositivo hardware specifico e spesso controlla funzionalità hardware specifiche. La scelta tra queste opzioni dipende dalle esigenze del progetto, dalle prestazioni richieste e dalla flessibilità desiderata.

1.3

LA TRADUZIONE DEI LINGUAGGI DI PROGRAMMAZIONE



Lo scopo di ogni linguaggio è la comunicazione tra due parti, un mittente e un destinatario. Con i linguaggi naturali, la comunicazione è tra due persone le quali in maniera alternata si inviano dei messaggi scambiandosi di volta in volta i ruoli. Con i linguaggi di programmazione, la comunicazione tra il programmatore e il calcolatore viene svolta mediante un programma di traduzione del linguaggio. Lo scopo di questo programma è quello di prendere in carico un processo descritto in un determinato linguaggio di programmazione e *tradurlo* in una sequenza di byte comprensibili dalla macchina.

Esistono due approcci fondamentali per tradurre un linguaggio:

1. **Compilazione**
2. **Interpretazione**

I programmi di traduzione che seguono il primo approccio vengono chiamati **compilatori**, mentre i secondi **interpreti**. Mentre un interprete traduce ed esegue un costrutto alla volta (vedi lo schema in Figura 1.3), un compilatore prima traduce l'intero programma e poi esegue la sua versione tradotta.

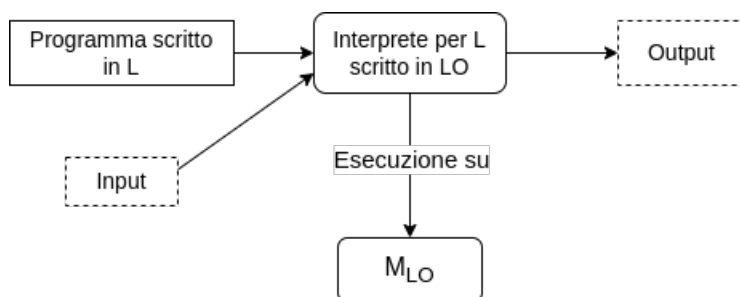


Figura 1.3: Processo di interpretazione

1.3.1 Il processo di compilazione

Lo scopo di un compilatore è quello di tradurre un programma scritto in un linguaggio di programmazione in un programma equivalente espresso in un linguaggio eseguibile direttamente dalla macchina. Questi due programmi sono chiamati rispettivamente **programma sorgente** e **programma oggetto**.

Il linguaggio del programma oggetto viene detto **linguaggio macchina**, **linguaggio oggetto** o **linguaggio target**. La Figura 1.5a mostra uno schema del processo di compilazione nel caso semplice, dove il programma oggetto è direttamente eseguibile. Il tempo durante il quale il compilatore è in esecuzione è noto come **tempo di compilazione**. Il tempo durante il quale viene invece eseguito il programma oggetto viene chiamato **tempo di esecuzione** o **run time**.

Il processo di compilazione può essere suddiviso in varie fasi. Seguendo lo schema mostrato nella Figura 1.4, il programma passa attraverso tre fasi intermedie: la **stringa di token**, il **parse tree** e il **programma astratto**.

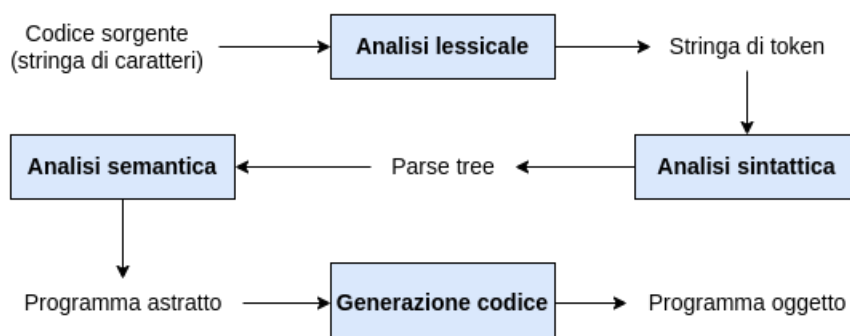
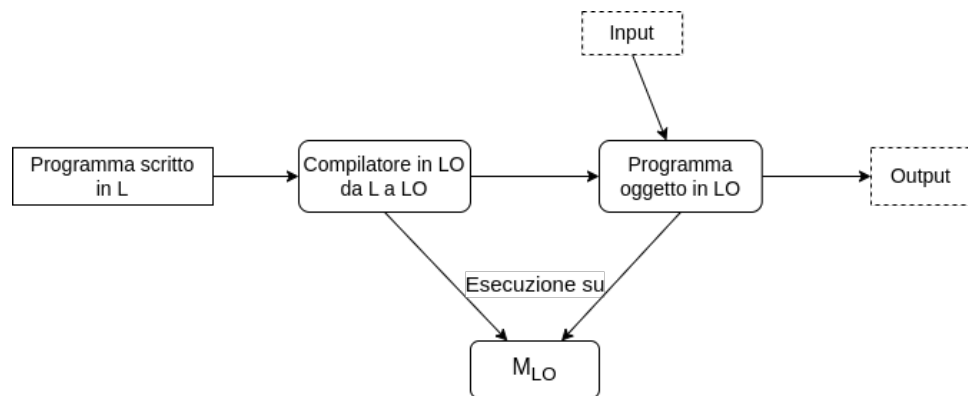
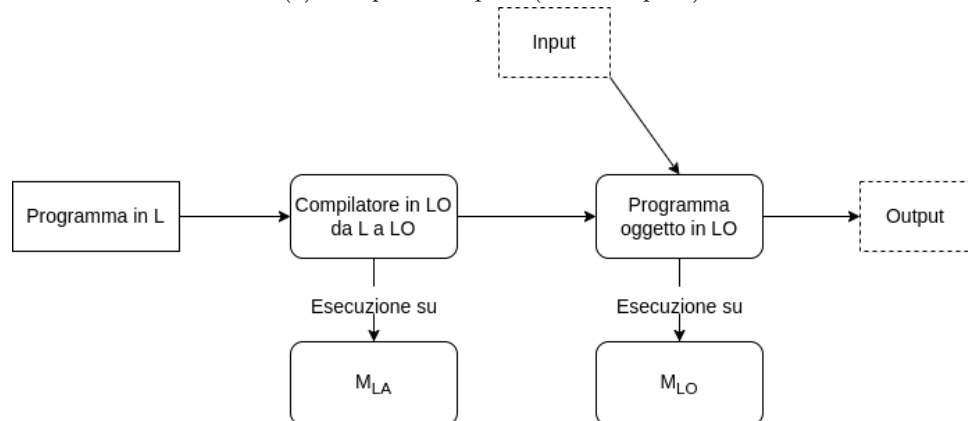


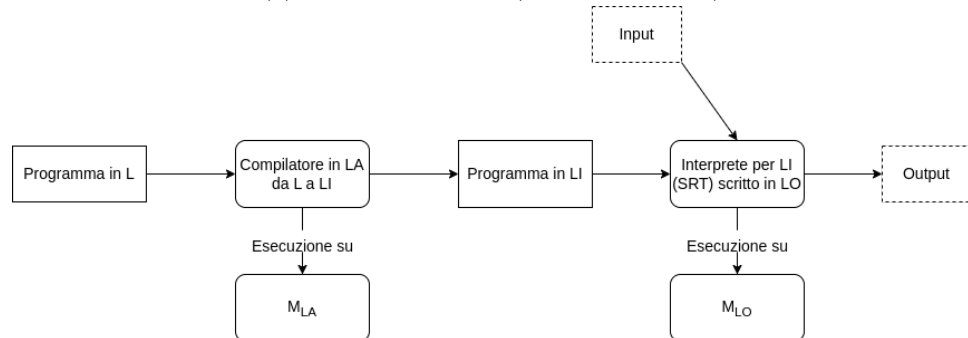
Figura 1.4: Fasi della compilazione



(a) Compilazione pura (Caso semplice)



(b) Compilazione pura (Caso più generale)



(c) Compilazione per macchina intermedia

Figura 1.5: Tipologie di compilazione



Lo scopo principale di un linguaggio di programmazione è quello di aiutare il programmatore all'interno del processo di sviluppo. Questo deve includere l'assistenza durante la fase di progettazione, implementazione, testing, verifica e mantenimento del software. Ci sono numerose proprietà di un linguaggio di programmazione che possono contribuire a questo scopo.

1.4.1 ■ Semplicità

Un linguaggio di programmazione dovrebbe sforzarsi di essere semplice da usare sia sotto il punto di vista della semantica che quello della sintassi.

- **Semplicità semantica:** numero minimo di concetti e strutture;
- **Semplicità semantica:** unica rappresentabilità di ogni concetto.

Un linguaggio troppo coinciso, però, non è detto possa essere facile da leggere e quindi da capire. Un buon linguaggio di programmazione deve quindi cercare di trovare un compromesso tra questi due poli.

1.4.2 ■ Astrazione

Un'**astrazione** è una rappresentazione di un oggetto che include solo gli attributi rilevanti dell'oggetto originale, ignorando tutti gli attributi irrilevanti. L'abilità di un linguaggio di programmazione di esprimere e usare l'astrazione è importante sia a livello della rappresentatività dei dati che a livello procedurale. Per quanto riguarda i dati, l'astrazione permette al programmatore di poter lavorare in maniera più semplice usando astrazioni più semplici che non includono dettagli irrilevanti del modello. A livello procedurale invece, l'astrazione facilita la progettazione e la modularità del codice.

1.4.3 ■ Espressività

L'**espressività** si riferisce alla facilità con cui un oggetto può essere rappresentato. In relazione ai linguaggi di programmazione, ciò significa che il linguaggio deve consentire la rappresentazione naturale sia degli oggetti dati che delle procedure. Strutture di dati e strutture di controllo adeguate ne sono un esempio adeguato. L'espressività può andare in conflitto però con la semplicità poiché maggiore il linguaggio è espressivo e maggiore sarà la complessità del linguaggio.

1.4.4 ■ Ortogonalità

La semplicità richiede che un linguaggio incorpori il minor numero possibile di concetti. L'espressività richiede che i concetti corrispondano strettamente agli oggetti che rappresentano, mentre l'astrazione elimina i dettagli irrilevanti. L'**ortogonalità** si riferisce all'interazione tra i concetti, vale a dire al grado in cui concetti diversi possono essere combinati tra loro in modo coerente. Le violazioni dell'ortogonalità si verificano quando due concetti non possono interagire tra di loro o quando interagiscono in modo non coerente con le loro.

1.4.5 ■ Portabilità

La possibilità di mantenere dei programmi è influenzata da tutte le quattro proprietà precedenti, nel senso che tutte favoriscono la facilità di comprensione e di modifica dei programmi. Un problema di manutenzione importante che queste caratteristiche non affrontano è lo spostamento di un programma da un computer ad un altro. La **portabilità** di un linguaggio è molto maggiore quando esiste uno standard indipendente dalla macchina per quel linguaggio.



Un **paradigma di programmazione** è un modello che permette di descrivere astrattamente l'algoritmo (cioè il metodo di soluzione di un problema).

I principali paradigmi di programmazione sono:

- **Imperativo:** basato sul punto di vista del calcolatore. Ciò si riflette nell'*esecuzione sequenziale* dei comandi e nell'uso di una memoria modificabile, concetti che si basano sul modo in cui i computer eseguono i programmi a livello di linguaggio macchina. Il paradigma imperativo è stato il paradigma predominante per i linguaggi, in quanto tali linguaggi sono più facili da trasporre in una forma adatta all'esecuzione meccanica. Il programma di questo modello consiste in una *sequenza di modifiche* alla memoria del computer.

- **Funzionale:** il modello funzionale si concentra sul processo di risoluzione del problema. Il punto di vista funzionale si traduce in programmi che descrivono le operazioni che devono essere eseguite per risolvere il problema.
- **Logico:** Il modello orientato alla logica è più strettamente legato alla prospettiva della persona. Il programma è una descrizione logica del problema espressa in modo formale, simile al modo in cui un cervello umano ragionerebbe sul problema.
- **Orientato agli oggetti:** Il modello orientato agli oggetti riflette più fedelmente il problema reale. Un programma in questo modello consiste in oggetti che si inviano messaggi l'un l'altro. Gli oggetti del programma corrispondono direttamente a oggetti reali, come persone, macchine, reparti, documenti e così via.
- **Parallelo:** programmi che descrivono entità distribuite che sono eseguite contemporaneamente ed in modo asincrono.

Gli ultimi due paradigmi sono *ortogonali* rispetto ai primi tre.

Esempio

Il listato 1.1 mostra un esempio di definizione di funzione che calcola il fattoriale di un intero n . Sfruttando la possibilità di poter accedere direttamente alla memoria, modificandone il valore, i programmi imperativi sono caratterizzati dal forte uso delle **assegnazioni** e dell'approccio iterativo. Al contrario, un programma scritto secondo il paradigma funzionale non può eseguire assegnazioni in memoria. Il programma nel Listato 1.2 definisce la stessa funzione per il calcolo del fattoriale senza eseguire assegnazioni.

```
int factI(int n)
{
    int result = 1;
    for(int i=1, i≤n, i++)
        result = result*i;
    return result;
}
```

Codice 1.1: Paradigma imperativo

```
int factR(int n)
{
    if(n==1)
        return 1;
    else
        return n*factR(n-1);
}
```

Codice 1.2: Paradigma funzionale

IL PARADIGMA IMPERATIVO

2.1

PERCHÉ IMPERATIVO?



Il **modello imperativo** fu creato per imitare, il più fedelmente possibile, le azioni che vengono eseguite dal calcolatore a livello del linguaggio macchina. A quel livello, i calcolatori operano con due unità principalmente:

- La **CPU**, dove vengono eseguiti tutti i calcoli;
- La **memoria**, dove i dati vengono conservati. Questa consiste in un insieme di “contenitori di dati” come le parole della memoria centrale. Queste sono tipicamente rappresentate dal loro **indirizzo**. A ciascun indirizzo viene associato successivamente un **valore**. Concettualmente la memoria è una funzione da uno *spazio di locazioni* ad uno *spazio di valori*:

$$mem : Locazioni \longrightarrow Valori \quad (2.1)$$

La tipica **unità di esecuzione** nel linguaggio macchina consiste nei seguenti quattro passi:

1. Recupero dell'indirizzo della locazione per il risultato e dei vari operandi
2. Recupero dei valori dalle locazioni degli operandi;
3. Calcolo dell'espressione;
4. Memorizzazione del risultato nella sua locazione.

Al centro di questi passi sta il concetto di **assegnazione**. Il modello imperativo usa dei nomi, le **variabili**, come astrazione degli indirizzi di locazioni di memoria. La forma BNF¹ di questo tipo istruzione è la seguente:

`<assegnazione> ::= <name> <assignment-operator> <expression>`

dove `<name>` rappresenta il nome della variabile, ovvero un segnaposto per la locazione dove viene memorizzato il risultato dell'assegnazione mentre in `<expression>` sono specificati una computazione e i riferimenti ai valori necessari a tale computo.

Esempio

Si consideri il seguente listato:

```
1 a := b+c;
```

Codice 2.1: Esempio di assegnazione in Pascal

In questo caso l'assegnazione prevede la somma dei valori presenti nelle variabili `b` e `c` e infine il salvataggio di tale espressione nella cella di memoria indicata dalla variabile `a`.

¹La **BNF** (Backus-Naur Form o Backus Normal Form) è una metasintassi, ovvero un formalismo attraverso cui è possibile descrivere la sintassi di linguaggi formali (il prefisso meta ha proprio a che vedere con la natura circolare di questa definizione). Si tratta di uno strumento molto usato per descrivere in modo preciso e non ambiguo la sintassi dei linguaggi di programmazione, dei protocolli di rete e così via, benché non manchino in letteratura esempi di sue applicazioni a contesti anche non informatici e addirittura non tecnologici. La BNF viene usata nella maggior parte dei testi sulla teoria dei linguaggi di programmazione e in molti testi introduttivi su specifici linguaggi.



Ambiente di esecuzione

Si definisce **ambiente** l'insieme di nomi di variabili (non gli indirizzi, piuttosto i loro identificatori) e parametri associati a qualcosa dal quale si può risalire al *valore* (all'interno del programma) della variabile o del parametro.

Concettualmente l'ambiente è una funzione il cui dominio è dato dall'*insieme di identificatori* (i nomi) mentre il codominio dipende dal paradigma computazionale del linguaggio. Nel paradigma imperativo, la funzione *env* associa gli identificatori a locazioni di memoria, le quali, a loro volta, sono associate mediante la funzione *mem* al contenuto di memoria:

$$env : Identificatori \longrightarrow Locazioni \quad (2.2)$$

Di conseguenza il **valore** di una variabile x sarà data da:

$$mem(env(x))$$

Ovvero, data una variabile x , ne recuperiamo la locazione ad essa associata e di questa locazione ne leggiamo il contenuto in memoria.

Esempio

Si consideri la seguente assegnazione:

```
1 x := x+1;
```



La variabile che compare a sinistra dell'assegnazione indica la locazione dove salvare il risultato, ovvero l'ambiente del nome x :

$$env(x)$$

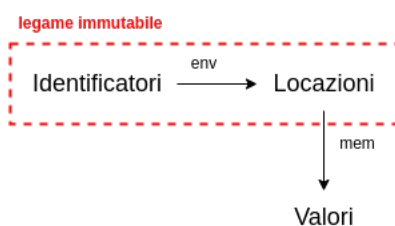
A destra dell'espressione, invece, il nome della variabile x indica il valore da utilizzare all'interno dell'espressione che verrà poi memorizzata:

$$mem(env(x)) + 1$$

○ Nel paradigma funzionale non esiste la funzione *mem* e la funzione *env* ha come codominio direttamente l'insieme dei valori. Qualsiasi sia il paradigma computazionale, però, il valore $env(x)$ è **immutabile** finché esiste l'identificatore x . ○

Paradigma imperativo

Nel paradigma imperativo l'ambiente mappa gli identificatori sulla loro locazione di memoria. Sarà poi attraverso la funzione *memory* che si potrà accedere al valore in essa conservato.



Paradigma funzionale

Nel paradigma funzionale gli environment mappano gli identificatori direttamente sul loro valore invece di una locazione di memoria (il cui contenuto può cambiare).

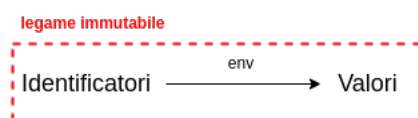


Figura 2.1: Confronto tra paradigma imperativo e paradigma funzionale

Nel paradigma imperativo, quindi, la locazione di memoria associata ad un nome non può cambiare all'interno di un blocco di esecuzione. Anche nel paradigma funzionale puro l'associazione *identificatore-valore* non cambia mai.

Esempio

Si consideri la seguente porzione di codice:

```
1 int *x,y;  
2 x=&y;
```



In questo caso abbiamo un puntatore x e una variabile di tipo intero y . Effettuare l'assegnamento $x=\&y$ significa far puntare da x la cella della variabile y , sarà allora:

$$mem(env(x)) = env(y)$$

Non è possibile fare assegnazioni come mostrato di seguito:

```
1 int x,y;  
2 &y = x;
```



Un programma del genere infatti genererebbe un errore simile:

error: lvalue required as left operand of assignment

A sinistra dell'assegnazione infatti abbiamo una espressione del tipo $env(env(y))$ che è chiaramente malformata.

Per non incorrere in errori simili ricordiamo che a sinistra di una assegnazione si possono trovare solo i nomi di variabili oppure, nel caso di puntatori, espressioni del tipo:

***(<pointer expression>)**

Ovvero il valore (l'indirizzo) salvato all'interno di un puntatore. Queste espressioni prendono il nome di **lvalue**. Reciprocamente, le espressioni che si trovano a destra di una assegnazione prendono il nome di **rvalue**. Come esempio, consideriamo l'assegnazione:

```
1 *(x+2)= ...
```



che avrà come traduzione una formula del tipo:

$$mem(env(x)) + 2 = \dots$$

Consideriamo il caso:

```
1 int *x,y;  
2 y=*x;
```



Si ha in questo caso: $env(y) = mem(mem(env(x)))$. Nel caso invece si avesse avuto l'assegnazione:

```
1 *x=y;
```



si sarebbe avuto: $mem(env(x)) = mem(env(y))$. Infatti, in C, l'operatore di dereferenziazione indica la cella di memoria il cui indirizzo è memorizzata nella variabile x . Si consideri la porzione di codice:

```
1 int x,y;  
2 x = &y;
```



In questo caso il compilatore segnala un warning del tipo:

warning: assignment to 'int' from 'int*' makes integer from pointer without a cast.

In quanto si sta tentando di assegnare un indirizzo ($\&y = env(y)$) ad una variabile di tipo intero.

Tabella 2.1: "In un vettore possiamo usare sia la notazione indicizzata sia l'aritmetica dei puntatori. Si consideri un vettore di interi dichiarato come `int a[10]`. Nonostante si possano usare entrambe le notazioni bisogna ricordare che il nome di un vettore non è un puntatore. Il nome di un vettore identifica infatti l'indirizzo (l'ambiente) della prima cella dell'array."

Descrizione	Notazione indicizzata	Notazione puntatori	Espressione lvalue	Espressione rvalue
Valore del primo elemento dell'array	<code>a[0]</code>	<code>*a</code>	<i>env(a)</i>	<i>mem(env(a))</i>
Indirizzo del primo elemento dell'array	<code>&a[0]</code>	<code>a</code>	Not an lvalue	<i>env(a)</i>
Valore dell' <i>i</i> -esimo elemento dell'array	<code>a[i]</code>	<code>*(a+i)</code>	<i>env(a) + mem(env(i))</i>	<i>mem(env(a) + mem(env(i)))</i>
Indirizzo dell' <i>i</i> -esimo elemento dell'array	<code>&a[i]</code>	<code>a+i</code>	Not an lvalue	<i>env(a) + mem(env(i))</i>

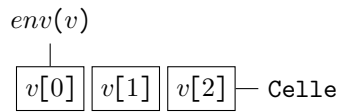
Esempio (Array e puntatori)

Supponiamo di avere un array di interi:

```
1 int v[3];
```



In questo caso l'espressione $env(v)$ indica la prima cella del vettore v :



Si consideri la seguente porzione di codice:

```
1 v = v+1;
```



In casi come questi il compilatore restituisce un errore del tipo:

error: assignment to expression with array type

Questo perché si sta provando a modificare il valore dell'ambiente della variabile v . Consideriamo l'assegnamento:

```
1 y=v[1];
```



In questo caso avremo a sinistra $env(y)$ mentre a destra sarà necessario accedere alla seconda cella di memoria sommando un offset di una posizione all'indirizzo salvato in v e successivamente leggere il contenuto di tale cella: $mem(env(v) + 1)$.

Sia ora:

```
1 v[1]=y;
```



In questo caso si ha a sinistra l'espressione $env(v) + 1$, che esprime l'indirizzo della seconda cella di memoria, mentre a destra si avrà $mem(env(y))$.

Sia x un puntatore e consideriamo l'assegnamento:

```
1 y=x[1]+v[1];
```



In questo caso si vuole salvare all'interno della variabile y il valore ottenuto dalla somma tra il secondo elemento dell'array puntato da x e il secondo elemento dell'array v . Si avrà quindi in questo caso:

$$env(y) = mem(mem(env(x)) + 1) + mem(env(v) + 1)$$

Da notare qui il differente uso delle funzioni mem ed env nel caso in cui queste vengano applicate ad un puntatore o direttamente ad un vettore. Infatti, nel caso di un puntatore sarà necessario prima leggere la cella di memoria del puntatore per risalire all'indirizzo della prima cella del vettore puntato, sommare l'offset e infine leggere il valore della cella così ottenuto. Nel caso del vettore, invece, è sufficiente sommare l'offset direttamente alla quantità $env(v)$, che contiene già l'indirizzo della prima cella del vettore, e infine applicare la funzione mem .

Consideriamo il caso in cui si voglia usare un puntatore a sinistra di un assegnamento:

```
1 x[1]=y;
```



Si ha in questo caso si vuole assegnare il valore della variabile y all'interno della seconda cella dell'array puntato da x . Si ha quindi:

$$mem(env(x)) + 1 = mem(env(y))$$

Consideriamo ora il caso dell'assegnamento:

```
1 y=*x*v;
```



In questo caso si vuole salvare in y il valore della cella puntata da x e del primo elemento del vettore v . Infatti, in un vettore, sono simili le scritture $v[0]$ e $*v$ in quanto entrambe fanno riferimento al valore della prima cella (si osservi la Tabella 2.1). Si ha quindi:

$$env(y) = mem(mem(env(x))) + mem(env(y))$$

Mentre, avessimo avuto:

```
1 *v=y;
```




Ogni variabile, parametro od oggetto può essere rappresentato mediante un **data object**.

Data Object

Un **data object** è una quadrupla (L, N, V, T) , dove:

- L: locazione;
- N: nome;
- V: valore;
- T: tipo.

Binding

Un **legame** (*binding*) è la determinazione di una delle componenti di un data object. Si distinguono quindi in legami di locazione, di nome, di valore e di tipo.

Osservazione

Nel paradigma imperativo il legame di locazione corrisponde alla funzione *env* mentre risalire al legame di valore di una variabile di nome *x* corrisponde al calcolo della funzione *mem(env(x))*.

La Figura 2.2 mostra una visualizzazione di un data object e dei suoi legami. Le quattro linee si dipartono dal data object fino ai relativi spazi di arrivo. Lo **spazio di memoria** dal quale sono selezionati i legami di locazione è l'insieme delle locazioni di memoria virtuali disponibili nel momento in cui il programma è in esecuzione.

Il momento in cui questi legami possono avvenire rappresenta un'informazione importante nel momento in cui si parla di linguaggi di programmazione:

1. **Tempo di compilazione:** quando il programma viene tradotto in linguaggio macchina;
2. **Tempo di caricamento:** quando il programma in linguaggio macchina viene assegnato a specifiche locazioni di memoria all'interno della memoria;
3. **Tempo di esecuzione:** quando il programma viene eseguito.

Il *location binding* avviene durante il caricamento in memoria, oppure a run time. Il *name binding* avviene durante la compilazione, nell'istante in cui il compilatore incontra una dichiarazione. Il *type binding* avviene di solito durante la compilazione, nell'istante in cui il compilatore incontra una dichiarazione di tipo. Un tipo è definito dal sottospazio di valori (e dai relativi operatori) che un data object può assumere. Quando i legami avvengono a tempo di compilazione (immutabili) si è soliti segnalarlo **evidenziando in grassetto la linea** mentre i legami che possono essere modificati a tempo di esecuzione sono descritti da linee sottili.

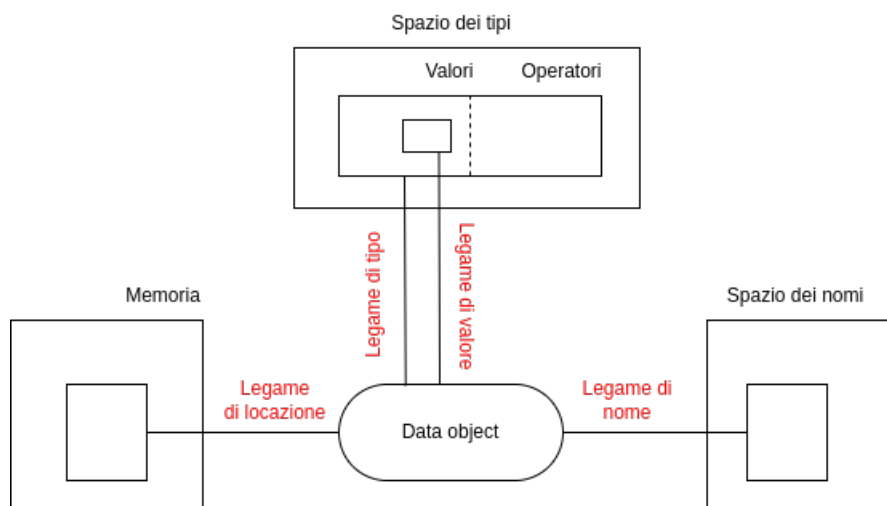


Figura 2.2: Visualizzazione concettuale di un data object

Esempio

Si consideri la seguente assegnazione:

```
1 A: integer;
```



il data object A avrà una raffigurazione come mostrato nella Figura 2.3. Inizializzando la variabile si avrà un'assegnazione di tipo a tempo di compilazione mentre il legame di valore avviene a tempo di load time:

```
1 B: integer := 1;
```



il data object B avrà una rappresentazione come mostrato in Figura 2.4. Solo dichiarando una costante si può avere un legame di tipo e valore a tempo di compilazione (si veda la Figura 2.5):

```
1 C: constant integer := 1;
```

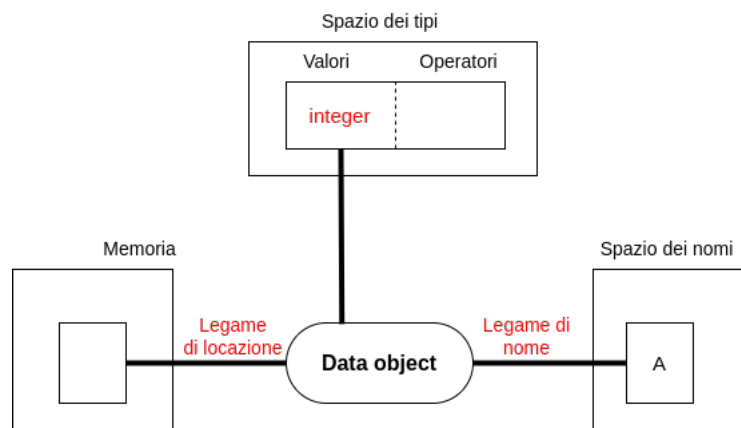


Figura 2.3

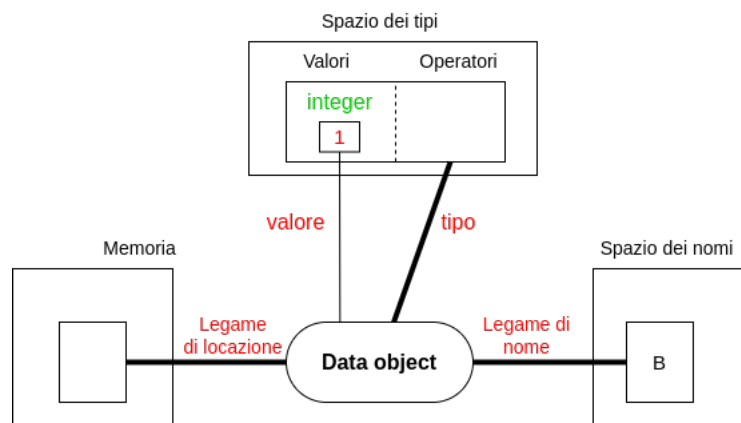


Figura 2.4

2.3.1 I puntatori

Il data object dei puntatori si differenzia da quelli per gli altri tipo di dato (che hanno un nome, un valore e una singola locazione) per il fatto di *coinvolgere ben due data object*. Infatti, il valore di un data object di tipo puntatore è esso stesso la locazione di un altro data object il quale può non avere un legame di nome o di valore (come nel caso di allocazione dinamica mediante `malloc` in C). La definizione di una variabile puntatore è mostrata nella Figura 2.6.

I due data object coinvolti sono quindi:

1. Il data object del puntatore (Data Object 1);
2. Il data object della variabile puntata o dello spazio di memoria puntato (Data Object 2).

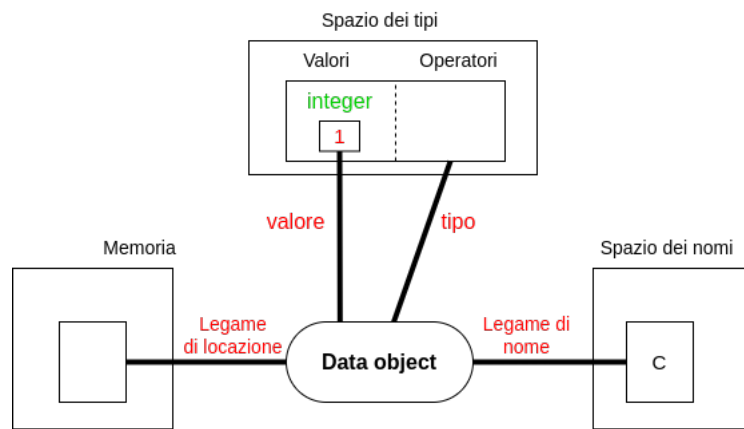


Figura 2.5

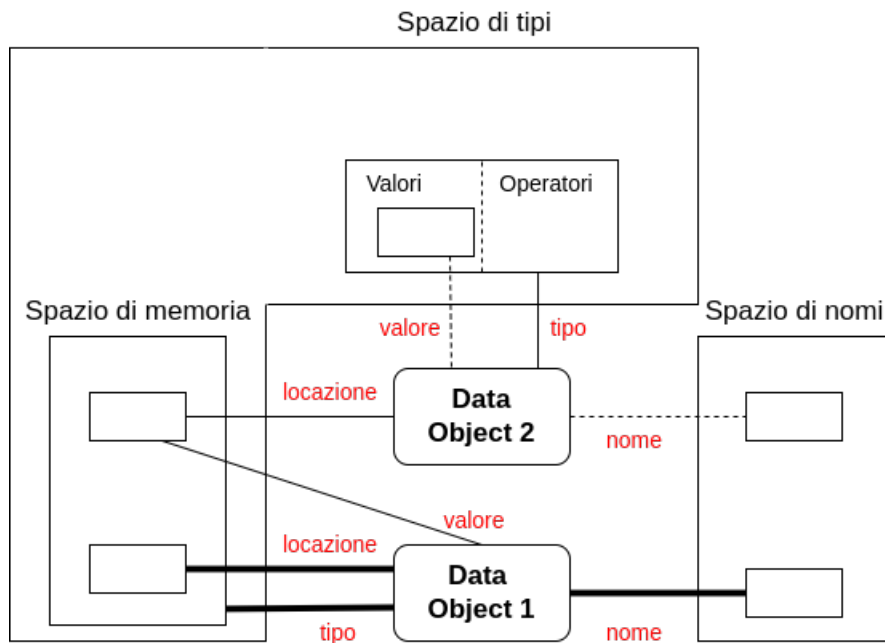


Figura 2.6: Legami di un tipo di dato puntatore

Nel caso del puntatore, il data object è legato a un *nome* e ad una *locazione* come per gli altri tipi. Il *valore* legato ad un data object di tipo puntatore, tuttavia, è un elemento dello spazio di memoria. Come per gli altri tipi, i *legami di tipo* e di *nome* si verificano al momento della compilazione, il *legame di locazione* si verifica al momento del caricamento mentre il *legame di valore* in tempo di esecuzione.

L'oggetto puntato (Data Object 2) è molto diverso da quelli che abbiamo visto in precedenza. Innanzitutto, notiamo che *non ha alcun legame con lo spazio dei nomi*. Infatti, non ci si riferisce ad esso mediante un suo identificatore bensì dal nome del puntatore legato ad esso. In secondo luogo, i legami di posizione e di tipo di questo oggetto *devono avvenire a tempo di esecuzione*, dal momento che il data object stesso non esiste fino a quando non viene assegnato al puntatore come valore. In questo caso è necessaria la **deallocazione** della memoria in quanto la modifica del legame di valore genera di solito dati non più accessibili per nome o riferimento. Alcuni linguaggi possiedono meccanismi di recupero automatico di memoria come il **garbage collector**.

2.3.2 Il legame di tipo

Sebbene i tipi siano legati ai data object in fase di compilazione nella maggior parte dei linguaggi, spesso è possibile che questo legame avvenga anche a tempo di esecuzione. I linguaggi APL e Smalltalk, ad esempio, implementano questo tipo di legame dinamico. Il tipo di un data object è determinato quindi dal tipo del suo valore. Pertanto, ogni volta che un data object viene legato a un valore diverso, assume il tipo del nuovo valore ad esso associato. Considerando che le dichiarazioni sono usate per legare i data objects ai tipi in fase di compilazione, i linguaggi a tipizzazione dinamica non hanno bisogno di dichiarazioni.

Un linguaggio è detto **dinamicamente tipizzato** se il legame (e le variazioni di legame) e di conseguenza anche il controllo di consistenza (se avviene) avvengono durante il tempo di esecuzione. Un linguaggio invece è detto **staticamente tipizzato** se il legame avviene durante la compilazione. In questo caso il controllo di consistenza (se avviene) può avvenire in entrambe le fasi.

Type checking Il **type checking** è processo mediante il quale si determina il tipo di un data object controllando la consistenza della coppia valore-tipo. L'esecuzione di questo tipo di controllo da parte del calcolatore può essere di grande aiuto nel rilevamento e nella prevenzione degli errori.

Il type checking può avvenire a tempo di compilazione, di esecuzione, o non avvenire per nulla. I linguaggi che eseguono il controllo del tipo a tempo di compilazione vengono chiamati **fortemente tipati**. Il linguaggio Java è un esempio di linguaggio fortemente tipizzato. Il linguaggio Pascal è quasi fortemente tipizzato (una sola eccezione dei record di assenza di controllo: record con varianti).

Un linguaggio è detto **debolmente tipizzato** se il controllo di consistenza non avviene sempre. Il linguaggio C ne è un esempio. Infatti in esso esiste la possibilità di eseguire operazioni di **casting**, che consentono di forzare, in esecuzione, l'interpretazione di un qualunque valore secondo un qualunque tipo (anche un tipo diverso da quello a cui il valore è stato precedentemente associato).

Type equivalence All'inizio dell'informatica esistevano solo i tipi elementari. Successivamente sono stati introdotti i tipi definiti dall'utente i quali erano semplici *ridenominazioni* di tipi elementari oppure nomi di record.

A stabilire se un assegnamento del tipo $x=y$ fosse valido stava alla base il concetto di **type equivalence** adottata dal linguaggio. Esistono due forme di equivalenza:

1. **Name equivalence:** i tipi di x e y devono avere lo stesso nome, ovvero lo stesso tipo;
2. **Structural equivalence:** i tipi di x e y devono avere la stessa rappresentazione interna. Apparentemente più snello e flessibile ma aumenta la possibilità di errori.

Esempio

Il linguaggio Pascal adotta il *name equivalence* mentre il linguaggio C/C++ entrambi i tipi (quasi sempre structural) tranne che per le struct):

```
1  typedef int money;
2  typedef int apples;
3
4  typedef struct{int a;} S1;
5  typedef struct{int a;} S2;
6
7  int main(){
8      money x = 0;
9      apples y = 0;
10     int z = x+y; // Non fa una piega
11     S1 a;
12     S2 b;
13     a = b; // Questo non lo compila
14 }
```



questo perché verificare se due struct sono strutturalmente equivalenti richiede di verificare una proprietà chiamata **bisimulazione**.

Con l'avvento dei linguaggi a oggetti il concetto di equivalenza viene rimpiazzato dal concetto di **compatibilità**. Nei linguaggi object oriented più comuni la compatibilità è basata sul nome piuttosto che la struttura. Di conseguenza due *classi* con nomi diversi sono diverse anche se hanno gli stessi attributi e gli stessi metodi. Inoltre, una classe per essere sottotipo di un'altra deve essere esplicitamente dichiarata tale mediante il costrutto **extends**.

2.4

BLOCCHI DI ISTRUZIONE



Le **unità di esecuzione** sono gruppi di istruzioni che sono raggruppate insieme per uno scopo preciso. L'unità più grande è il programma stesso, questo è poi suddiviso in **blocchi di istruzioni**. Ci sono vari motivi per avere diversi blocchi d'istruzione all'interno di un programma. Questi servono infatti a definire meglio:

1. L'ambito delle strutture di controllo;
2. L'ambito di una procedura;
3. Unità di compilazione separate;
4. L'ambito dei legami di nome.

```

1  ...
2  BLOCK A;
3    DECLARE I;
4    BEGIN A
5      ...      {I from A}
6    END A;
7  ...

```

Codice 2.2: Struttura di un blocco

2.4.1 ■ Ambito del name binding

I blocchi che definiscono l'ambito di validità di un nome contengono due parti: una sezione di dichiarazione del nome e una sezione che comprende gli enunciati sui quali ha validità il legame. Dal punto di vista sintattico, questo richiede di solito un marcatore per l'inizio della sezione di dichiarazione, un marcatore che separa la sezione di dichiarazione dalle dichiarazioni e un marcatore che indica la fine del blocco. Svilupperemo un piccolo pseudolinguaggio simile al Pascal per esprimere gli esempi in questa sezione. Questo pseudolinguaggio userà **BLOCK** per iniziare il blocco, **BEGIN** per separare le dichiarazioni dagli enunciati e **END** per indicare la fine del blocco. La struttura generale di un blocco nel nostro pseudolinguaggio sarà quindi come mostrato nel Listato 2.2. L'istruzione **DECLARE** in questo esempio viene utilizzata per legare il nome *I* a un data object all'ingresso del blocco *A*. In questo caso, il riferimento a *I* è al legame fatto nel blocco *A*.

All'interno di ogni blocco, esistono due tipi di visibilità:

- **locale**: per le variabili specificate all'interno del blocco;
- **non locale**: per le variabili specificate all'esterno del blocco.

Anche se alcuni linguaggi richiedono una dichiarazione separata dei legami che sono ereditati dal blocco corrente, è consuetudine che un blocco erediti direttamente i legami dall'ambiente che lo contiene. Questo è particolarmente utile quando si ha a che fare con blocchi annidati.

```

1  PROGRAM P;
2  DECLARE X;
3  BEGIN P
4    ...      {X from P}
5    BLOCK A;
6      DECLARE Y;
7      BEGIN A
8        ...      {X from P, Y from A}
9      BLOCK B;
10        DECLARE Z;
11        BEGIN B
12          ...      {X from P, Y from A, Z from B}
13        END B;
14        ...      {X from P, Y from A}
15      END A;
16      ...      {X from P}
17    BLOCK C;
18      DECLARE Z;
19      BEGIN C
20        ...      {X from P, Z from C}
21      END C;
22      ...      {X from P}
23  END P;

```

Codice 2.3: Blocchi annidati

Nell'esempio mostrato nel Listato 2.3 si vede come ciascun blocco erediti i legami dall'ambiente all'interno del quale è stato definito.

Questo tipo di politica di scoping viene anche chiamata **scoping lessicale** o **scoping statico** e può essere sintetizzata come segue:

1. Se un nome ha una dichiarazione all'interno del blocco, quel nome è legato all'oggetto specificato nella dichiarazione;
2. Se un nome non ha una dichiarazione all'interno di un blocco, il nome è legato allo stesso oggetto a cui era legato nel blocco che lo contiene. Se il blocco non ha un blocco contenente o il nome non è vincolato nel blocco contenente, allora il nome è svincolato nel blocco attuale.

Un'alternativa allo scoping statico è lo **scope dinamico** che assume senso maggiore quando vi sono procedure chiamanti e chiamate. In questo caso la procedura chiamata vede e usa i legami visti e usati dalla procedura chiamante.

Il mascheramento Una situazione nota come **mascheramento** avviene quando un blocco ridefinisce un nome già vincolato ad un oggetto dell'ambiente circostante. In questo caso, la dichiarazione locale sovrascrive il legame non locale rendendo il data object precedentemente dichiarato irraggiungibile nel blocco corrente.

```
1 PROGRAM P;  
2 DECLARE X,Y;  
3 BEGIN P  
4 ...           {X from P, Y from P}  
5 BLOCK A;  
6 DECLARE X,Z;  
7 BEGIN A  
8 ...           {X from A, Y from P, Z from A}  
9 END A;  
10 ...           {X from P, Y from P}  
11 END P;
```

Codice 2.4: Mascheramento

2.4.2 ■ Ambito del location binding

I legami di locazione possono essere **statici** e **dinamici**. Gli unici linguaggi dove tutti i legami di memoria venivano fatti staticamente erano quelli arcaici come il primo Fortran che non disponevano di memoria dinamica. Esempio di variabile con legame di tipo statico possono essere le variabili globali del linguaggio C.

Si dice **allocazione statica di memoria** quando le variabili conservano il proprio valore ogni volta che si rientra in un blocco. Il legame di locazione quindi viene fissato e resta costante al tempo di caricamento. Se un linguaggio prevede un legame del genere è possibile far funzionare programmi come quello mostrato nel Listato 2.5.

Si dice **allocazione dinamica di memoria** quando il legame di locazione (e anche di nome) è creato all'inizio dell'esecuzione di un blocco e viene rilasciato a fine esecuzione. Questo meccanismo è quello adottato comunemente nei linguaggi moderni e realizzato mediante i **record di attivazione** dei blocchi posti all'interno dello **stack di attivazione**. Un record di attivazione contiene tutte le informazioni sull'esecuzione del blocco necessarie per riprendere l'esecuzione dopo che essa è stata sospesa. Può contenere informazioni complesse ma per realizzare un legame dinamico di locazione in blocchi annidati è sufficiente che contenga *le locazioni dei dati locali* più un *puntatore al record di attivazione del blocco immediatamente più esterno*.

```
1 PROGRAM P;  
2 DECLARE I;  
3 BEGIN P  
4   FOR I:=1 TO 10 DO  
5     BLOCK A;  
6     DECLARE J;  
7     BEGIN A  
8     IF I=1 THEN  
9       J:=1;  
10    ELSE  
11      J:=J*I;  
12    END IF;  
13    END A;  
14  END P;
```

Codice 2.5: Allocazione statica di memoria

2.4.3 ■ Lo stack di attivazione

In ogni momento dell'esecuzione lo stack di attivazione contiene sempre e solo i record "attivi":

- Il top dello stack contiene sempre il record del blocco correntemente in esecuzione;
- Ogni volta che si entra in un blocco, il record di attivazione del blocco viene posto sullo stack (push);
- Ogni volta che si esce da un blocco, viene eliminato il record al top dello stack.

Esempio

Si consideri ad esempio il programma mostrato nel Listato 2.6. L'evoluzione dello stack di attivazione è mostrato nella Figura 2.7.

```

1  PROGRAM P;
2  DECLARE I,J;
3  BEGIN P
4    BLOCK A;
5    DECLARE I,K;
6    BEGIN A
7    BLOCK B;
8    DECLARE I,L:INTEGER;
9    BEGIN B
10   ...
11   END B;
12   ...
13   END A;
14   BLOCK C;
15   DECLARE I,N;
16   BEGIN C
17   ...
18   END C;
19   ...
20 END P;

```

Codice 2.6: Esempio stack di attivazione

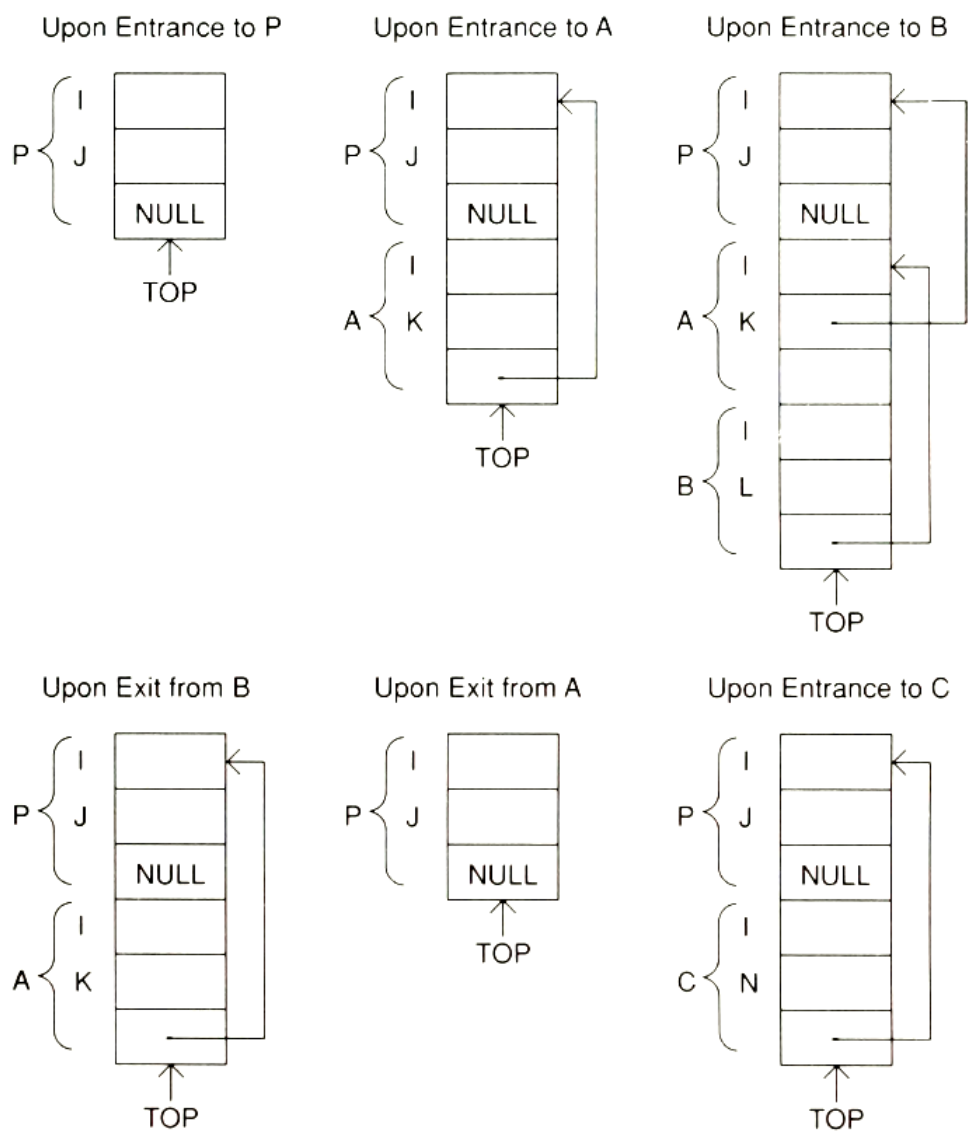


Figura 2.7: Contenuto dello stack di attivazione per il programma 2.6

ASTRAZIONE PROCEDURALE

Tra gli strumenti più potenti contenuti in un linguaggio di programmazione vi sono quelli che consentono l'**astrazione procedurale**. Un'astrazione è una rappresentazione di un oggetto che nasconde quelli che potrebbero essere considerati dettagli irrilevanti di quell'oggetto.

L'astrazione procedurale comporta la separazione dei dettagli rilevanti di una procedura da quelli irrilevanti¹. Ciò avviene in due modi: attraverso l'*astrazione parametrica* e l'*astrazione delle specifiche*.

3.1

LE PROCEDURE COME ASTRAZIONI



Una **procedura** è un'astrazione di parti di programma in unità di esecuzione più piccole, come enunciati o espressioni, in modo da nascondere i dettagli irrilevanti ai fini del loro riuso. Astrazioni del genere sono utili in quanto:

- **Rendono i programmi più semplici da scrivere, leggere o modificare:** Nascondendo i livelli di dettaglio inferiori, un'unità può concentrarsi su un singolo compito. Questa proprietà è importante per l'implementazione della progettazione top-down.
- **Le unità di programma sono indipendenti:** l'astrazione permette che le azioni di una procedura siano indipendenti dal suo utilizzo. Pertanto, il programma che utilizza l'astrazione non è influenzato dai dettagli dell'implementazione dell'astrazione.
- **Le unità di programma sono riutilizzabili:** Una procedura, una volta definita, può essere utilizzata all'interno di diversi ambienti di programmazione. Questo elimina ridondanti sforzi di programmazione e riduce gli errori.

La dichiarazione di una procedura specifica tutti i legami che devono avvenire a tempo di compilazione. Il processo è analogo a quanto avviene per una dichiarazione di tipo con l'unica differenza che gli unici legami specificati sono il *nome della procedura* ed il *blocco eseguibile*.

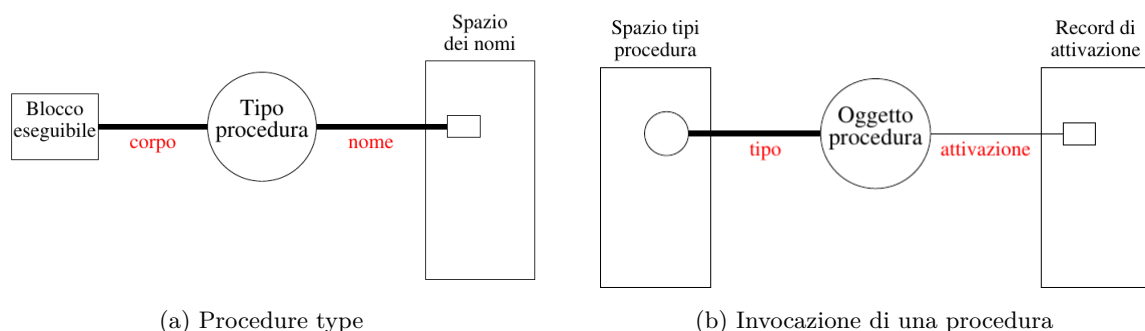


Figura 3.1: Dichiarazione ed invocazione di una procedura

L'invocazione di una procedura invece causa la generazione di un oggetto analogo al Data Object. Il processo avviene durante l'esecuzione, nel momento in cui avviene l'invocazione della procedura. Ogni invocazione diversa della stessa procedura causa la generazione di un nuovo "oggetto procedura" con lo stesso legame di tipo, ma con diverso record di attivazione.

Il secondo livello di astrazione procedurale è dato dalle **funzioni** che altro non sono che procedure che restituiscono un valore all'unità chiamante. Queste vengono realizzate o creando una **pseudovariabile** nell'ambiente locale della procedura chiamata

¹Liskov e Guttag (1986)

o utilizzando una istruzione di **return** per restituire esplicitamente il controllo alla procedura chiamante inviandole allo stesso tempo il valore di una espressione.

3.2

AMBIENTE DI UNA PROCEDURA



3.2.1 ■ Record di attivazione

Analogamente a quanto avviene per un blocco d'istruzioni, di cui una procedura è astrazione, il **record di attivazione** rappresenta l'intero ambiente di esecuzione di una procedura. La forma generale di un record di attivazione consiste di tre parti:

1. L'**ambiente locale**: tutti i data object che sono definiti all'interno della procedura;
2. L'**ambiente non locale**: tutti i data object la cui definizione è propagata da altre procedure;
3. L'**ambiente dei parametri**: contiene informazioni sui dati che sono passati dalla e alla procedura.

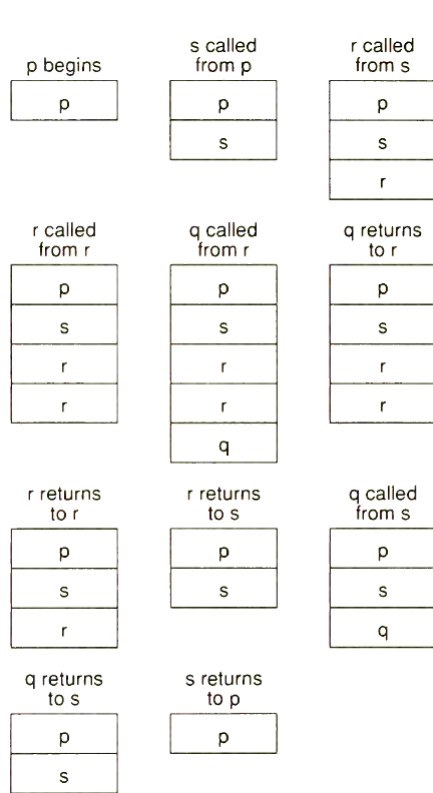
Ogni volta che una procedura viene invocata il suo record di attivazione viene aggiunto al cosiddetto **stack di attivazione**. Sul top dello stack c'è sempre il record relativo alla procedura correntemente in esecuzione. Alla terminazione della procedura, il record di attivazione viene rimosso dallo stack.

Esempio

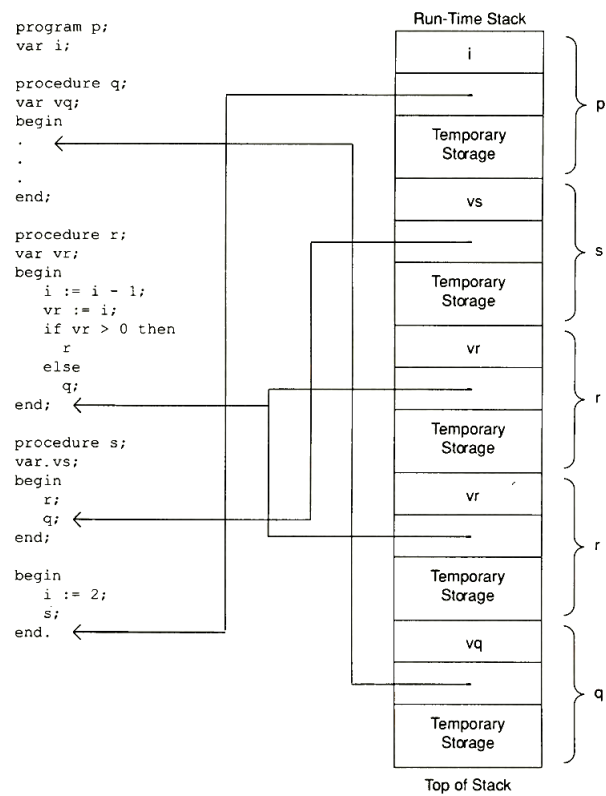
Si consideri il Listato 3.1. La Figura 3.2a mostra lo stack di attivazione relativo.

```
1  program p;  
2  var i;  
3  procedure q;  
4  begin  
5  ...  
6  end;  
7  
8  procedure r;  
9  begin  
10     i:=i-1;  
11     if i>0 then  
12         r;  
13     else  
14         q;  
15     end;  
16  
17  procedure s;  
18  begin  
19     r;  
20     q;  
21  end;  
22  
23  begin p;  
24     i:=2;  
25     s;  
26  end.
```

Codice 3.1: Esempio di invocazioni a procedure



(a) Stack di attivazione del Listato 3.1



(b) Esempio di ambiente locale

Figura 3.2

3.2.2 Ambiente locale

L'**ambiente locale** di una procedura include:

- Tutte le variabile dichiarate localmente.
- Un puntatore alla prossima istruzione (IP) che permette di riprendere l'esecuzione quando il controllo viene restituito alla procedura chiamate.
- Una memoria temporanea necessaria alla valutazione delle espressioni contenute nella procedura (altamente dipendente dalla realizzazione).

Estendendo l'esempio della figura 3.2a aggiungendo vari data object e mostrando il contenuto dell'ambiente locale si ottiene lo stack mostrato in Figura 3.2b.

3.2.3 Ambiente non locale

La maggior parte dei linguaggi di programmazione imperativi permettono alle procedure di accedere ad altri data objects oltre a quelli definiti all'interno dell'ambiente locale. L'ambiente dove avviene il legame con questi oggetti viene chiamato **ambiente non locale**.

Questo ambiente può essere rappresentato in un record di attivazione da un puntatore al record di attivazione il cui ambiente locale e globale determinerà l'ambiente globale dell'attuale record di attivazione. In altre parole, se viene richiesto l'accesso ad un dato che non è definito localmente, esso viene ricercato in modo ricorsivo nei record di attivazione precedenti.

Esistono tre tipologie per realizzare questa propagazione dei data object:

1. **Propagazione in ambito statico:** in questo caso l'ambiente non locale di una procedura è propagato dal programma che la contiene sintatticamente.
2. **Propagazione in ambito dinamico:** in questo caso l'ambiente non locale di una procedura è propagato dal programma chiamante. Nella propagazione in ambito dinamico il puntatore all'ambiente non locale non è più necessario. Per questo motivo è praticamente impossibile la determinazione dell'ambiente di esecuzione di una procedura durante la scrittura del codice sorgente in quanto questo viene definito solo a run-time.
3. **Nessuna propagazione:** l'uso di ambienti non locali è scoraggiato perché produce **effetti collaterali** non facilmente prevedibili.

Esempio

Si consideri il programma mostrato nel Listato 3.2.

```

1  program p;
2  var a,b,c: integer;
3
4  procedure q;
5  var a,c: integer;
6  procedure r;
7  var a: integer;
8  begin r {variabili: a da r; b da p; c da q; procedure: q da p; r da q}
9  ...
10 end;
11 begin q {variabili: a da q; b da p; c da p; procedures: q ed s da p; r da q}
12 ...
13 end;
14
15 procedure s;
16 var b: integer;
17 begin s
18 ... {variabili: a da p; b da s; c da p; procedures: q ed s da p}
19 end;
20 begin p {variabili: a, b, c da p; procedures: q, s da p}
21 ...
22 end;

```

Codice 3.2: Ambiente globale, scope statico

Supponendo una sequenza di attivazione (p, s, q, q, r), lo stack di attivazione di esecuzione avrà la forma come mostrato in Figura 3.3a. In ambito dinamico, la stessa sequenza di attivazione genera invece lo stack di esecuzione come mostrato in Figura 3.3b.

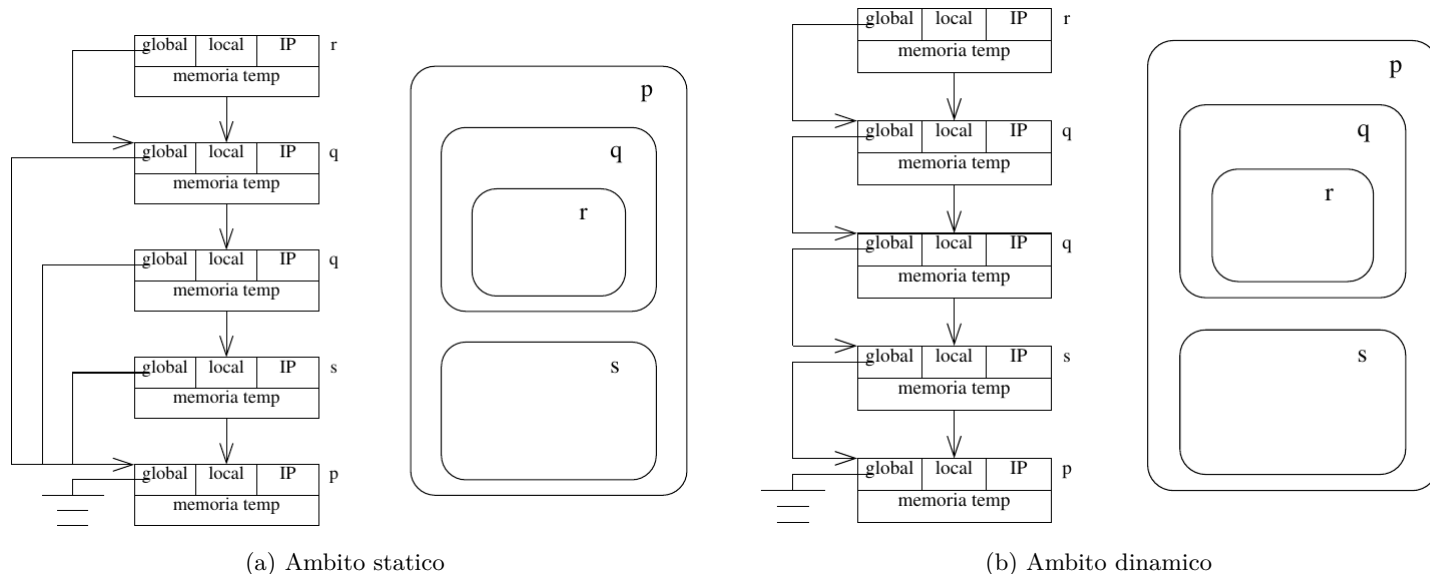


Figura 3.3



L'ambiente dei parametri è la terza parte necessaria a specificare l'ambiente di esecuzione di una procedura. I **parametri** sono i mezzi attraverso i quali l'informazione viene passata tra le unità chiamanti e le unità chiamate. I parametri vengono suddivisi in tre tipologie:

1. **Parametri IN:** sono passati dalla unità chiamante all'unità chiamata al momento dell'invocazione;
2. **Parametri OUT:** sono passati dall'unità chiamata all'unità chiamante al momento della terminazione della prima;
3. **Parametri IN OUT:** servono a far transitare le informazioni in entrambe le direzioni.

Un parametro deve essere specificato in due punti del programma:

- Nella definizione della procedura: in questo caso prendono il nome di **parametri formali**;
- Nell'invocazione della procedura: in questo caso prendono il nome di **parametri attuali**.

3.3.1 ■ Modalità di associazione

In una procedura, il parametro formale è rappresentato da un nome che è come una variabile all'interno della procedura. È inoltre consuetudine che il tipo del parametro venga specificato nell'intestazione della procedura: i parametri attuali e formali corrispondenti devono essere dello stesso tipo.

Esistono due eccezioni a questo requisito che permettono al parametro formale di essere vincolato a un tipo al momento della chiamata della procedura. L'approccio più radicale è quello di *lasciare il parametro formale senza alcuna specifica di tipo*. Il legame si instaura così **durante l'esecuzione** prendendo lo stesso tipo del parametro attuale. Questo però non è compatibile con il controllo di tipo a tempo di compilazione, poiché il tipo del parametro formale in questa situazione è sconosciuto a tempo di compilazione. Un'alternativa meno severa è quello di *permettere come eccezione solo quella degli array a dimensione variabile*. Il parametro formale assume quindi i vincoli di indice del corrispondente parametro attuale al momento dell'invocazione. In questo modo viene consentito il controllo del tipo in tempo di compilazione, poiché il tipo di base dell'array di parametri formali viene specificato nell'intestazione della procedura.

Esistono due metodi possibili per associare i parametri attuali ai parametri formali:

- **Per posizione:** a seconda della posizione relativa nella sequenza dei parametri;
- **Per nome:** il nome del parametro formale è aggiunto come prefisso al parametro attuale;

3.3.2 ■ Implementazione del passaggio dei parametri

Parametri IN Possono essere realizzati in due modi:

1. Con un **riferimento**: in questo caso la locazione del parametro attuale diventa la locazione del parametro formale. Poiché il parametro formale è di tipo IN, allora **si deve impedire la modifica** (ovvero la scrittura) all'interno della procedura.
2. Con una **copia**: in questo caso il record di attivazione contiene una locazione dove scriverà il valore del parametro formale. Questa implementazione impedisce la modifica del parametro rispettando il vincolo IN. Il linguaggio C, ad esempio, supporta solo questa modalità di passaggio.

Il secondo modo è meno efficiente del primo, sia rispetto allo spazio sia al tempo, ma è più flessibile e richiede meno variabili locali.

Parametri OUT Possono essere realizzati:

1. Con un **riferimento**;
2. Con una **copia**;

Questa tipologia di parametro rappresenta il risultato della procedura, per questo motivo alcuni linguaggi assumono che i parametri OUT **non possano essere inizializzati** e ne **proibiscono quindi la lettura** come uso a destra di un assegnamento o passaggio a un parametro IN o IN OUT di un'altra procedura.

Parametri IN OUT Sono una combinazione dei due precedenti. Anch'essi possono essere realizzati:

1. Con un **riferimento**: non ci sono limitazioni all'uso interno della procedura;
2. Con una **copia**: avvengono due processi di copia, uno durante l'attivazione ed uno durante la terminazione della procedura.

Esempio

In linguaggi come C, C++ e Java il passaggio dei parametri viene effettuato solo in modalità IN per copia. Tuttavia, in C e C++, è possibile simulare il passaggio per riferimento grazie ai puntatori. Infatti, *copiando* all'interno del parametro formale l'indirizzo della di memoria del parametro attuale è possibile simulare questa tipologia di associazione.

```
int foo ( int *var)
{
    *var = *var + 1;
}

//Chiamata alla funzione, passo alla funzione un puntatore
foo(&y);
```

Codice 3.3: Associazione dei parametri in C e C++

3.3.3 Aliasing

Per **aliasing** si riferisce alla possibilità di riferirsi alla stessa locazione con identificatori diversi. Questo nel passaggio dei parametri può creare confusione.

Esempio

Si determini l'uscita del programma del Listato 3.4 nel caso in cui i parametri siano realizzati per riferimento e nel caso in cui siano realizzati per copia.

```
program MAIN;
var A: integer;
procedure TEST ( var X, Y: integer);
begin
    X := A + Y;
    writeln(A, X, Y);
end;
begin
    A:= 1;
    TEST(A,A);
end.
```

Codice 3.4: Esempio di aliasing

Consideriamo il caso in cui i parametri sono realizzati per riferimento. Nel momento in cui si va a chiamare la procedura all'interno dello stack di attivazione sono presenti due record: uno per la procedura MAIN e uno per TEST. Nel primo record è definito solo la variabile A mentre in TEST sono presenti due variabili X e Y passate per riferimento: queste punteranno a locazioni di memoria di variabili presenti all'interno del programma chiamante come mostrato in Figura 3.4a. Quindi sarà:

$$\begin{aligned}env(x) &= env(a) \\ env(y) &= env(a)\end{aligned}$$

Dal momento in cui la variabile A viene inizializzata ad 1, si avrà quindi:

$$\begin{aligned}X &= A + Y; \\ X &= 1 + 1; \\ X &= 2;\end{aligned}$$

dove però X è un riferimento alla locazione di memoria di A:

$$mem(X) = mem(env(A)) = 2;$$

quindi si è modificato il valore all'interno della cella di memoria della variabile A. Nel momento della stampa si avrà quindi:

2, 2, 2

Consideriamo adesso il caso del passaggio di parametro per copia. Nel momento in cui si va a chiamare la procedura all'interno dello stack di attivazione sono presenti due record: uno per la procedura MAIN e uno per TEST. Nel primo record è definito solo la variabile A mentre in TEST sono presenti due variabili X e Y con le loro rispettive locazioni di memoria. Quando si va ad eseguire l'operazione $X = A + Y$; l'effetto ottenuto è quella di una modifica locale all'interno della cella di memoria della variabile X come mostrato in Figura 3.4b e la procedura stamperà la sequenza:

1, 2, 1

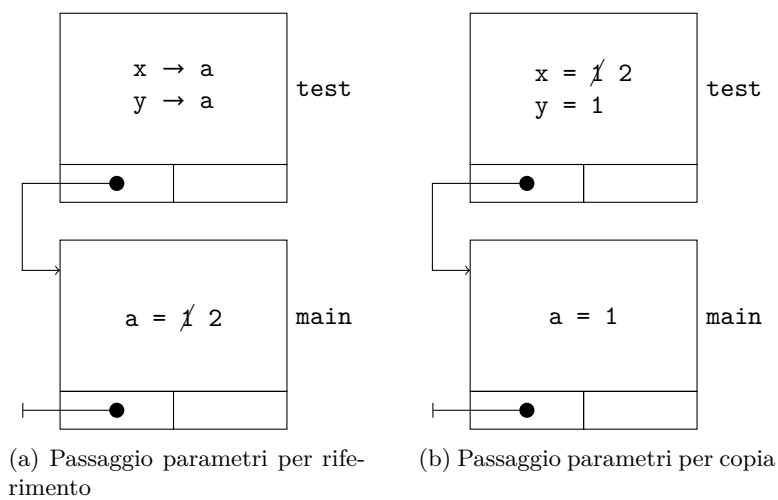


Figura 3.4

3.3.4 Procedure come parametri di procedura

Alcuni linguaggi permettono l'uso di procedure come argomento di altre procedure.

```

1 program MAIN;
2 VAR a: real;
3 procedure TESTPOS (X: real; procedure ERROR (MSG: string));
4 begin
5   if X ≤ 0 then ERROR ('Negative X in TESTPOS');
6 end;
7 procedure E1 (M: string);
8 begin
9   writeln('E1 error: ', M);
10 end;
11 procedure E2 (M: string);
12 begin E2 (M: string);
13 begin
14   writeln('E2 error: ',M);
15 end;
16 begin
17   readln(A);
18   TESTPOS(a, E1);
19   TESTPOS(A, E2);
20 end.

```



3.3.5 Macro

A partire da Algol 60 fu introdotto un modo diverso per il passaggio dei parametri: il **passaggio per nome**. Nei linguaggi moderni questa modalità è conosciuta sotto il nome di **macro** o anche **parametri passati per nome**. Questi si comportano alla stessa maniera dei parametri di IN OUT in quanto i nomi dei parametri attuali sostituiscono di fatto i nomi dei parametri formali. In linguaggi come il C, macro del genere vengono implementate mediante la sostituzione delle occorrenze con le relative espansioni da parte del preprocessore.

```
#DEFINE M(x,y) X=Y
```

Esempio

Ad esempio, data la procedura:

```
procedure swap (a,b: integer);  
var temp: integer;  
begin  
  temp:= a;  
  a:= b;  
  b:= temp;  
end;
```

Allora la chiamata `swap(x,y)` esegue il brano di codice:

```
temp:=x;  
x:=y;  
y:=temp;
```

L'utilizzo di espansioni del genere però comporta vari problemi che scaturiscono dal fatto che queste "procedure" di fatto non hanno alcun ambiente di esecuzione, le variabili utilizzate sono quindi le stesse delle blocco chiamante.

Esempio

Determinare i problemi che nascono dai due programmi, se `swap` è una macro:

```
1 program main;  
2 var  
3 i: integer;  
4 m: array[1...100] of integer;  
5 ...  
6 begin  
7 ...  
8 swap(i,m[i]);  
9 ...  
10 end.
```

Codice 3.5: Esempio macro swap 1

Nel primo programma la sostituzione del codice non produrrà uno swap corretto tra le due variabili in quanto prima dell'ultima operazione, il valore dell'indice viene modificato:

```
temp = i;  
i = m[i]; //Modifica del valore di i  
m[i]= temp; //L'indice non e' piu' corretto
```

```
1 program main;  
2 var  
3 i,temp : integer;  
4 ...  
5 begin  
6 ...  
7 swap(i,temp);  
8 ...  
9 end.
```

Codice 3.6: Esempio macro swap 2

In questo caso invece si utilizza un nome già presente all'interno della macro che per ovvie ragioni produce confusione a livello di calcolo:

```
1 temp = i;  
2 i = temp;  
3 temp = temp;
```





Una procedura q è detta **annidata** in un blocco B se è definita dentro B .

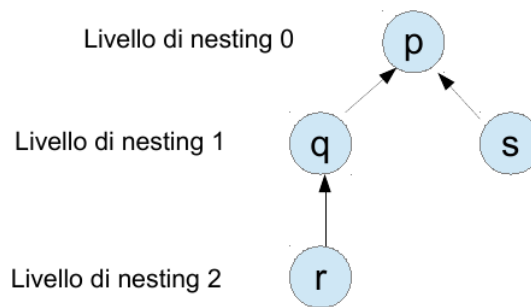
```

1  program p;
2  var a,b,c int;
3
4  procedure q;
5  var a,c int;
6
7  procedure r;
8  var a int;
9  ...
10 ...
11
12 procedure s;
13 var b int;
14 ...
15 ...

```

Codice 3.7: Procedure innestate

All'interno del Listato 3.7 le procedure q ed s sono annidate in p mentre r è annidata in q . Il **livello di nesting di una procedura** è dato dal numero di blocchi che la contengono. Il livello di nesting di q ed s sarà quindi 1 mentre quello di r è 2. In caso di scoping statico l'**ambiente non locale** di una procedura è dato dall'ambiente delle procedure in cui è innestata. L'annidamento si può rappresentare come un albero i cui livelli corrispondono ai livelli di nesting (vedi Figura 3.5).



Le frecce indicano l'ambiente non locale

Figura 3.5: Livelli di nesting

Per rappresentare una variabile non locale è possibile usare una coppia ordinata del tipo (livello, offset) utile ad eseguire la seguente espressione per individuare la locazione in memoria:

$$env(x) = v[livello] + offset \quad (3.1)$$

3.4.1 ■ Mantenimento del vettore degli ambienti non locali

La Figura 3.6 raffigura la struttura dei blocchi nello stack di attivazione durante l'esecuzione del Listato 3.7. Ogni blocco contiene un vettore di puntatori detto **vettore degli ambienti non locali**.

Una procedura F **può chiamare** una procedura G se e solo se G è definita in F oppure G è definita in uno dei blocchi che contiene F , quindi se e solo se F e G hanno una parte di ambiente non locale in comune.

Esempio

Nel caso in cui la procedura p chiami la procedura s si aggiunge un nuovo blocco sullo stack e si aggiunge un elemento al vettore $v[]$ che punta al blocco di p aumentando di conseguenza il livello di nesting (vedi Figura 3.7a).

Nel caso in cui la procedura q chiama la procedura r (vedi Figura 3.7b) si aumenta il livello di nesting, si aggiunge un elemento al vettore $v[]$ e infine si copiano gli elementi presenti nel blocco precedente.

Nel caso di chiamate a procedure definite esternamente, ad esempio sia la procedura s a chiamare la procedura q , quello che avviene è la copia degli elementi di $v[]$ all'interno del vettore del blocco q (Vedi Figura 3.7c).

Sia ora la procedura r a chiamare la procedura s . In questo caso il livello di nesting decresce e si copiano solo gli elementi di livello minore o uguale a quello di s . In questo caso la struttura ad albero e lo scoping statico garantiscono sempre di trovare l'ambiente non locale della procedura chiamata nel record del chiamante (vedi Figura 3.7d).

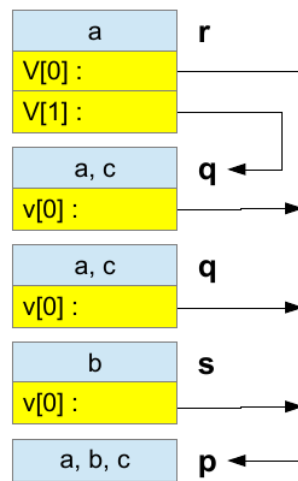


Figura 3.6: Stack di attivazione del Listato 3.7

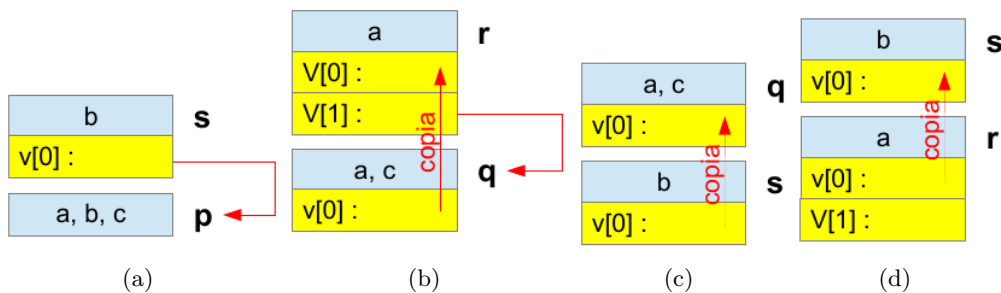


Figura 3.7: Mantenimento del vettore degli ambienti non locali

3.4.2 ■ Proprietà dell'implementazione mediante vettore dell'ambiente non locale

1. Accesso alle variabili in tempo costante:
 - Accesso a vettore + somma del puntatore ivi contenuto e dell'offset
 - Indipendente dai livelli di nesting
 - Calcolo supportato direttamente dalle istruzioni macchina
2. Creazione dei record di attivazione lineare nel livello di nesting grazie alla copia degli elementi di $V[]$
 - Indipendente dall'esecuzione
 - Dipende solo dal testo del sorgente;
3. Tempo costante. L'implementazione naive avrebbe richiesto di percorrere le liste di puntatori all'ambiente non locale a tempo di esecuzione. In questo caso si ha:
 - Accesso alle variabili in tempo lineare nel livello di nesting
 - Creazione dei record di attivazione in tempo lineare nel livello di nesting.

IL PARADIGMA OBJECT ORIENTED: IL LINGUAGGIO JAVA

Il linguaggio Java è un linguaggio di programmazione, con sintassi simile a C++ e semantica simile a SmallTalk. È di solito menzionato per produrre applet: applicazioni web che risiedono sul server ma sono eseguite dal browser web del cliente. Le applicazioni Java sono programmi autonomi che non richiedono un browser per essere eseguiti. Tali programmi richiedono solo un elaboratore su cui sia installato **Java Runtime Environment (JRE)**.

4.1 LE FASI DI UN PROGRAMMA



Un programma Java attraversa generalmente cinque fasi per essere eseguito:

1. scrittura/modifica;
2. compilazione;
3. caricamento;
4. verifica;
5. esecuzione.

4.1.1 Fase 1: Scrittura e modifica di un programma

La fase 1 è quella della **scrittura e modifica** di un file. Allo scopo si usa un programma detto **editor**. Un programmatore digita il programma con l'editor e lo modifica se necessario. Il programma viene successivamente salvato su un dispositivo di memorizzazione secondaria, come per esempio il disco rigido. I nomi dei file dei programmi Java terminano generalmente con l'estensione `.java`.

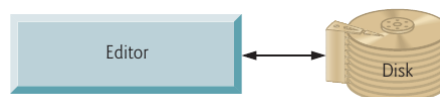


Figura 4.1: Il programma viene scritto su un editor e salvato con l'estensione `.java`

4.1.2 Fase 2: Compilazione di un programma Java in bytecode

Nella fase 2 il programmatore **compila** il programma attraverso il comando `javac`. Il compilatore Java **converte il codice Java in bytecode**, ovvero le istruzioni comprese dall'**interprete Java**, producendo un nuovo file salvato con l'estensione `.class`. Il codice bytecode contiene le operazioni che verranno eseguite durante la Fase 5 (esecuzione). Per poter visualizzare il codice bytecode è possibile eseguire il comando `javap -p`.

I bytecode vengono eseguiti dalla **Java Virtual Machine**, ossia una parte del **Java Development Kit**¹ e un componente

¹Un insieme di strumenti per lo sviluppo di programmi in Java

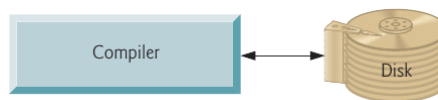


Figura 4.2: Il compilatore Java crea il codice bytecode e lo salva in un file `.class`

fondamentale della piattaforma Java. Una **macchina virtuale** (VM) è un'applicazione software che simula un computer, nascondendo ai programmatori che con essa interagiscono sia il sistema operativo, sia l'hardware sottostanti. Se la stessa macchina virtuale è disponibile per diverse piattaforme, le stesse applicazioni eseguite dalla VM possono automaticamente essere utilizzate in ognuna di tali piattaforme. La macchina virtuale Java è una delle macchine virtuali più diffuse al momento.

A differenza del linguaggio macchina, che è specifico per ciascun diverso tipo di hardware, il bytecode è costituito da istruzioni che sono *indipendenti* dalla piattaforma, ossia non sono specifiche per una particolare piattaforma hardware. In questo modo, il bytecode Java risulta essere **portabile**: lo stesso bytecode può essere eseguito in una qualsiasi piattaforma che contiene una JVM compatibile con la versione di Java nella quale il bytecode è stato originariamente compilato. La JVM è richiamabile dal terminale con il comando `java`.

4.1.3 Fase 3: Caricamento di un programma in memoria

La Fase 3 viene chiamata **caricamento**. Prima che un programma possa essere eseguito, deve essere posto nella memoria. Il programma **class loader** (o **caricatore delle classi**) prende il file (o i file) `.class` contenente i bytecode e lo trasferisce nella memoria primaria. Il class loader può anche caricare eventuali file `.class` messi a disposizione da Java e utilizzati dal nostro programma. Il file `.class` può essere caricato da un disco sul sistema o da una rete.

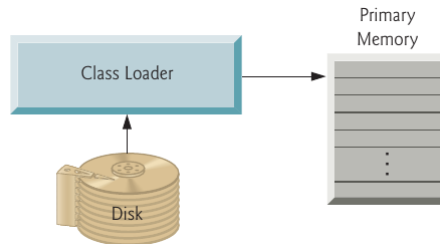


Figura 4.3: Il class loader legge dal disco il file `.class` contenente il codice bytecode e lo carica in memoria.

4.1.4 Fase 4: Verifica del bytecode

Quando le classi vengono caricate, i loro bytecode vengono verificati dal **verificatore di bytecode** nella fase 4. In questo modo si garantisce che i bytecode delle classi siano validi e rispettino le norme di sicurezza di Java. Java prevede infatti regole molto severe nell'ambito della sicurezza, affinché i programmi Java provenienti dalla rete non possano causare danni ai file e al sistema, come invece avviene con i virus.

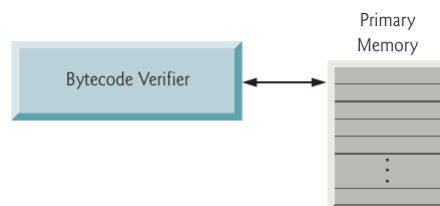


Figura 4.4: Il verificatore di bytecode controlla che tutto il codice sia valido.

4.1.5 Fase 5: Esecuzione

Finalmente, nella fase 5, l'**interprete**, sotto il controllo del sistema operativo, interpreta il programma, un bytecode per volta, eseguendo tutte le azioni previste dal programma stesso. Nelle prime versioni di Java, la JVM era semplicemente un'interprete di bytecode. Questa caratteristica rendeva i programmi in esecuzione molto lenti poiché la JVM interpretava ed eseguiva un bytecode alla volta. Le JVM attuali, invece, eseguono i bytecode sfruttando una combinazione di due tecniche: l'interpretazione e la cosiddetta compilazione **just-in-time** (JIT).

Durante questo processo, la JVM analizza i bytecode non appena essi vengono interpretati, cercando di localizzare quelle parti di bytecode eseguite più spesso, dette **hot spot**. Una volta isolate queste parti, esse vengono passate al compilatore just-in-time,

il quale le traduce nel linguaggio macchina specifico per la piattaforma corrente. Quando la JVM ritrova le stesse parti una seconda volta durante l'esecuzione del programma, ciò che viene effettivamente eseguito è il più veloce linguaggio macchina.

In questo modo i programmi Java passano per due fasi di compilazione distinte:

- La prima in cui il codice sorgente viene tradotto in bytecode (favorendo la portabilità verso JVM presenti in diverse piattaforme)
- La seconda nella quale, durante l'esecuzione, i bytecode vengono tradotti in linguaggio macchina specifico per il computer sul quale il programma è in esecuzione.

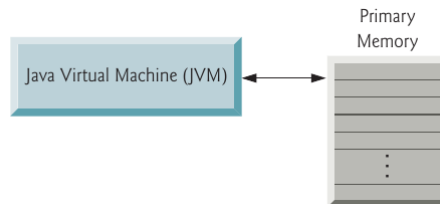


Figura 4.5: La JVM legge i bytecode e il compilatore JIT lo converte in linguaggio macchina.

4.2

LA JAVA RUNTIME ENVIRONMENT



La **Java Runtime Environment**, o JRE, è uno strato software che viene eseguito all'interno di un sistema operativo e fornisce le librerie e le varie risorse di cui ha bisogno un'applicazione Java per essere eseguita. La JRE è una dei tre componenti necessari allo sviluppo e all'esecuzione dei programmi scritti in Java:

- Il **Java Development Kit**, o JDK, è un insieme di strumenti per sviluppare applicazioni Java. Gli sviluppatori scelgono quale JDK utilizzare in base alla versione o all'edizione. Ogni JDK fornisce un JRE compatibile dato che l'esecuzione di un programma fa parte del processo di sviluppo dello stesso.
- La **Java Virtual Machine** è una macchina immaginaria che è emulata dalla macchina reale. I programmi per la JVM sono contenuti in file `.class`, ciascuno dei quali contiene al più una classe pubblica.

La Java Virtual Machine quindi:

- fornisce le specifiche della piattaforma hardware;
- legge i codici bytecode compilati, che sono indipendenti dalla piattaforma;
- è realizzata in software o in hardware;
- è realizzata come strumento di sviluppo software oppure in un browser Web.

Dal punto di vista della realizzazione, la Java Virtual Machine specifica:

- l'insieme di istruzioni della CPU;
- l'insieme dei registri;
- il formato del file Class;
- lo stack di esecuzione;
- lo heap gestito dal garbage collector;
- l'area di memoria.

4.2.1 ■ Il garbage collector

Il **garbage collector** è una delle misure adottabili per evitare problemi di *memory leakage*. La memoria allocata che non è più necessaria deve essere resa disponibile; in altri linguaggi la deallocazione è a totale responsabilità del programmatore ma Java fornisce un processo a livello di sistema operativo che è capace di tracciare l'allocazione di memoria che ricerca e libera la memoria non più utilizzata. Il garbage collector viene eseguito autonomamente ed è dipendente dalle varie realizzazioni della Java Virtual Machine.



Java definisce otto tipi primitivi di dati: **byte**, **short**, **int**, **long**, **char**, **float**, **double** e **boolean**. I tipi primitivi vengono chiamati anche **semplici**. Questi possono essere raggruppati in quattro gruppi:

1. **Interi:** questo gruppo include **byte**, **short**, **int** e **long** che sono numeri interi con segno.
2. **Numeri a virgola mobile:** questo gruppo include i valori **float** e **double** che rappresentano i numeri di tipo razionale.
3. **Letterali:** questo gruppo include i valori di tipo **char** che rappresenta i simboli come lettere e numeri. Un **char** rappresenta un carattere Unicode (16 bit) incluso tra apici singoli.
4. **Booleani:** questo gruppo include il tipo **boolean** che è un tipo speciale per rappresentare i valori true e false. Il tipo **boolean** ha due letterali: **true** e **false**. Non c'è cast tra tipi interi e boolean a differenza del linguaggio C. Interpretare valori numerici come valori logici non è permesso in Java.

4.3.1 I tipi interi

I tipi interi rappresentano i valori mostrati nella tabella 4.1.

Lunghezza	Tipo	Range
8 bit	byte	$-2^7 \dots 2^7 - 1$
16 bit	short	$-2^{15} \dots 2^{15} - 1$
32 bit	int	$-2^{31} \dots 2^{31} - 1$
64 bit	long	$-2^{63} \dots 2^{63} - 1$

Tabella 4.1: Intervallo dei valori dei tipi interi

I **litterali interi** hanno tre forme: decimale, ottale ed esadecimale. Di default la forma è decimale ma si può specificare la forma tramite lo 0 per denotare valori ottali e lo 0x per i valori esadecimali. Assumono come tipo di default **int**. Mediante il suffisso L oppure l se si vuole che siano di tipo **long**.

```

1 public class Types
2 {
3     public static void main(String[] args)
4     {
5         int a = 1;    // il valore decimale e' 2
6         int b = 077;  // lo zero iniziale denota un valore ottale
7         int c = 0xBAAC; // la parte 0x iniziale denota un valore esadecimale
8         System.out.println("a=" + a + ", b=" + b + ", c=" + c);
9
10        int d = 2L;   // e' 2, rappresentato come long
11        int e = 077L;  // e' un valore ottale, rappresentato come long
12        int f = 0xBAACL; // e' un valore esadecimale, rappresentato come long
13        System.out.println("d =" + d + ", e =" + e + ", f =" + f);
14    }
15 }
```

Codice 4.1: Litterali interi

Output:

```
a=1, b=63, c=47788
d=2, e=63, f=47788
```

Quando si assegna il valore di un litterale ad una variabile, il compilatore **determina la dimensione del litterale** a seconda della variabile:

```
1 short s = 9; // OK in quanto 9 e' un valore ammissibile
```



Quando si assegna il valore di una espressione ad una variabile, la dimensione del valore **non è più modificabile**:

```
1 short s1 = 9 + s; // Errore di compilazione: 9+s e' un int e non uno short
```



4.3.2 Tipi a virgola mobile

Il tipo di dato **float** è utilizzato per la rappresentazione di numeri reali in virgola mobile a singola precisione (32 bit) secondo lo standard IEEE 754, mentre il **double** per quelli double precision (64 bit). Hanno come tipo di default il tipo **double** e, come per gli interi, possono avere dei suffissi per forzare la forma: **F** o **f** per literal di tipo **float** e **D** o **d** per literal di tipo **double**.

```
1 public class Types {
2     public static void main(String[] args) {
3         double a = 3.14; // e' 3.14, rappresentato come double
4         float b = 3.14f; // e' 3.14, rappresentato come float
5         double c = 3.14e2; // e' 314, rappresentato come double
6         float d = 2.718F; // e' 2.718, rappresentato come float
7         double e = 123.4E-5D; // e' 0.001234, rappresentato come double, la D e' superflua
8         System.out.println("a=" + a + ", b=" + b + ", c=" + c);
9         System.out.println("d=" + d + ", e=" + e);
10    }
11 }
```

Codice 4.2: Literal a virgola mobile

Output:

```
a=3.14, b=3.14, c=314.0
d=2.718, e=0.001234
```

Esempio

In questo listato si mostra un esempio di assegnazioni a variabili di tipo primitivo e di tipo stringa:

```
1 public class Assign {
2     public static void main(String[] args) {
3         int x, y;
4         float z = 3.1414f;
5         double w = 3.14145;
6         boolean truth = true;
7         char c;
8         String str;
9         String str1 = "Hello";
10        c = 'A';
11        str = "Hello World";
12        x = 6;
13        y = 10000;
14    }
15 }
```



Sono assegnazioni illegali istruzioni come le seguenti:

```
1 x = 3.14159; // 3.14159 non e' un int; richiede casting esplicito
2 w = 173,000; // la virgola invece del punto decimale
3 truth = 1; // errore comune tra programmatori C/C++
4 z = 3.121213; // 3.121213 non e' un float richiede casting
```



4.3.3 Casting dei tipi primitivi

Conversioni implicite

Chi ha un po' di esperienza nel campo informatico saprà che è abbastanza comune la prassi di assegnare un valore di un determinato tipo ad una variabile di un altro tipo. Quando due tipi di dato sono *compatibili* si esegue una **conversione automatica** o **implicita**. Ad esempio, è sempre possibile assegnare un valore intero ad una variabile di tipo **long**.

Non tutti i tipi di dato però sono compatibili tra di loro e, per questo motivo, non sono possibili tutti i tipi di conversioni automatiche. Infatti, non è possibile effettuare una conversione automatica dal tipo **double** al tipo **byte**. Le **conversioni legali**, cioè quelle che avvengono automaticamente, sono quelle per cui *esiste un percorso nel grafo* mostrato in Figura 4.6.

Quando si assegna un tipo di dato ad una variabile di un tipo diverso si può avere una **conversione automatica** sono in questi due casi:

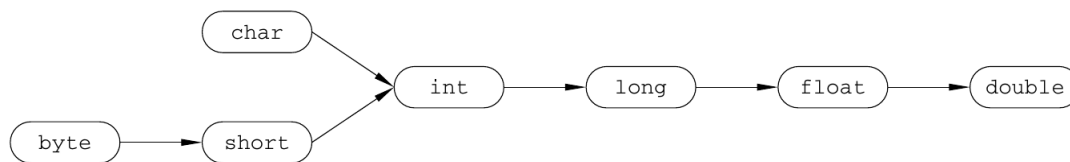


Figura 4.6: Conversioni automatiche in Java

1. I due tipi sono compatibili;
2. Il tipo della variabile di destinazione è più grande della sorgente.

Quando queste due condizioni sono soddisfatte si ottiene una **conversione per ampliamento**. Ad esempio, il tipo `int` sarà sempre largo abbastanza per contenere un valore di tipo `byte` e per questo motivo non sarà necessario fare un cast esplicito. Per le conversioni di ampliamento, i tipi numerici, compresi i tipi interi e a virgola mobile, sono compatibili tra loro. Tuttavia, non ci sono conversioni automatiche dai tipi numerici a `char` o `boolean`. Inoltre, `char` e `boolean` non sono compatibili tra loro.

Come già detto, Java esegue una conversione automatica del tipo anche quando si memorizza una costante letterale intera in variabili di tipo `byte`, `short`, `long` o `char`.

Conversioni esplicite

Sebbene le conversioni automatiche dei tipi siano utili, non soddisfano tutte le esigenze. Ad esempio cosa succede se si vuole assegnare un valore `int` a una variabile `byte`? Questa conversione non verrà eseguita automaticamente, perché un `byte` è più piccolo di un `int`. Questo tipo di conversione viene talvolta chiamato **downcast** o conversione di **restringimento** (*narrowing*), poiché si rende esplicitamente il valore più stretto in modo che si adatti al tipo di destinazione. Per creare una conversione tra due tipi incompatibili, bisogna usare un **cast**. Un cast è semplicemente una **conversione di tipo esplicito**. La sua forma generale è questa:

(target-type) value

Qui, *target-type* specifica il tipo desiderato in cui convertire il valore specificato. Ad esempio, il frammento che segue esegue la conversione di un `int` in un `byte`. Se il valore dell'intero è più grande dell'intervallo di un `byte`, sarà ridotto in modulo (il resto della divisione di un intero per l'intervallo del `byte`):

```
int a;
byte b;
// ...
b = (byte) a;
```

Codice 4.3: Esempio di cast esplicito

Quando un valore in virgola mobile viene assegnato a un numero intero, si verifica un altro tipo di conversione: il **troncamento**. Come è noto, i numeri interi non hanno componenti frazionarie. Pertanto, quando un valore in virgola mobile a un tipo intero, la componente frazionaria viene persa. Ad esempio, se il valore 1,23 viene assegnato a un numero intero, il valore risultante sarà semplicemente 1.

Attenzione

Il casting esplicito non è necessario per i literal interi (non i floating point) che ricadono nel range legale del tipo di destinazione.

```
1 int i = 12;
2 byte b = 12;
3 byte b = 128; // Error: e' superiore a 127
4 byte c = i;   // Error: vale 12 ma non e' un literale
5 float x = 3.14; // Illegale: eccezione non valida per f.p
```

■

Promozione aritmetica

In una espressione aritmetica, le variabili sono automaticamente promosse ad una forma più estesa per non causare perdita di informazioni. Le regole per gli operatori unari sono le seguenti:

- Se l'operando è un `byte`, uno `short` o un `char`, esso è convertito ad `int` (a meno che l'operatore non sia `++` o `--`, nel qual caso non avviene nessuna conversione).

- Altrimenti, nessuna conversione.

Le regole per operatori binari:

- Se uno degli operatori è un `double` allora l'altro operando è convertito a `double`
- Se uno degli operandi è un `float`, allora l'altro operando è convertito a `float`
- Se uno degli operandi è un `long`, allora l'altro operando è convertito a `long`.
- Altrimenti, entrambi gli operandi sono convertiti ad `int`.

```
short s = 9;
int i = 10;
float f = 11.1f;
double d = 12.2;
double result = -s * i + f / d;
System.out.println("result = " + result);
```

Codice 4.4: Promozione aritmetica

Output:

result = -89.09016390315821

4.4

TIPI REFERENCE: LE CLASSI



Il concetto di **classe** è il nucleo di Java. Le classi definiscono la forma e la natura degli oggetti e per questo motivo costituiscono la base del paradigma ad oggetti. Quando si definisce una classe, si dichiarano la sua forma e la sua natura esatta, specificando i dati che essa contiene e il codice che agisce su tali dati. Anche se le classi molto semplici possono contenere solo codice o solo dati, moltissime classi del mondo reale possono contenere entrambe.

Il blocco di codice 4.5 mostra la forma generale per la definizione di una classe. Ogni classe viene dichiarata mediante la keyword **class**.

```
class nomeClasse{
    //Definizione degli attributi della classe
    tipoAccesso tipo variabileIstanza1;
    tipoAccesso tipo variabileIstanza2;
    ...
    tipoAccesso tipo variabileIstanzaN;

    //Definizione dei metodi
    tipoAccesso tipo nomeMetodo( elenco parametri){
        //Corpo del metodo
    }
    tipoAccesso tipo nomeMetodo2( elenco parametri){
        //Corpo del metodo
    }
    ...
    tipoAccesso tipo nomeMetodoN( elenco parametri){
        //Corpo del metodo
    }
}
```

Codice 4.5: Sintassi generale per la dichiarazione di una classe

I dati, o le variabili, definiti all'interno di una istruzione **class**, sono denominati **attributi** o **variabili di istanza** perchè ogni **istanza della classe** (ossia ogni oggetto della classe) contiene una propria copia di tali dati. Gli attributi relativi a un oggetto sono unici e a sè stanti rispetto agli attributi relativi ad un altro oggetto della medesima classe.

Il codice è contenuto all'interno dei **metodi**. Nel loro insieme, i metodi e le variabili di istanza definiti all'interno di una classe sono definiti **membri della classe**. Nella maggior parte delle classi, sono i metodi definiti per tale classe ad agire sulle variabili di istanza e ad accedere ad esse. Quindi sono i metodi a determinare in che modo possono essere utilizzati i dati di una classe.

Una classe **definisce un nuovo tipo di dati** che, al contrario dei tipi primitivi, vengono chiamati **tipi riferimento**. I "valori" dei tipi riferimento sono chiamati **oggetti** o **istanze**. Una differenza fondamentale tra tipi primitivi e tipi riferimento (ma non l'unica) è la seguente:

- il programma manipola direttamente i valori di tipi primitivi: ad esempio, i valori vengono immessi direttamente nelle variabili, e sono passati direttamente agli operatori;

- invece, il programma accede a un oggetto sempre e solo tramite un riferimento (chiamato *reference*), che svolge un ruolo analogo a quello del puntatore del C.

In Java, tutti i tipi non primitivi sono detti tipi **reference**. Una variabile *reference* altro non è che la “maniglia” di un oggetto: contiene ovvero l’indirizzo dello spazio dove viene allocato l’oggetto.

4.4.1 ■ Il costruttore

A differenza del linguaggio C, per allocare spazio e creare un nuovo oggetto bisogna invocare il **costruttore della classe** dell’oggetto mediante l’istruzione:

```
new Xxx()
```

il quale scatena i seguenti processi:

1. Viene allocato lo spazio per il nuovo oggetto e le variabili dell’istanza sono inizializzate al loro valore di default;
2. Viene eseguita ogni inizializzazione esplicita degli attributi;
3. Viene eseguito un costruttore;
4. Viene assegnato il riferimento finale all’oggetto.

Esempio

Si consideri il seguente listato:

```
1 public class MiaData {
2     private int giorno = 1;
3     private int mese = 1;
4     private int anno = 2006;
5
6     MiaData (int a, int b, int c){
7         giorno = a;
8         mese = b;
9         anno = c;
10    }
11 }
12
13 public class TestMiaData{
14     public static void main (String [] args){
15         MiaData oggi = new MiaData();
16     }
17 }
```

La dichiarazione:

`MiaData oggi`

causa l’allocazione dello spazio memoria del solo riferimento (ancora non inizializzato).

Ogni volta che si crea un’istanza di una classe, si crea un oggetto contenente una propria copia di ogni variabile definita dalla classe. Per accedere alle variabili istanza, occorre utilizzare l’**operatore dot** (.) che collega il nome dell’oggetto al nome di una variabile di istanza. In generale, l’operatore punto viene utilizzato per accedere sia alla variabile di istanza sia ai metodi all’interno di un oggetto.

Attenzione

I costruttori non sono metodi di una classe, non vengono ereditati e non hanno tipi di ritorno. Gli unici modificatori validi per un costruttore sono **public**, **protected**, **private**.

In ogni classe c’è sempre almeno un costruttore che prende il nome di **costruttore di default** il quale non ha argomenti né corpo. Questo permette di creare istanze di classi senza dover scrivere obbligatoriamente un costruttore.

nascita	????	nascita	????
giorno	0	giorno	1
mese	0	mese	1
anno	0	anno	2006

(a) L'uso della parola chiave **new** alloca spazio per un oggetto di tipo **MiaData** (inizializzata ai valori di default)

(b) Successivamente, per gli attributi sono usate le inizializzazioni esplicite all'interno della classe, quelle che eventualmente sono scritte nel momento della definizione dell'attributo

nascita	????	nascita	0x0123abcd
giorno	23	giorno	23
mese	4	mese	4
anno	1964	anno	1964

(c) Viene invocato il costruttore. Con esso si possono sostituire inizializzazioni personali dell'utente dell'oggetto a quelle di default previste dal progettista della classe. Si possono anche passare argomenti, così che il codice che richiede la costruzione del nuovo oggetto possa controllare l'oggetto che verrà creato

(d) L'assegnazione infine inserisce l'indirizzo del nuovo oggetto nella locazione del riferimento

Figura 4.7: Fasi durante la creazione di un oggetto in Java

Attenzione

Se si dichiara un costruttore con argomenti la JVM non includerà automaticamente un costruttore vuoto. Quindi qualsiasi chiamata ad un eventuale costruttore sifatto causa un errore in fase di compilazione.

4.4.2 I metodi

La sintassi generale di un metodo è il seguente:

```
1 tipo nomemetodo (elenco parametri){
2     //corpo del metodo
3 }
```

Codice 4.6: Sintassi di un metodo Java

In questo caso, al pari delle funzioni all'interno del paradigma procedurale, *tipo* specifica il tipo di dato restituito dal metodo. Può trattarsi di un qualunque tipo valido, compresi i *tipi reference* creati dal programmatore. Se il metodo non restituisce un valore, il tipo di ritorno deve essere **void**. Il nome del metodo è specificato dal *nomemetodo*. L'elenco parametri è una sequenza di coppie di tipo e identificatore separate da virgole. I parametri sono fondamentalmente delle variabili che ricevono il valore dagli argomenti passati al metodo quando viene chiamato. Se il metodo non ha parametri, l'elenco dei parametri è vuoto. I metodi che hanno un tipo di ritorno diverso da **void** restituiscono un valore al chiamante utilizzando la seguente sintassi dell'istruzione **return**:

return valore;

In questo caso *valore* è il valore restituito.

I metodi definiscono il funzionamento interno delle classi. Attraverso i modificatori di accesso inoltre è possibile nascondere il layout specifico della struttura dei dati interna e il funzionamento dei vari algoritmi. Oltre a definire metodi che garantiscono l'accesso ai dati, è anche possibile definire infatti anche metodi che vengono utilizzati internamente dalla classe stessa.

Ad esempio possiamo aggiungere un metodo **Volume** alla classe **Box** per ottenere il valore del volume di un suo oggetto. Sarà quindi:

```
1 void Volume(){
2     System.out.println("Volume =" + width*height*depth);
3 }
```

Codice 4.7: Esempio di metodo in Java

Nella classe **STARTER** sarà quindi sufficiente invocare il metodo **Volume** dell'oggetto **mybox** per ottenere il calcolo del suo volume come mostrato nel Codice 4.11.

I metodi getter e setter

Gli attributi presenti in una classe vengono solitamente specificati ad accesso privato (seguendo un principio chiamato **incapsulamento**) per non permettere ad altre classi di potervi accedere direttamente e leggere o modificarne il valore. Di conseguenza, un **cliente di un oggetto**, ossia una qualsiasi classe che ne chiama i metodi può chiamare solo i metodi definiti pubblici al fine di manipolare i suoi campi privati. Per ovviare a questa restrizione le classi forniscono dei metodi pubblici che permettono ai loro clienti di impostare (**set**) dei valori alle proprie variabili d'istanza, oppure di ottenere (**get**) tali valori:

- Un metodo **setter** non restituisce nessun dato al termine della propria esecuzione. Per questo motivo, il suo tipo di ritorno sarà **void**. Il metodo però riceve un parametro che rappresenta il valore che viene passato come argomento da assegnare all'attributo.

```
1 public void setWidth(double w) {
2     width = w;
3 }
```

Codice 4.8: Metodo setter

- Un metodo **getter** restituisce il valore del particolare attributo dell'oggetto che esegue tale metodo. Il metodo ha una lista vuota di parametri, per cui non richiede ulteriori dati per essere eseguito, mentre restituisce un valore del tipo dell'attributo che viene così passato al metodo chiamante.

```
1 public double getWidth() {
2     return width;
3 }
```

Codice 4.9: Metodo getter

Esempio

Costruiamo una classe denominata Box (Codice 4.10) che definisce tre variabili di istanza:

- width;
- height;
- depth.

e dei metodi, chiamati **getter** e **setter**, che permettono di accedere e modificare indirettamente tali attributi.

```
1  class Box{
2      //Attributi
3      private double width;
4      private double height;
5      private double depth;
6      //Metodi
7      public double getWidth() {
8          return width;
9      }
10
11     public void setWidth(double w) {
12         width = w;
13     }
14
15     public double getHeight() {
16         return height;
17     }
18
19     public void setHeight(double h) {
20         height = h;
21     }
22
23     public double getDepth() {
24         return depth;
25     }
26
27     public void setDepth(double d) {
28         depth = d;
29     }
30 }
```

Codice 4.10: Definizione della classe Box

Esempio

Per dichiarare oggetti della classe BOX e chiamare i loro metodi è necessario dichiarare una seconda classe che conterrà il metodo **main**:

```
1  class Starter{
2      public static void main(String [] args){
3          Box myBox = new Box(); //Dichiarazione di un oggetto di tipo Box
4      }
5  }
```



```
1  class Starter{
2      public static void main(String [] args){
3          Box myBox = new Box();
4          myBox.setWidth(3);
5          myBox.setHeight(3);
6          myBox.setDepth(3);
7          myBox.Volume();
8      }
9  }
```

Codice 4.11: Utilizzo dei metodi getter e setter e chiamata del metodo Volume

Le variabili *definite all'interno di un metodo* vengono dette **locali**; esse vengono create quando il metodo viene invocato e sono distrutte alla sua terminazione; esse **devono essere inizializzate** esplicitamente prima di essere utilizzate. Le variabili usate come parametro di metodi, visto il meccanismo di passaggio di parametri sono variabili locali.

Le variabili *definite all'esterno di un metodo* sono create quando viene costruito un oggetto. Ve ne sono di due tipi:

- **Variabili di classe:** sono dichiarate usando il modificatore `static`; sono create nel momento in cui una classe è caricata in memoria (esistono a prescindere dalle istanze); continuano ad esistere finché la classe rimane in memoria.
- **Variabili di istanza:** quelle dichiarate senza modificatore `static`; sono create durante la costruzione di una istanza; continuano ad esistere finché l'oggetto esiste; per ogni istanza di una classe esistono diverse variabili di istanza.

Esempio

Si consideri il seguente listato:

```
1 public class EsempioAmbito {
2     private int i = 1;
3
4     public void primoMetodo(){
5         int i = 4, j = 5;
6         this.i = i+j;
7         System.out.println("Valore di i: " + this.i);
8         secondoMetodo(7);
9     }
10
11     public void secondoMetodo(int i){
12         int j = 8;
13         this.i = i+j;
14         System.out.println("Valore di i: " + this.i);
15     }
16 }
17
18 public class TestAmbito{
19     public static void main(String[] args) {
20         EsempioAmbito ambito = new EsempioAmbito();
21         ambito.primoMetodo();
22     }
23 }
```

Output:

```
Valore di i: 9
Valore di i: 15
```

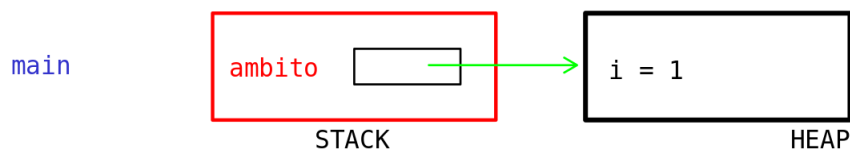
Nessuna variabile può essere usata prima di essere inizializzata. Le variabili non locali sono inizializzate dalla Java Virtual Machine nel momento in cui la classe è caricata in memoria o in cui è allocato spazio per il nuovo oggetto. Le variabili locali invece **devono essere inizializzate manualmente** prima dell'uso.

4.4.3 Il passaggio dei parametri

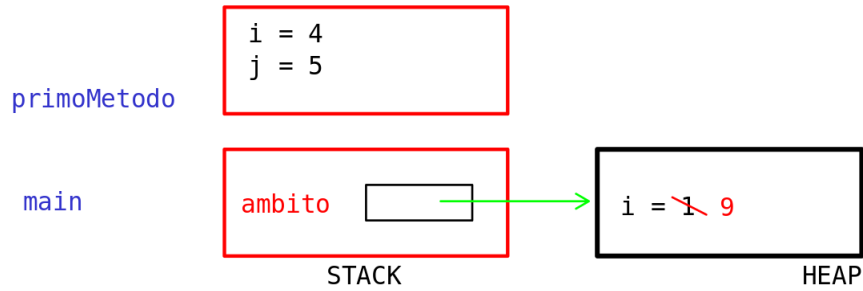
Java permette il **passaggio dei parametri per valore** (parametri IN realizzati per copia). Il passaggio per riferimento (che permette la modifica del valore del parametro nel contesto della procedura chiamante) è **proibito in Java**. Quando si passa una istanza di oggetto come argomento di un metodo, quello che si sta passando non è l'oggetto ma solo un riferimento a quell'oggetto. In quest'ultimo caso sarà possibile la modifica, nel contesto del chiamato, dell'oggetto (mai del valore della variabile) a cui fa riferimento quella variabile.

Si consideri la seguente porzione di codice:

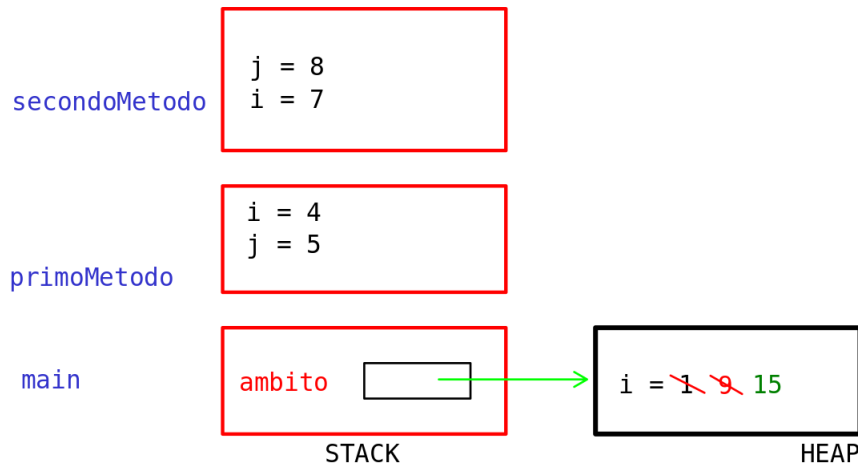
```
1 public class PassTest{
2     public static void cambiaValore (int v){ v = 55; }
3     public static void cambiaOggetto (MiaData d){
4         d = new MiaData(1,1,2005);
5     }
6     public static void cambiaAttributo(MiaData d){
7         d.setGiorno(16);
8     }
9     public static void main(String[] args) {
10        MiaData data = new MiaData(1,1,2000);
11        int val = 11;
12        cambiaValore(val);
13        System.out.println("Valore val: " + val);
14        cambiaOggetto(data);
15        data.print();
16        cambiaAttributo(data);
17    }
```



(a) Nel main viene dichiarato un oggetto chiamato ambito



(b) L'oggetto invoca il metodo `primoMetodo` che modifica lo stato dell'oggetto



(c) Il metodo `primoMetodo` invoca `secondoMetodo(int)` che effettua un'ulteriore modifica della variabile d'istanza `i`

```
17 data.print();
18 }
19 }
```



Nel caso delle variabili di tipo reference però, essendo queste ultime dei “puntatori” all’area di memoria dove è stato creato l’oggetto, è possibile modificare gli attributi dell’oggetto puntato ma non sostituirlo con un oggetto differente. Infatti si ha:

```
Valore val: 11
Data: 1/1/2000
Data: 16/1/2000
```

4.4.4 ■ Lo shadowing: la parola chiave `this`

La parola chiave `this` può essere usata per fare riferimento, all’interno di un metodo o di un costruttore locale, ad attributi o metodi locali. Questa tecnica è usata per risolvere ambiguità in alcuni casi in cui la variabile locale di un metodo maschera un attributo locale dell’oggetto. Oltre a questo la parola chiave `this` permette ad un oggetto di passare il riferimento a sé stesso come parametro ad un altro metodo o costruttore.

```
1 public class MiaData{
2     private int giorno;
3     private int mese;
4     private int anno;
5
6     public MiaData(int giorno, int mese, int anno){
7         this.giorno = giorno;
8         this.mese = mese;
9         this.anno = anno;
10    }
```

```

11
12 public MiaData(MiaData d){
13     this.giorno = d.giorno;
14     this.mese = d.mese;
15     this.anno = d.anno;
16 }
17
18 public void setGiorno(int giorno){
19     this.giorno = giorno;
20 }
21
22 public MiaData addGiorni (int g){
23     MiaData nuovaData = new MiaData(this);
24     nuovaData.giorno += g;
25     return nuovaData;
26 }
27
28 public void print(){
29     System.out.println("Data: " + giorno + "/" + mese + "/" + anno);
30 }
31 }

```

Codice 4.12: La parola chiave this

4.5

EREDITARIETÀ E POLIMORFISMO



4.5.1 La keyword extends

L'**ereditarietà** è uno dei capisaldi della programmazione ad oggetti in quanto permette la creazione di strutture gerarchiche. Usando l'ereditarietà, è possibile creare una classe generale che definisce un tratto comune di un insieme di classi imparentate. Questa classe sarà possibile essere ereditata da altre classi più specifiche, ciascuna delle quali si dovrà preoccupare di definire solo e soltanto i suoi tratti particolari. Nella terminologia Java, una classe che viene ereditata è chiamata **superclasse**. La classe che eredita le caratteristiche della superclasse viene chiamata invece **sottoclasse**. Una sottoclasse può diventare a sua volta una superclasse per una classe futura. Per questo motivo, una sottoclasse è una versione più specializzata della superclasse. Essa eredita tutti i membri definiti dalla superclasse e ne aggiunge di suoi, del tutto particolari.

Per ereditare i membri di una superclasse basta incorporare la sua definizione attraverso l'utilizzo della parola chiave **extends** all'interno della definizione della sottoclasse.

```

1 public class Impiegato {
2     private String nome = "";
3     private double stipendio;
4     private Date dataNascita;
5
6     public String getDettagli(){
7         ...
8     }
9 }
10
11 public class Quadro extends Impiegato{
12     private String reparto;
13     public String getReparto(){
14         return reparto;
15     }
16 }

```

Codice 4.13: Un semplice esempio di ereditarietà

4.5.2 La keyword super

Consideriamo il listato 4.14. In questo esempio, la classe **BoxWeight** non è stata definita in maniera molto efficiente. Per esempio, il costruttore **BoxWeight** inizializza esplicitamente i campi **width**, **height** e **depth** di **Box**. Questo non solo duplica il codice che viene ereditato dalla superclasse ma implica anche il fatto che la classe derivata debba avere accesso diretto a quei campi.

Nel caso questi attributi fossero stati privati il compilatore Java avrebbe riportato sicuramente un errore. Dal momento in cui l'incapsulamento è uno dei pilastri del paradigma orientato agli oggetti, Java viene incontro al programmatore per superare questo problema. Quando una sottoclasse necessita di riferirsi alla sua superclasse, può farlo usando la parola chiave **super**.

```
1  class Box{
2      double width;
3      double height;
4      double depth;
5
6      Box(Box ob){
7          width = ob.width;
8          height = ob.height;
9          depth = ob.depth;
10     }
11
12     Box(double width, double height, double depth){
13         this.width = width;
14         this.height = height;
15         this.depth = depth;
16     }
17
18     Box(){
19         width = -1;
20         height = -1;
21         depth = -1;
22     }
23
24     Box( double lenght){
25         width = height = depth = lenght;
26     }
27
28     double volume(){
29         return width * height * depth ;
30     }
31 }
32
33 class BoxWeight extends Box{
34     double weight;
35     BoxWeight(double width, double height, double depth, double weight){
36         this.width = width;
37         this.height = height;
38         this.depth = depth;
39         this.weight = weight;
40     }
41 }
```

Codice 4.14: Un esempio pratico di ereditarietà

Utilizzo di **super** per chiamare il costruttore della superclasse

Una sottoclasse può chiamare un costruttore definito dalla sua superclasse attraverso l'utilizzo della keyword **super**. La sua sintassi generale è la seguente:

```
1  super(arg-list);
```

Codice 4.15: Sintassi della keyword **super**

Qui, **arg-list** specifica gli argomenti che sono necessari al costruttore nella superclasse. **super()** deve essere sempre specificato come prima istruzione all'interno dei costruttori delle sottoclassi. Quando si crea un oggetto del tipo della sottoclasse per creare in memoria un qualcosa che estenda, siamo obbligati a creare un oggetto padre. La Java Virtual Machine crea la classe padre automaticamente, poi crea lo spazio per le variabili aggiuntive (le extend). È per questo motivo che il costruttore **super()** viene specificato come prima istruzione all'interno dei costruttori delle classi figlie.

Per vedere come funziona si osservi il listato 4.16:

```
1  class BoxWeight extends Box{
2      double weight;
3      BoxWeight(double width, double height, double depth, double weight){
4          super(width, height, depth);
5          this.weight = weight;
6      }
7  }
```

Codice 4.16: Utilizzo di **super** per chiamare il costruttore della superclasse

Analogamente alla keyword **this** è possibile utilizzare la keyword **super** per accedere ad un membro della superclasse che è stato oscurato da un membro della sottoclasse. La sua sintassi generale è la seguente:

`super.member;`

Questa seconda forma di **super** si applica nei casi in cui i nomi dei membri della superclasse sono stati oscurati o riscritti nella sottoclasse.

```
1 class A{
2     int i;
3 }
4 class B extends A{
5     int i;
6     B(int a, int b){
7         super.i = a; //i in A
8         i = b;
9     }
10    void show(){
11        System.out.println("i nella superclasse: "+super.i);
12        System.out.println("i nella sottoclasse: "+i);
13    }
14 }
```

Codice 4.17: Utilizzo di super per risolvere problemi di shadowing

4.5.3 Il polimorfismo in Java

Il **polimorfismo** è un altro dei capisaldi del paradigma ad oggetti ed è utilissimo per la riusabilità del codice: permette infatti di riutilizzare attributi e metodi di una classe esistente quando si crea una nuova classe. La parola **polimorfismo** deriva dal greco e significa “molte forme”. In un linguaggio di programmazione orientato agli oggetti possiamo ottenere il polimorfismo nel caso in cui si abbiano molte classi che sono legate tra loro dall’ereditarietà.

Java supporta due tipi di polimorfismo:

1. il polimorfismo orizzontale (o per metodi) che utilizza i meccanismi di override e overload dei metodi che, come vedremo più avanti, permettono ad uno stesso metodo di comportarsi in modo diverso a seconda dell’oggetto che lo invoca;
2. il polimorfismo verticale (o per classi, o per dati) che è dato dall’ereditarietà: grazie alla gerarchia che si ottiene, infatti, è possibile riferirsi con un riferimento della superclasse ad oggetti con forme diverse (credendo, però, di riferirsi ad un oggetto di tipo superclasse).

Esempio

Consideriamo il listato 4.13. È possibile scrivere quindi sia:

```
1 Impiegato i = new Impiegato();
```



che:

```
1 Impiegato i = new Quadro();
```



La variabile `i` può essere quindi usata per riferirsi a due oggetti distinti. Questo tipo di assegnazione prende anche il nome di **downcast**. Il compilatore, in entrambi i casi, crederà di riferirsi ad una istanza di `Impiegato`, non sarà pertanto legale operare con chiamate del tipo:

```
1 String reparto = i.getReparto();
```



anche se `i` contiene l’indirizzo di un oggetto di tipo `Quadro`.

4.5.4 L'operatore instanceof

Poiché è lecito scambiarsi oggetti usando riferimenti ai loro antenati (nella gerarchia ad albero), potrebbe essere a volte necessario sapere esattamente la forma dell’oggetto con cui si ha a che fare.

Per esempio, supponiamo che vi sia una gerarchia di classi:

```
1 public class Quadro extends Impiegato
```

```
2 public class Segretario extends Impiegato
```



Se si riceve un oggetto usando un riferimento ad `Impiegato`, esso potrebbe essere anche realmente un `Quadro` o un `Segretario`. Per saperlo si utilizza l'operatore `instanceof`:

```
1 public void faQualcosa ( Impiegato i ) {  
2     if ( i instanceof Quadro ) {  
3         // elabora un Quadro  
4     } else if ( i instanceof Segretario ) {  
5         // elabora un Segretario  
6     } else {  
7         // elabora un Impiegato qualunque  
8     }  
9 }
```



4.5.5 ■ La keyword `super`

La keyword `super` viene utilizzata in Java per riferirsi alla superclasse e ai suoi membri attraverso la dot notation. I membri accessibili sono anche quelli che la superclasse ha ereditato e non solo quelli esplicitamente definiti in essa.

Una sottoclasse può chiamare un costruttore definito dalla sua superclasse attraverso l'utilizzo della keyword `super`. La sua sintassi generale è la seguente:

```
1 super(arg-list);
```

Codice 4.18: Sintassi della keyword `super`

Qui, `arg-list` specifica gli argomenti che sono necessari al costruttore nella superclasse. `super()` deve essere sempre specificato come *prima istruzione all'interno dei costruttori delle sottoclassi*. Quando si crea un oggetto del tipo della sottoclasse per creare in memoria un qualcosa che estenda, siamo obbligati a creare un oggetto padre. La Java Virtual Machine crea la classe padre automaticamente, poi crea lo spazio per le variabili aggiuntive (le `extend`). Se, come prima linea nel proprio costruttore, non c'è né `this`², né `super`, il compilatore procede a porre una chiamata implicita a `super()`, ovvero al costruttore di default della superclasse. Se non esiste tale costruttore di default, perché sovrascritto da nuovi costruttori con argomenti, allora il compilatore genera un errore.

Per vedere come funziona si osservi il listato 4.19:

```
1 class Box{  
2     double width;  
3     double height;  
4     double depth;  
5  
6     Box(Box ob){    // costruttore di copia  
7         width = ob.width;  
8         height = ob.height;  
9         depth = ob.depth;  
10    }  
11  
12    Box(double width, double height, double depth){  
13        this.width = width;  
14        this.height = height;  
15        this.depth = depth;  
16    }  
17  
18    Box(){  
19        width = -1;  
20        height = -1;  
21        depth = -1;  
22    }  
23  
24    Box( double lenght){  
25        width = height = depth = lenght;  
26    }  
27  
28    double volume(){
```

²Per riferirsi ad un altro costruttore della classe stessa

```

29     return width * height * depth ;
30 }
31 }
32
33 ass BoxWeight extends Box{
34     double weight;
35     BoxWeight(double width, double height, double depth, double weight){
36         super(width, height, depth);    // utilizzo del costruttore della classe Box
37         this.weight = weight;
38     }

```

Codice 4.19: Utilizzo di super per chiamare il costruttore della superclasse

Analogamente alla keyword **this** è possibile utilizzare la keyword **super** per accedere ad un membro della superclasse che è stato oscurato da un membro della sottoclasse. La sua sintassi generale è la seguente:

super.member;

Questa seconda forma di **super** si applica nei casi in cui i nomi dei membri della superclasse sono stati oscurati o riscritti nella sottoclasse.

```

1  class A{
2      int i;
3  }
4  class B extends A{
5      int i;
6      B(int a, int b){
7          super.i = a; //i in A
8          i = b;
9      }
10     void show(){
11         System.out.println("i nella superclasse: "+super.i);
12         System.out.println("i nella sottoclasse: "+i);
13     }
14 }

```

Codice 4.20: Utilizzo di super per risolvere problemi di shadowing

4.5.6 ■ Overloading

In Java è possibile definire due o più metodi all'interno della **stessa classe** che condividono lo **stesso nome**, a patto che abbiano **argomenti diversi**. In questo caso si parla di **overloading** dei metodi o *sovraccaricamento*. L'overloading è uno dei modi in cui Java supporta il polimorfismo.

Quando viene invocato un metodo sovraccaricato, Java usa il tipo o il numero degli argomenti per determinare quale versione del metodo sovraccaricato bisogna chiamare. Per questo motivo, i metodi sovraccaricati *devono essere distinti nel tipo o nel numero dei loro parametri*. Il meccanismo attraverso il quale Java risolve una chiamata ad un metodo sovraccaricato prende il nome di **risoluzione del sovraccaricamento** oppure **dynamic method dispatch**.

```

1  class OverloadDemo{
2      void test(){
3          System.out.println("Nessun parametro");
4      }
5
6      //Effettuo l'overload del metodo test
7      void test(int a){
8          System.out.println("a: "+a);
9      }
10
11     //Effettuo l'overload del metodo test con due parametri
12     void test(int a, int b){
13         System.out.println("a e b: "+a +" "+b);
14     }
15
16     //Effettuo l'overload del metodo test con un solo parametro di tipo double
17     double test(double a){
18         System.out.println("a: "+a);
19         return a*a;
20     }
21 }
22

```

```

23 class Overload{
24     public static void main(String[] args){
25         OverloadDemo ob = new OverloadDemo();
26         double result;
27
28         //Chiamo tutte le versioni del metodo test()
29         ob.test();
30         ob.test(10);
31         ob.test(10,20);
32         result = ob.test(123.25);
33         System.out.println("Risultato di ob.test(123.25): "+result);
34     }
35 }

```

Codice 4.21: Esempio di overloading

Questo programma genera il seguente output:

Nessun parametro

a: 10

a e b: 10 20

double a: 123.25

Risultato di ob.test(123.25): 15190.5625

Il metodo `test()` viene sovraccaricato quattro volte. La prima versione non prende nessun parametro in input, la seconda prende un solo parametro intero, la terza me prende due mentre l'ultimo metodo prende un parametro di tipo `double` e ne restituisce il quadrato. Il fatto che l'ultima versione restituisca pure un valore non influisce sulle proprietà dell'overloading in quanto **il tipo di ritorno non gioca nessun ruolo nella risoluzione del sovraccaricamento**.

Overloading del costruttore

Nell'esempio mostrato in Figura 4.7 l'istruzione "`new Box()`" specifica una chiamata al costruttore della classe privo di argomenti. Questo è chiamato **costruttore di default**, anche detto **costruttore vuoto**, che viene incluso dalla JVM in ogni classe definita dal programmatore. Quando si dichiara una classe si possono fornire molti costruttori con parametri diversi per specificare l'inizializzazione degli attributi. Si effettua in questo modo l'**overloading del costruttore**.

```

1  class Box{
2      //Attributi
3      private double width;
4      private double height;
5      private double depth;
6      //Costruttori
7      Box(){} //Costruttore vuoto
8      Box(double w, double h, double d){ //Costruttore con tre parametri
9          width=w;
10         height=h;
11         depth=d;
12     }
13     //Metodi getter e setter
14     public double getWidth() {
15         return width;
16     }
17
18     public void setWidth(double w) {
19         width = w;
20     }
21
22     public double getHeight() {
23         return height;
24     }
25
26     public void setHeight(double h) {
27         height = h;
28     }
29
30     public double getDepth() {
31         return depth;
32     }
33
34     public void setDepth(double d) {
35         depth = d;
36     }

```

```

37     //Metodo Volume
38     void Volume(){
39         System.out.println("Volume ="width*height*depth);
40     }
41 }

```

Codice 4.22: Definizione di un costruttore con parametri

In questo modo sarà possibile inizializzare nuovi oggetti nella classe `STARTER` tramite il nuovo costruttore che prende tre parametri (`double w`, `double h`, `double d`):

```

1  class Starter{
2      public static void main(String [] args){
3          Box myBox = new Box(); //Creazione di un oggetto con il costruttore vuoto
4          Box myBox2 = new Box(2,3,4); //Creazione di un secondo oggetto con il costruttore definito nella
           classe Box
5      }
6  }

```



A differenza di un metodo, i **costruttori non restituiscono nulla** (nemmeno `void`). Ciò è dovuto al fatto che il tipo di ritorno implicito di un costruttore di una classe è il tipo di classe stessa. È compito del costruttore inizializzare lo stato interno di un oggetto affinché il codice che crea un'istanza abbia immediatamente un oggetto completamente inizializzato ed utilizzabile.

4.5.7 Override

L'**override** è un concetto molto simile all'overloading ma relativo all'ereditarietà e al polimorfismo. Attraverso di esso infatti una classe figlio può sovrascrivere un metodo ereditato e visibile, a patto che, rispetto al metodo della superclasse, il nuovo metodo abbia lo stesso nome, tipo di ritorno e lista di argomenti e visibilità non inferiore.

Ad esempio, si pensi a una superclasse chiamata `Animale` che ha un metodo chiamato `animalSound()`. Le sottoclassi di `Animal` potrebbero essere `Maiali`, `Gatti`, `Cani`, `Uccelli` e anch'esse hanno la loro implementazione di un suono animale:

```

1  class Animale{
2      public void Verso(){System.out.println("L'animale emette un verso");}
3  }
4
5  public class Gatto extends Animal{
6      @Override
7      public void Verso() {System.out.println("Miao miao");}
8  }
9
10 class Cane extends Animale{
11     @Override
12     public void Verso(){System.out.println("Bau Bau");}
13 }
14
15 public class Main {
16     public static void main(String[] args){
17         Animal myAnimal = new Animal();
18         Cane myCane = new Cane();
19         Gatto myGatto = new Gatto();
20         myAnimal = myCane;
21         myAnimal.Verso();
22         myAnimal = myGatto;
23         myAnimal.Verso();
24     }
25 }

```

Codice 4.23: Un primo esempio di polimorfismo

L'output sarà:

```

L'animale emette un verso
Bau bau
Miao miao

```

L'annotazione `@Override` dice al compilatore Java che vogliamo sovrascrivere un metodo dalla superclasse. Sebbene non sia necessario utilizzare `@Override` ogni volta che vogliamo implementarlo in un processo, è consigliato utilizzarlo perché è possibile commettere errori durante la creazione dei metodi. Ad esempio, potremmo fornire parametri diversi nella classe figlio, il che

finirebbe per essere un overload e non un override. Per superare l'errore, usiamo `@Override` sopra il nome del metodo nelle classi figlie che dice al compilatore che vogliamo sovrascrivere il metodo. Se commettiamo un errore, il compilatore genererà un errore.

Da come si può osservare nel Listato 4.23, è possibile assegnare un riferimento di ogni classe derivata ad una variabile riferimento del tipo della superclasse. Infatti, attraverso la variabile `myAnimal` è stato possibile invocare il metodo `Verso` delle classi `Cane` e `Gatto`³. Sarebbe un errore però pensare di potere accedere a tutti i membri della sottoclasse se questi non sono stati definiti nella superclasse.

È importante comprendere che è il tipo della variabile riferimento e non il tipo dell'oggetto puntato che determina a quali membri è possibile accedere. Quando si assegna un riferimento di una sottoclasse ad una variabile del tipo della superclasse infatti, si avrà accesso solo a quelle parti definite nella superclasse. Se ci pensiamo questa cosa ha perfettamente senso poiché non è dato sapere alla classe quali sono le parti in più che sono state definite nelle sue sottoclassi. Questo concetto, insieme all'override dei metodi, sta alla base del **polimorfismo** e consente quindi in questo modo di operare attraverso la superclasse e ottenere comportamenti diversi sulla base dell'implementazione dei metodi sovrascritti nelle classi derivate.

Esempio

Grazie al polimorfismo possiamo ottenere collezioni eterogenee di oggetti come nel seguente caso:

```
1 public class Animale {
2     public void emettiVerso() {
3         System.out.println("Gli animali emettono un verso");
4     }
5 }
6
7 public class Cane extends Animale {
8     @Override
9     public void emettiVerso() {
10         System.out.println("Bau bau");
11     }
12 }
13
14 public class Gatto extends Animale {
15     @Override
16     public void emettiVerso() {
17         System.out.println("Miao miao");
18     }
19 }
20
21 public class Main {
22     public static void main(String[] args) {
23         Animale[] zoo = new Animale[4];
24         Cane cane1 = new Cane();
25         Gatto gatto1 = new Gatto();
26         Cane cane2 = new Cane();
27         Gatto gatto2 = new Gatto();
28
29         zoo[0] = cane1;
30         zoo[1] = gatto1;
31         zoo[2] = gatto2;
32         zoo[3] = cane2;
33
34         for(Animale param: zoo) {
35             param.emettiVerso();
36         }
37     }
38 }
```

Codice 4.24: Secondo esempio polimorfismo

Output:

```
Bau bau
Miao miao
Miao miao
Bau bau
```

³Modificando preventivamente l'indirizzo dell'oggetto puntato

4.5.8 Casting di tipi reference

I riferimenti, come i primitivi, partecipano alla conversione automatica per assegnamento (e per passaggio di parametri) e al casting. A differenza dei primi però *non c'è promozione aritmetica tra tipi riferimento* poiché i riferimenti non possono essere operandi aritmetici. Nella conversione e nel casting di riferimenti vi sono maggiori combinazioni tra vecchi e nuovi tipi (e quindi maggior numero di regole).

La conversione automatica di riferimenti avviene *nella fase di compilazione*, poiché il compilatore ha tutte le informazioni per determinare se la conversione è legale. Essa può essere **forzata con un casting esplicito** come per i primitivi. In questo caso, può accadere che, sebbene la conversione sia autorizzata come legale dal compilatore, il tipo effettivo dell'oggetto riferito non sia compatibile in esecuzione. Infatti in una assegnazione entrano in gioco tre fattori: il tipo dei due riferimenti e quello proprio dell'oggetto.

Conversioni automatiche Nella conversione automatica di riferimenti (per ampliamento, in una assegnazione o in un passaggio di parametri), vengono nascoste alcune caratteristiche dell'oggetto riferito. Qui il tipo di nascita dell'oggetto non interessa; i riferimenti possono essere:

- Ad una classe;
- Ad una interfaccia;
- Ad un array.

La conversione può avvenire nel modo seguente:

```
1 Oldtype x = new Oldtype();
2 Newtype y = x;
```



	Oldtype è una classe	Oldtype è un'interfaccia	Oldtype è un array
Newtype è una classe	Oldtype deve essere una sottoclasse di Newtype	Newtype deve essere Object	Newtype deve essere Object
Newtype è una interfaccia	Oldtype deve implementare l'interfaccia Newtype	Oldtype deve essere una sottointerfaccia di Newtype	Newtype deve essere Cloneable o Serializable
Newtype è un array	Errore di compilazione	Errore di compilazione	I tipi delle componenti dei due array devono essere riferimenti auto-convertibili.

Tabella 4.2: Regole per la conversione automatica tra tipi reference

A proposito di overloading e overriding (vedi 4.5.6), in attinenza alle conversioni automatiche di riferimenti, la scelta del metodo da associare ad una invocazione avviene:

- in *compilazione* per i metodi sovraccaricati (basandosi sul tipo dei riferimenti nei parametri);
- in *esecuzione* per i metodi sovrapposti (basandosi sulla effettiva presenza e visibilità del metodo nell'oggetto).

Quindi, se si considera il seguente codice:

```
1 public class A {}
2
3 public class B extends A{
4     public void g(A a){
5         System.out.println("A");
6     }
7 }
8
9 public class UseAB extends B{
10     public void f(A a){
11         System.out.println("A");
12     }
13     public void f(B b){
14         System.out.println("B");
15     }
16     public void g(A b){
17         System.out.println("B");
18     }
19
20     public void metodo() {
```

```

21     A a = new A();
22     B b = new B();
23     A x = b;    // automatic upcasting
24     f(a);
25     f(b);
26     f(x);
27     f(new B());
28     g(a);
29     g(b);
30     g(x);
31     g(new B());
32     super.g(a);
33     super.g(b);
34     super.g(x);
35 }
36
37 public static void main(String[] args) {
38     UseAB u = new UseAB();
39     u.metodo();
40 }
41 }

```



Output:

A B A B B B B B A A A

Casting esplicito Una volta appurato quali sono le conversioni che il compilatore accetta di fare da sè (quelle per assegnazione, o passaggio di parametri, di ampliamento) possiamo passare a studiare le conversioni che il programmatore chiede esplicitamente (casting).

- Un **cast di ampliamento**, la cui conversione sarebbe avvenuta anche senza di esso, viene autorizzato e non causa problemi né in compilazione né in esecuzione;
- Detto rozzamente, un **upcast** (un casting di riferimenti verso l'alto), nella gerarchia di ereditarietà, è sempre sia implicitamente che esplicitamente legale;
- Purtroppo, per i riferimenti e a differenza dei primitivi, nel **downcasting** (ovvero un casting da una superclasse ad una sottoclasse), il compilatore non può aiutare completamente e può accadere che, sebbene la conversione sia autorizzata come legale, essa fallisca in esecuzione. Infatti, nel casting esplicito, entrano in gioco tre tipi: quello dei due riferimenti (a sinistra e a destra dell'assegnazione e quello dell'oggetto (il tipo di nascita)).

Il casting esplicito avviene quando:

```

1 Newtype nt;
2 Oldtype ot;
3 nt = (Newtype) ot;

```



Le regole applicate dal compilatore sono illustrate nella tabella 4.3.

Tabella 4.3: Regole per il cast esplicito tra tipi reference

	Oldtype è una classe non-final	Oldtype è una classe final	Oldtype è una interfaccia	Oldtype è un array
Newtype è una classe non final	Oldtype deve estendere Newtype o viceversa	Oldtype deve estendere Newtype	Sempre OK	Newtype deve essere Object
Newtype è una classe final	Newtype deve estendere Oldtype	Oldtype e Newtype devono coincidere	Newtype deve implementare l'interfaccia o implementare Serializable	Errore di compilazione
Newtype è una interfaccia	Sempre OK	Oldtype deve implementare l'interfaccia Newtype	Sempre OK	Errore di compilazione
Newtype è un array	Oldtype deve essere Object	Errore di compilazione	Errore di compilazione	I tipi delle componenti dei due array devono essere riferimenti convertibili per cast.

4.6

FILE JAVA, PACKAGE E CARTELLE



4.6.1 ■ File sorgenti

In Java, un file sorgente contiene il codice della classe. Un esempio di file sorgente potrebbe apparire come segue:

```

1 package mypackage;
2
3 import java.util.List;
4 import java.io.*;
5
6 public class MyClass {
7     public static void main(String[] args) {
8         System.out.println("Hello, World!");
9     }
10 }
```

Codice 4.25: Struttura di un file Java

Il nome di un file sorgente deve essere lo stesso dell'unica classe pubblica contenuta nel file; se il file sorgente non contiene classi pubbliche allora il nome del file può essere qualsiasi.

4.6.2 ■ I package

Un *package* in Java è un meccanismo per organizzare le classi correlate in una singola unità. Un file sorgente può dichiarare il suo package utilizzando la keyword **package**. Ad esempio:

```

1 // MyClass.java
2 package com.example;
3
4 public class MyClass {
5     // ...
6 }
```

Codice 4.26: I package in Java

In questo esempio, il package è dichiarato come **com.example**. La convenzione è utilizzare il formato di dominio inverso (**inverted domain name**) per evitare collisioni di nomi. I file sorgenti appartenenti a uno specifico package devono essere inseriti in una struttura di directory che rifletta la gerarchia dei package. Ad esempio, se il package è dichiarato come **package com.example;**, il file sorgente deve essere inserito in una cartella **com/example** nella gerarchia delle cartelle.

Se non viene fornita alcuna dichiarazione di package, la classe appartiene al cosiddetto **default package**. Tuttavia, è una pratica consigliata sempre dichiarare un package per evitare potenziali problemi di collisione di nomi.

I nomi dei package devono rispettare le seguenti regole:

1. I nomi di package sono composti da lettere minuscole.
2. Evita caratteri speciali e spazi nei nomi dei package.
3. Possono contenere lettere, numeri e underscore.

Per utilizzare classi da altri package, si utilizza la keyword `import`:

```
1 // AnotherClass.java
2 package com.example;
3
4 import com.example.MyClass;
5
6 public class AnotherClass {
7     public static void main(String[] args) {
8         MyClass myObject = new MyClass();
9         // ...
10    }
11 }
```

Codice 4.27: La keyword `import`

4.6.3 ■ Cartelle

Per mantenere il codice sorgente organizzato, è consigliabile utilizzare una struttura ad albero per il progetto. Ecco un esempio pratico:

```
MyProject
├-- src
│   ├── com
│   │   └-- example
│   │       ├── MyClass.java
│   │       └-- AnotherClass.java
└-- bin
```

Codice 4.28: Struttura gerarchica di un progetto Java

La struttura prevede due directory principali:

- **src:** Contiene i file sorgenti del progetto organizzati secondo la gerarchia dei package.
- **bin:** Contiene i file compilati, mantenendo la stessa struttura di directory di `src`.

Normalmente, infatti, il compilatore pone i file `.class` nella stessa cartella dei file sorgenti. Per reindirizzare i compilati nella cartella `bin` si utilizza l'opzione `-d` del comando `javac`.

Esempio

Si consideri la classe `MyClass` presente nella cartella `src/com/example`:

```
1 package com.example;
2
3 public class MyClass {
4     public static void main(String[] args) {
5         System.out.println("Hello, World!");
6     }
7 }
```



Per salvare il compilato nella classe `bin` sarà necessario compilare nel seguente modo:

```
1 $ javac -d bin src/com/example/MyClass.java
2 # Se si sta compilando un insieme di file all'interno della gerarchia dei pacchetti (cioè non nella
   cartella principale dei sorgenti), allora bisogna anche specificare l'opzione -sourcepath:
3 $ javac -sourcepath MyProject/src/com/example -d /MyProject/bin *.java
```





Il linguaggio Java fornisce un ambiente molto ricco di operatori. La maggior parte di essi può essere suddiviso in quattro gruppi:

- Aritmetici
- Logici
- Relazionali
- Bitwise

4.7.1 ■ Operatori logici

Gli operatori logici operano solo su operandi di tipo booleano. Tutti gli operatori di questo tipo sono duali e restituiscono un valore booleano.

```

1 public class BooleanOperators {
2     public static void main(String[] args) {
3         boolean a = true;
4         boolean b = false;
5         boolean c = a | b;
6         boolean d = a & b;
7         boolean e = a ^ b;
8         boolean f = !(a & b) | (a & !b);
9         boolean g = !a;
10        System.out.println("a = " + a);
11        System.out.println("b = " + b);
12        System.out.println("a|b = " + c);
13        System.out.println("a&b = " + d);
14        System.out.println("a^b = " + e);
15        System.out.println("!(a&b) | (a & !b) = " + f);
16        System.out.println("!a = " + g);
17    }
18 }
```

Codice 4.29: Operatori booleani

Output:

```

a = true
b = false
a|b = true
a&b = false
a^b = true
!(a&b) | (a & !b) = true
!a = false
```

Java fornisce inoltre due operatori chiamati **operatori booleani con corto circuito**: `||` (OR), `&&` (AND). Questi operatori permettono di risparmiare in overhead seguendo la definizione di operazione logica: infatti l'OR logico restituisce sempre vero quando il primo dei due operatori è vero, a prescindere dal valore del secondo operando e allo stesso modo l'AND logico restituisce sempre falso quando il primo operatore è falso. Usando le forme `||` e `&&` al posto degli operatori `|` e `&` si può far evitare alla JVM di calcolare tutta l'espressione. Questo è molto utile quando l'operando a destra dipende dal valore dell'operando di sinistra per funzionare in modo appropriato. Ad esempio, nell'istruzione:

```
if ( denom != 0 && num/denom > 10)
```

dato che si sta usando la forma corto circuito dell'operatore AND, non c'è rischio di incorrere in eccezioni a tempo di esecuzione in quanto, quando `denom` è zero, l'operatore restituirà automaticamente `false` senza dover calcolare il secondo operando che genererebbe l'eccezione.

4.7.2 ■ Operatori bitwise

Java definisce vari operatori bitwise che possono applicati ai tipi interi: `long`, `int`, `short`, `char` e `byte`. Questi operatori agiscono sui singoli bit dei loro operandi. Questi sono elencati nella Tabella 4.4.

Tutti i tipi di numeri interi sono rappresentati da numeri binari di larghezza variabile. Ad esempio, il valore della variabile

```
byte a = 42;
```

Operatore	Risultato
~	Bitwise unary NOT
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR
>>	Shift right
<<	Shift left
>>>	Shift right zero fill

Tabella 4.4: Operatori bitwise in Java

è 00101010, dove ogni posizione rappresenta una potenza di due, a partire da 2^0 nel bit più a destra. Tutti i tipi di interi (tranne i `char`) sono interi con segno. Ciò significa che possono rappresentare valori negativi e positivi. Java utilizza la codifica del complemento a due per la rappresentazione dei numeri relativi, il che implica che i numeri negativi sono rappresentati invertendo (cambiando gli 1 in 0 e viceversa) tutti i bit e aggiungendo poi 1 al risultato. Ad esempio, -42 viene rappresentato invertendo tutti i bit di 42, o 00101010, che dà come risultato 11010101, e infine aggiungendo 1, che dà come risultato 11010110, ovvero -42.

Il NOT bitwise Anche chiamato **complemento bitwise**, l'operatore unario NOT (~) inverte tutti i bit del suo operando. Ad esempio:

```
1 a = 00101010 = 42
2 ~a = 11010101 = -43
```



L'AND bitwise L'operatore AND, &, produce un 1 se entrambi gli operatori hanno 1. Viene restituito uno zero negli altri casi. Ad esempio:

```
1 a = 00101010 = 42
2 b = 00001111 = 15
3 a&b = 00001010 = 10
```



L'OR bitwise L'operatore OR, |, produce un 1 se almeno un bit tra i due operatori è 1. Viene restituito uno zero negli altri casi. Ad esempio:

```
1 a = 00101010 = 42
2 b = 00001111 = 15
3 a|b = 00101111 = 47
```



Lo XOR bitwise L'operatore XOR, ^, restituisce 1 solo quando uno dei due operandi è uno, zero altrimenti. Ad esempio:

```
1 a = 00101010 = 42
2 b = 00001111 = 15
3 a^b = 00100101 = 37
```



Shift a sinistra L'operatore shift a sinistra, <<, scorre n bit a sinistra. Gli spazi vuoti a destra sono riempiti con degli zeri, mentre quelli che escono a sinistra sono eliminati. Questa operazione equivale a **moltiplicare** il numero di partenza per una potenza di due. Infatti:

```
1 int a = 60; // = 0011 1100
2 int c = a<<2; // = 1111 0000 ossia c = 60*2^2 = 60*4 = 240
```



Shift a destra L'operatore shift a destra, `>>`, scorre n bit a destra. Gli spazi vuoti a sinistra sono riempiti con degli zeri mentre quelli che escono a destra **sono eliminati**. Questa operazione equivale a **dividere** il numero di partenza per una potenza di due. Nello shift a destra aritmetico **il bit di segno viene preservato**, infatti:

```
1 int a = 60;    // = 0011 1100
2 int c = a >> 2; // = 0000 1111 ossia c = 15 = 60/4
```



Shift a destra logico L'operatore shift a destra logico o senza segno, `>>>`, sposta i bit a destra di n posizioni. La differenza tra shift logico e aritmetico è la *gestione del bit di segno*. L'operatore `>>>` infatti lavora solo con interi positivi. Se l'operando di sinistra è un `int`, prima di applicare lo shift si riduce l'operando di destra modulo 32; se l'operando di sinistra è un `long`, prima di applicare lo shift si riduce l'operando di destra modulo 64. L'operatore `>>>` è ammesso solo per i tipi interi `int` e `long`; se viene usato per `short` o `byte` questi valori sono promossi, con estensione di segno, ad `int` prima di applicare lo shift. Ad esempio:

```
1 a = 128 = 1000 0000
2 128 >> 1 = 64 = 0100 0000
3 128 >> 3 = 16 = 0001 0000
4 -128 >> 3 = -16 = 1111 0000
5 -1 >>> 30 = 3 = 0000 0011
6 23 >>> 64 = 23 = 0001 0111
7 60 >>> 2 = 15 = 0000 1111
```



4.8

ENUNCIATI JAVA



4.8.1 ■ Enunciati branch

I blocchi `if ...else`

In Java la sintassi dei blocchi condizionali `if else` è la seguente:

```
1 if ( < boolean expression > ) {
2   < statement >*
3 }
4 // oppure
5 if ( < boolean expression > ) {
6   < statement >*
7 } else if ( < boolean expression > ) {
8   < statement >*
9 } else {
10  < statement >*
11 }
```

Codice 4.30: Blocco condizionale `if else`

Attenzione



Attenzione all'espressione condizionale: deve essere una espressione booleana, non un intero come nel linguaggio C o C++.

Il blocco `switch`

La sintassi del blocco `switch` è la seguente:

```
1 switch ( < expr > ) {
2   case < constant1 >:
3     < statement >*
4     break ;
5   case < constant2 >:
6     < statement >*
7     break ;
8   default :
```

```

9    < statement >*
10   break ;
11 }

```

Codice 4.31: Blocco condizionale **switch**

Attenzione

Se non è presente l'enunciato opzionale **break** l'esecuzione continua nella stanza successiva.

4.8.2 Enunciati loop

I cicli **for**

Si ha la seguente sintassi:

```

1  for ( <init_expr>; <bool_expr>; <alt_expr> ) {
2    < statement >*
3  }

```

Codice 4.32: Sintassi ciclo **for**

Esempio

```

1  for ( int i =0; i <10; i ++ ) {
2    System.out.println("Sono nel loop");
3  }
4  System.out.println("Fine");

```

I cicli **while**

Si ha la seguente sintassi:

```

1  while ( < bool_expr > ){
2    < statement >*
3  }

```

Codice 4.33: Sintassi ciclo **while**

Esempio

```

1  int i =0;
2  while ( i<10 ) {
3    System.out.println("Sono nel loop");
4    i++;
5  }
6  System . out . println ( " Fine " );

```

I cicli **do ...while**

Si ha la seguente sintassi:

```

1  do {
2    < statement >*
3  } while ( < bool_expr > );

```

Codice 4.34: Sintassi ciclo **do while**

Esempio

```
1 int i = 0;
2 do {
3     Sytem.out.println ("Sono nel loop");
4     i++;
5 } while (i < 10);
6 System.out.println (" Fine ");
```



Tabella 4.5: Controlli speciali

Istruzione	Descrizione
<code>break [label];</code>	Usata per uscire in modo prematuro da un enunciato <code>switch</code> , da enunciati di loop o da blocchi etichettati.
<code>continue [label];</code>	Usata per saltare direttamente alla fine del corpo di un loop.
<code>label: <statement></code>	Usata per identificare un qualunque enunciato verso il quale può essere trasferito il controllo.

4.9

GLI ARRAY IN JAVA



Gli **array** sono strutture dati composte da dati omogenei, chiamati **componenti** o **elementi**, aventi lo stesso tipo ai quali ci riferiamo attraverso lo stesso nome comune. Gli array in Java sono **oggetti**, sono considerati quindi tipi reference. Per riferire un elemento particolare dell'array, è necessario specificare il nome del riferimento all'array e il *numero di posizione* dell'elemento desiderato nell'array. Gli array, inoltre, sono entità a lunghezza fissa. Ciò significa che essi rimangono della stessa lunghezza una volta creati, anche se un riferimento a un array può essere riassegnato ad un nuovo array di dimensioni diverse.

4.9.1 Dichiarazione di un array monodimensionale

Si possono dichiarare array di tipi primitivi o di riferimenti ad oggetti:

```
1 char s[];
2 Point p[];
3 char[] s;
4 Point[] p;
```

Codice 4.35: Dichiarazione di array in Java

Nella fase di dichiarazione non viene creato nessun oggetto in quanto l'allocazione dello spazio richiede l'uso dell'operatore **new** come mostrato nel Listato 4.36 (Figura 4.9a). Un array infatti è un oggetto: le dichiarazioni precedenti creano solo il riferimento al rispettivo oggetto.

```
1 public char[] creaArray() {
2     char[] s;
3
4     s = new char[26];
5     for ( int i = 0; i < 26; i++){
6         s[i] = (char) ('A'+i);
7     }
8     return s;
9 }
```

Codice 4.36: Creazione di un array in Java

L'istruzione

```
s = new char[26];
```

crea un array di 26 caratteri: essi sono inizializzati al loro valore di default. I valori sono accessibili nel range da 0 a 25 e ogni tentativo di accesso oltre questo range causa il lancio di una eccezione a runtime. Oltre agli array di tipi primitivi è possibile dichiarare anche array di oggetti come mostrato nel Listato 4.37.

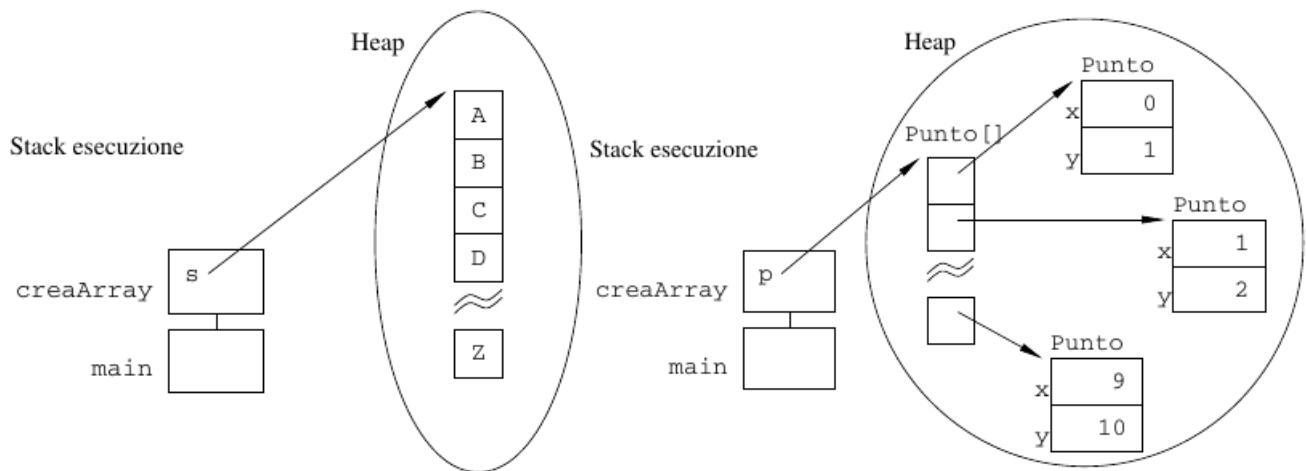
```
1 public Punto[] creaArray(){
2     Punto[] p;
```

```

3  p = new Punto[10];
4  for ( int i = 0 ; i < 10 ; i++)
5  p[i] = new Punto (i , i+1);
6  return p;
7  }

```

Codice 4.37: Creazione di un array di oggetti di tipo Punto



(a) Stato dell'heap in fase di chiamata a `creaArray()` (b) Stato dell'heap in fase di creazione del vettore `Punto [] p`

4.9.2 ■ Inizializzazione degli array

Poiché l'inizializzazione delle variabili è cruciale, Java fornisce due metodi abbreviati per gli array. Il primo consiste nella dichiarazione, costruzione ed inizializzazione in una singola riga:

```

1 String [] nomi = {"Antonio", "Marcello", "Anna"};

```

Codice 4.38: Inizializzazione di un array inline

equivalente a:

```

1 String [] nomi;
2 nomi [0] = "Antonio";
3 nomi [1] = "Marcello";
4 nomi [2] = "Anna";

```



Il secondo metodo è attraverso la costruzione ed inizializzazione di un array anonimo. Ad esempio:

```

1 int [] esempio;
2 esempio = new int [] {4,7,6};

```

Codice 4.39: Inizializzazione di un array anonimo

o, ancora meglio:

```

1 public class A{
2     void prendiArray ( int [] unArray) { // usa l'array }
3     public static void main (String [] args){
4         A a = new A ();
5         a.prendiArray ( new int[] {3,4,5,6,7});
6     }
7 }

```



Attenzione



Non si deve specificare la dimensione durante la creazione di un array anonimo.

Esempio

In Java tutti i parametri dei metodi sono di tipo **IN realizzati per copia**. Questo significa che il valore del parametro nel metodo chiamante non può essere modificato. Tuttavia è possibile creare un riferimento a tale parametro e realizzare ugualmente la modifica nel metodo chiamante. Per esempio, per creare un riferimento ad un tipo primitivo:

```
1 public class PrimitiveReference{
2     public static void main (String [] args){
3         int [] mioValore = {1};
4         modifica (mioValore);
5         System.out.println("mioValore contiene: " + mioValore[0]);
6     }
7
8     public static void modifica (int [] valore) {
9         valore[0]++;
10    }
11 }
```



4.9.3 ■ Array multidimensionali

In Java, gli **array multidimensionali** sono considerati come degli array monodimensionali che contengono altri array. Per dichiarare una variabile di un array multidimensionale bisogna specificare ogni indice aggiuntivo inserendo altre parentesi quadre. Ad esempio, l'istruzione che segue dichiara una variabile chiamata **dueDim** per un array bidimensionale:

```
1 int dueDim [][] = new int [3][];
2 dueDim[0] = new int [5];
3 dueDim[1] = new int [5];
4 dueDim[2] = new int [5];
```

Codice 4.40: Dichiarazione di un array bidimensionale

che è equivalente alla dichiarazione:

```
1 int dueDim[][] = new int [3][5];
```



Questa istruzione alloca un array di grandezza 4×5 elementi e la assegna alla variabile **twoD**. Internamente, questa **matrice** viene implementata come un array di array di interi. È possibile però dichiarare anche array non rettangolari:

```
1 int dueDim [][] = new int [3][];
2 dueDim[0] = new int [2];
3 dueDim[1] = new int [5];
4 dueDim[2] = new int [8];
```

Codice 4.41: Dichiarazione di un array bidimensionale non rettangolare

```
1 int [][] multiD = {{5,4,3,2}, {9,8}, {7,6,5}}
```

Codice 4.42: Costruzione ed inizializzazione

4.9.4 ■ La classe Arrays

Come già detto in anticipo, gli array in Java sono dei veri e propri oggetti e come tali hanno una loro classe. La classe **Arrays** è stata pensata per evitare di dover reinventare ripetutamente la ruota del carro per scrivere metodi per la manipolazione dei vettori.

L'attributo length

Il numero di elementi è parte dell'oggetto Array. Questa funzionalità viene fornita dall'attributo **length** della classe Arrays. Il listato 4.43 mostra i vantaggi dell'attributo length.

```
1 class Length{
2     public static void main(String[] args){
3         int[] a1 = new int[10];
4         int[] a2 = {3, 5, 6, 7, 8, 44, -10};
5         int[] a3 = {4, 3, 2, 1};
```

```

6
7     System.out.println("Lunghezza di a1: "+a1.length);
8     System.out.println("Lunghezza di a2: "+a2.length);
9     System.out.println("Lunghezza di a3: "+a3.length);
10 }
11 }

```

Codice 4.43: Dimostrazione dell'attributo `length` nella classe `Arrays`

Il programma mostra il seguente output:

```

Lunghezza di a1: 10
Lunghezza di a2: 7
Lunghezza di a3: 4

```

Come si può vedere, viene mostrata la lunghezza di ciascun vettore. Bisogna tenere a mente che il valore di `length` non ha nulla a che vedere con il numero di elementi che sono in uso nel vettore. Rappresenta solo il numero di elementi che il vettore può contenere al massimo.

È possibile osservare uno dei tanti vantaggi dell'utilizzo della variabile d'istanza `length` all'interno dei cicli sui vettori. Mentre in C è sempre necessario specificare la dimensione dell'array mediante un numero, rischiando di incorrere in errori in caso di modifiche dello stesso, in Java il problema è risolvibile passando come parametro il valore di `length`:

```

1 for ( int i = 0; i < array.length ; i++)
2 // Statement

```



4.9.5 ■ Assegnazione

In una assegnazione l'array deve avere lo stesso numero di dimensioni della variabile di riferimento. Per esempio:

```

1 public class ArrayTest {
2     public static void main(String[] args) {
3         int [] vettore;
4         int [][] matrice = new int[3][];
5         vettore = matrice; // Errore
6         int [] v = new int[3];
7         vettore = v;      // OK
8     }
9 }

```



produce il seguente errore:

```
ArrayTest.java:5: error: incompatible types: int[][] cannot be converted to int[]
```

4.9.6 ■ Copia di array: il metodo `arraycopy()`

Il metodo `arraycopy()` copia i valori contenuti negli elementi dell'array. Nel caso di array di oggetti (o di array multidimensionali), ciò significa che vengono copiati i riferimenti agli oggetti, non vengono cioè create nuove copie di oggetti.

```

1 // array originale
2 int vecchio [] = {1,2,3,4};
3 // nuovo array piu' lungo
4 int nuovo [] = {20,32,43,5423,5454,342,445};
5 // copia tutti i vecchi elementi nel nuovo array
6 System.arraycopy(vecchi, 0, nuovo, 0, vecchi.length);

```

Codice 4.44: Il metodo `arraycopy`



Nella sezione 4.4 era stata accennata la sequenza che avviene ogni volta che si costruisce ed inizializza un oggetto. Di seguito, noto il funzionamento dei costruttori, possiamo elencare la sequenza completa:

- Viene allocata nello heap, con l'operatore **new**, la memoria per il nuovo oggetto e sono inizializzate le variabili di istanza ai valori impliciti di default.
- Per ogni successiva chiamata ad un costruttore, cominciando dal costruttore individuato per primo, viene eseguita la sequenza di passi:
 1. Assegna i parametri del costruttore;
 2. Se è presente, come prima riga, una chiamata esplicita a **this(...)**, richiama ricorsivamente il nuovo costruttore e poi passa al punto 5;
 3. Richiama l'implicito o esplicito **super(...)**-costruttore, eccetto per **Object**, poiché **Object** non ha superclasse;
 4. Esegue ogni inizializzazione esplicita delle variabili di istanza;
 5. Esegue il corpo del costruttore corrente.

Eseguiamo l'istruzione:

```
new Quadro ("M 7","vendite");
```

dove:

```

1 public class Impiegato{
2     private String nome;
3     private double stipendio = 15000.00;
4     private Date dataNascita;
5
6     public Impiegato (String n, Date d) {
7         nome = n;
8         dataNascita = d;
9     }
10
11     public Impiegato (String n) {
12         this (n, null);
13     }
14 }
15
16 public class Quadro extends Impiegato {
17     private String reparto;
18
19     public Quadro (String n, String r) {
20         super (n );
21         reparto = r;
22     }
23 }
```



Se non ci fosse stato il richiamo esplicito al supercostruttore al rigo 20 il compilatore avrebbe messo in automatico un richiamo al costruttore vuoto della superclasse in quanto, forzando il costruttore, si obbliga di fatto ad inizializzare le variabili di istanza. Non essendo presente in **Impiegato**, però, alcun costruttore vuoto allora si sarebbe generato un errore di compilazione. Quindi data l'istruzione:

```
new Quadro ("M 7","vendite");
```

si ha:

1. Inizializzazione base (**new...**):
 - (a) Allocazione memoria per un oggetto **Quadro**
 - (b) Inizializzazione di **reparto** al valore di default **null**
2. Esecuzione del costruttore: **Quadro ("M 7", "vendite")**:
 - (a) Inizializzazione dei parametri del costruttore: **n= "M 7", r="vendite"**.
 - (b) Non c'è chiamata a **this(...)**;
 - (c) Invocazione di **super("M 7")** come istanza di **Impiegato(String n)**.
 - i. Inizializzazione del parametro del costruttore: **n = "M 7"**
 - ii. Esecuzione di **this("M 7", null)** come istanza di **Impiegato(String n, Date d)**
 - A. Inizializzazione dei parametri del costruttore: **n= "M 7", d = null**

- B. Nessuna chiamata esplicita a `this(...)`
 - C. Nessuna chiamata esplicita a `super(...)`: viene eseguito implicitamente `super()` come istanza di `Object`
 - Nessuna inizializzazione di parametri
 - Nessuna chiamata a `this(...)`
 - Nessuna chiamata a `super()`
 - Nessuna inizializzazione esplicita
 - Nessun corpo da eseguire
 - D. Inizializzazione esplicita della variabile: `stipendio = 15000.00`
 - E. Esecuzione del corpo del costruttore: `nome="M 7", dataNascita = null`
- iii. Saltato
- iv. Saltato
- v. Nessun corpo del costruttore: `nome= "M 7", dataNascita = null`
- (d) Saltato
- (e) Saltato
- (f) Nessun corpo in `Impiegato(String n)`
3. Nessuna inizializzazione esplicita per `Quadro`;
4. Esecuzione di `reparto = "Vendite"`.

4.11

LA CLASSE OBJECT



In Java esiste una classe speciale, chiamata **Object** dalla quale discendono tutte le classe ereditandone i metodi. Questo significa che una variabile riferimento di tipo `Object` può riferirsi ad un oggetto di qualsiasi classe.

La classe `Object` definisce i seguenti metodi, i quali quindi sono disponibili in ogni oggetto.

Metodo	Funzione
<code>Object clone()</code>	Crea un nuovo oggetto che è lo stesso dell'oggetto passato come argomento.
<code>boolean equals(Object object)</code>	Determina se un oggetto è uguale ad un altro.
<code>String toString()</code>	Restituisce una stringa che descrive l'oggetto.

4.11.1 Il metodo equals

In Java, per verificare che due oggetti siano uguali non possiamo usare l'operatore di confronto `==` in quanto quest'ultimo è applicabile solo ai tipi primitivi. Applicato a due variabili di tipi reference esso determina se i due riferimenti sono identici, cioè se riferiscono allo stesso, unico, oggetto. Il metodo `equals` determina se le due variabili si riferiscono ad oggetti (anche distinti) appartenenti alla stessa classe di equivalenza, rispetto ad una particolare relazione di equivalenza definita dall'utente. La realizzazione di `equals` nella classe `Object` fa uso di `==`. Quindi le classi di equivalenza di `==` e di `equals` coincidono in `Object`. Lo scopo del metodo è quello di essere ridefinito nelle classi sviluppate dagli utenti, in modo da realizzare nuove relazioni di equivalenza. Alcune classi di sistema lo fanno già. Per esempio la classe `String` realizza il proprio `equals`, confrontando le stringhe carattere per carattere. Viene raccomandato di sovrapporre, insieme con `equals`, anche il metodo `hashCode`.

4.11.2 La classe String

A differenza del C e del C++, dove il tipo di dati stringa non esiste (vengono usati array di `char`), in Java è stata introdotta la classe **String** che offre moltissimi metodi per la gestione delle sequenze di caratteri. `String` ha diversi costruttori, i più utili sono:

- `String()`: crea un oggetto `String` con valore la stringa vuota `""`;
- `String(String original)`: crea un oggetto `String` con i caratteri della stringa argomento;

- **String(char [] value)**: crea un oggetto **String** con i caratteri contenuti nell'array di char;
- **String(char[] value, int offset, int count)**: crea un oggetto **String** con una parte dei caratteri contenuti nell'array (count caratteri a partire da offset).

Il modo più comune di creare una stringa, però, è assegnare una *costante stringa* a una variabile di tipo **String**.

```
1 String stringa = "stringa";
```



che corrisponde all'uso di un costruttore:

```
1 String stringa = new String("stringa");
```



In pratica per ogni sequenza di caratteri tra virgolette viene creato un oggetto **String** inizializzato con il valore tra virgolette; questo consente di impiegare una costante stringa ovunque sia richiesto un oggetto della classe **String**. Dopo che un oggetto **String** è stato creato viene trattato come costante; nessun metodo di questa classe modifica l'oggetto stringa su cui opera: **le stringhe sono oggetti immutabili**⁴. Per ogni operazione che sembra modificare un oggetto **String**, in realtà viene creato un nuovo oggetto **String**, mentre l'oggetto originale resta inalterato.

Va ricordato che l'operatore **==** funziona sui tipi di dato primitivo e ne confronta il valore. Le stringhe, però, sono oggetti e in quanto tali sono ognuna diversa dall'altra pur avendo lo stesso valore per ogni variabile di istanza. Per questo motivo quando si applica l'operatore **==** tra due oggetti questo andrà a confrontare gli indirizzi piuttosto che il contenuto dell'oggetto. Seguendo questo ragionamento capiamo il perché le stringhe **s3** e **s4** non sono uguali. Applicando però il confronto tra le stringhe **s1** e **s2** questo restituisce una risposta affermativa. Questo perché, quando si dichiarano letterali stringhe con lo stesso valore, tutti i loro riferimenti **vengono fatti puntare allo stesso spazio di memoria** per ottimizzare la gestione della memoria.

Per motivi di efficienza, infatti, poiché nelle applicazioni i letterali **String** occupano molta memoria, la JVM riserva un'area speciale di memoria ad essi: la **String constant pool**. Quando il compilatore incontra un letterale **String**, esso controlla che non sia già presente nel pool. Se è presente, allora il riferimento al nuovo letterale è diretto alla stringa esistente, altrimenti lo aggiunge al pool. Per questo motivo gli oggetti **String** sono immutabili. Se ci sono molti riferimenti, in punti diversi del codice, allo stesso letterale, sarebbe deleterio permettere la modifica del letterale usando uno solo di tali riferimenti.

Se proprio si deve fare un uso intensivo di manipolazione di stringhe, allora è opportuno usare gli oggetti **StringBuffer**: essi sono un po' come gli oggetti **String**, ma non sono immutabili:

```
1 StringBuffer s = new StringBuffer("Linguaggi");
2 s.append(" Di Programmazione");
3 System.out.println(s);
```



Mentre la classe **String** sovrappone il metodo **equals** in modo da controllare l'uguaglianza del contenuto dei due oggetti, la classe **StringBuffer** non lo sovrappone ed usa quello ereditato da **Object** che funziona essenzialmente come l'operatore **==**. Le classi **String** e **StringBuffer** inoltre sono **final**. Non possono essere specializzate in modo da sovrapporre i loro metodi: l'invocazione virtuale di metodi sarebbe un grave problema di sicurezza.

4.11.3 Le classi "wrapper"

I tipi primitivi non sono oggetti. Per questo motivo, se si vuole manipolarli come oggetti è possibile avvolgere un singolo valore con un opportuno oggetto chiamato **wrapper**. La API Java fornisce infatti una classe wrapper per ciascun tipo di tipo primitivo come mostrato nella Tabella 4.6.

```
1 public class Wrapper {
2     public static void main(String[] args) {
3         // Demo of autoboxing and unboxing
4         Integer i1 = new Integer(100);
5         Double d1 = new Double(3.14);
6         Integer i3 = Integer.valueOf(100);
7         Double d2 = Double.valueOf(3.14);
8         int i2 = i1.intValue();
9         double d3 = d1.doubleValue();
10        System.out.println(i2);
11        System.out.println(d3);
12        String str = "123";
13        int y = Integer.valueOf(str).intValue();
14        int z = Integer.parseInt(str);
```

⁴Immutabilità significa che, una volta assegnato all'oggetto un valore, esso è fissato per sempre.

Tipo primitivo	Classe wrapper
boolean	Boolean
byte	Byte
char	Character
short	Short
int	Integer
long	Long
float	Float
double	Double

Tabella 4.6: Classi wrapper

```

15     System.out.println(y);
16     System.out.println(z);
17 }
18 }
```

Codice 4.45: Demo classi wrapper

Si può costruire un oggetto wrapper, passando il relativo valore al metodo `valueOf()` come mostrato nel Listato 4.45. Il valore da avvolgere può essere passato anche in una rappresentazione sotto forma di `String` per poi farne il parsing.

4.12

I MODIFICATORI



I modificatori sono delle parole riservate che danno al compilatore alcune informazioni sulla natura del codice, dei dati e delle classi. Un gruppo di modificatori, detti **di accesso**, specificano a quali classi è permesso usare quella caratteristica. Altri modificatori possono essere usati, in combinazione con i precedenti, per qualificare quella caratteristica.

4.12.1 I modificatori di accesso

Uno dei capisaldi della programmazione ad oggetti è quello dell'**incapsulamento**: ovvero la possibilità di **schermare le caratteristiche interne** di ogni classe. Una caratteristica di una classe può essere:

- La classe stessa
- Un attributo (variabile) della classe
- Un metodo o un costruttore della classe

Schermare una caratteristica offre infatti svariati vantaggi relativi alla sicurezza dell'applicazione che si sta andando a sviluppare. Questa funzionalità viene resa possibile grazie ai **modificatori di accesso**, i quali permettono di stabilire il livello di protezione. I modificatori di accesso sono:

1. **public**: accesso consentito a tutte le classi. L'accesso ad un attributo o un metodo **public** è subordinato all'accesso alla classe che lo contiene. Il solo modificatore di accesso permesso per una classe top-level è **public**.
2. **protected**: accesso consentito a tutte le classi nello stesso pacchetto e a tutte le sottoclassi in pacchetti diversi, ma solo per ereditarietà: una sottoclasse in un altro pacchetto può accedere ad un membro **protected** nella superclasse solo attraverso un riferimento ad un oggetto del proprio tipo o di un sottotipo.
3. **private**: può essere usato per attributi o metodi, non per classi top-level. Le variabili private possono essere nascoste anche allo stesso oggetto che le possiede.

Una caratteristica inoltre può avere al più un modificatore di accesso. Se non è presente il modificatore si intende **default** (non è una parola riservata) che significa: "accesso consentito dallo stesso pacchetto in cui è situata quella caratteristica". Inoltre, i metodi visibili di una superclasse non possono essere sovrapposti (override) in una sottoclasse da altri meno visibili.

4.12.2 Il modificatore final

Il modificatore **final** si applica a classi, metodi e variabili (non a costruttori). Il significato varia da contesto a contesto, ma la sua essenza consiste nel fatto che una caratteristica **final** non può essere modificata. Di conseguenza:

- Una classe **final** non può essere estesa;
- Una variabile **final** è praticamente una costante, cioè può essere solo inizializzata. Un riferimento **final** ad un oggetto non può essere modificato ma l'oggetto a cui esso fa riferimento sì;
- Un metodo **final** non può essere sovrapposto in una sottoclasse;
- Non ha senso dire **final** ad un costruttore, perché esso non è mai ereditato dalle sottoclassi.

4.12.3 ■ Il modificatore **abstract**

Il modificatore **abstract** si applica solo a classi e metodi. Un metodo **abstract** non possiede corpo:

```
1 abstract void getValore();
```

Codice 4.46: Esempio di metodo **abstract**

Una classe può essere marcata **abstract**. In questo caso il compilatore suppone che essa contenga (anche per ereditarietà) metodi di tipo **abstract**: essa non potrà essere istanziata, anche se non contiene affatto metodi astratti. Una classe deve essere marcata **abstract** se:

- contiene almeno un metodo astratto;
- eredita almeno un metodo astratto per il quale non fornisce una realizzazione
- dichiara di implementare una interfaccia ma non fornisce una realizzazione di tutti i metodi di quell'interfaccia.

In un certo senso, il modificatore **abstract** è opposto a **final**. Una classe **final**, per esempio, non può essere specializzata mentre una classe **abstract** esiste solo per essere specializzata.

4.12.4 ■ Il modificatore **static**

Il modificatore **static** si applica ad attributi, metodi ed anche blocchi di codice che non fanno parte di metodi. In generale, una caratteristica **static** appartiene alla classe, non alle singole istanze: essa è l'unica, indipendentemente dal numero (anche zero) di istanze di quella classe.

Attributi **static**

L'inizializzazione di un attributo **static** avviene nel momento in cui la classe viene caricata in memoria (anche se non esisterà mai nessuna istanza di quella classe):

```
1 class Ecstatic{
2     static int x = 0;
3     Ecstatic () {
4         x++;
5     }
6 }
```

Codice 4.47: Attributi statici in Java

L'accesso ad un attributo **static** di una classe può avvenire (con la dot notation):

- o partendo da un riferimento ad una istanza di quella classe;
- o partendo dal nome stesso della classe.

Metodi **static**

I metodi statici esistono dal momento in cui la classe viene caricata in memoria (anche senza istanze). Non possono accedere a membri non statici della stessa classe (perché potrebbero anche non esistere). Ad esempio il metodo **main** è di tipo **static**. I metodi statici non possono essere sovrapposti da metodi non statici e non possono sovrapporre metodi non statici.

Blocchi **static**

È lecito che una classe contenga blocchi di codice marcati come statici. Tali blocchi sono eseguiti una sola volta, nell'ordine in cui compaiono, quando la classe viene caricata in memoria ed, ovviamente, possono accedere (come i metodi statici) solo a caratteristiche statiche.

```
1 public class EsempioStatic {
2     static double d = 1.23;
3 }
```

```

4  static {
5      System.out.println("Blocco static 1: d="+d++);
6  }
7
8  public static void main(String args[]){
9      System.out.println("Funzione main : d=" + d++);
10 }
11
12 static {
13     System.out.println("Blocco static 2: d="+d++);
14 }
15 }

```

Codice 4.48: Esempio di blocco statico in Java

Output:

```

Blocco static 1: d=1.23
Blocco static 2: d=2.23
Funzione main : d=3.23

```

In Java, il metodo statico `main` è l'entry point del programma, cioè il punto di partenza dell'esecuzione del programma. Quando un programma Java viene avviato, la JVM (Java Virtual Machine) cerca il metodo `main` all'interno della classe specificata come punto di ingresso (di solito la classe contenente il metodo `main`).

Se ci sono altri blocchi statici all'interno della stessa classe, questi blocchi verranno eseguiti prima del metodo `main`. Ciò è dovuto al fatto che *Java esegue tutti i blocchi statici di una classe prima di eseguire qualsiasi metodo non statico o di istanza.*

La ragione per cui Java esegue i blocchi statici prima del metodo `main` è legata alla natura del caricamento delle classi in Java. Quando una classe viene caricata per la prima volta dalla JVM, tutti i suoi blocchi statici vengono eseguiti per inizializzare le variabili statiche della classe e per eseguire eventuali altre operazioni necessarie all'avvio della classe. Solo dopo che tutti i blocchi statici sono stati eseguiti, la classe è pronta per l'uso e i metodi non statici possono essere chiamati.

4.12.5 ■ I modificatori `native`, `transient`, `synchronized` e `volatile`

Il modificatore `native` si applica solo a metodi. Come per `abstract`, `native` indica che il corpo di un metodo deve essere trovato altrove, in questo caso all'esterno della JVM, in una libreria di codice dipendente dall'architettura fisica.

Il modificatore `transient` si applica solo ad attributi. Indica che quell'attributo contiene informazioni sensibili e che, nel caso in cui lo stato interno dell'oggetto debba essere salvato (nel gergo "serializzato"), il valore di quell'attributo non dovrà essere considerato.

Il modificatore `synchronized` si applica solo a metodi o a blocchi anonimi e controlla l'accesso al codice in programmi multi-thread.

Il modificatore `volatile` si applica solo ad attributi e avverte il compilatore che quell'attributo può essere modificato in modo asincrono in architetture multiprocessore.

4.13

LE CLASSI ASTRATTE E INTERFACCE



Quando si è parlato di polimorfismo, di ereditarietà e di incapsulamento si sono messi in luce i vantaggi da tali caratteristiche nella gestione di eventuali modifiche da apportare successivamente alla fase di rilascio del software stesso.

Il concetto di **astrazione dei dati** interviene a rafforzare ulteriormente questi punti di forza, in particolare per quanto riguarda il riutilizzo del codice. Più in particolare, si può affermare che l'astrazione dei dati viene utilizzata per gestire al meglio la complessità di un programma, ovvero viene applicata per decomporre sistemi software complessi in componenti più piccoli e semplici che possono essere gestiti con maggiore facilità ed efficacia.

Per chiarire meglio i concetti appena esposti è sicuramente utile descrivere un esempio pratico. Consideriamo una classe **Animale**, che incapsuli al suo interno tutte le caratteristiche (metodi e proprietà) che sono comuni a tutti gli animali. Ad esempio, si potranno definire i metodi `mangia()` e `respira()` e le proprietà `altezza` e `peso` (solo per citarne alcuni). Ma, riflettendoci un attimo, appare chiaro che creare una istanza di un oggetto **Animale** ha poco senso visto che è certamente più consono definire istanze di oggetti specifici come **Cavallo**, **Cane**, **Gatto**, etc. In un simile contesto la classe **Animale**, non potendo essere direttamente istanziata, rappresenta un dato astratto ed è denominata, in OOP, **classe astratta**, ovvero una classe che rappresenta, fondamentalmente, un modello per ottenere delle classi derivate più specifiche e più dettagliate.

Poiché queste classi sono utilizzate solo come superclassi nelle gerarchie di ereditarietà, normalmente prendono il nome di **superclassi astratte**. Non è possibile istanziare alcun oggetto di una superclasse astratta, perché incompleta della definizione

dei suoi metodi, tuttavia queste classi possono avere attributi inizializzati oltre che avere dei costruttori specifici⁵. Sono le sue sottoclassi a dover creare i “pezzi mancanti”.

```
1 abstract class A{
2     // Metodo astratto, non contiene un corpo che la definisce
3     abstract void chiamami();
4     //Una classe astratta può contenere metodi concreti
5     void callmetoo(){
6         System.out.println("Questo e' un metodo concreto");
7     }
8 }
9
10 // Classe che estende A e implementa il metodo astratto
11 class B extends A{
12     void chiamami(){
13         System.out.println("Sono stato implementato dalla classe B");
14     }
15 }
```

Codice 4.49: Esempio di classe astratta

4.13.1 ■ Le interfacce: la keyword interface

Usando la parola chiave **interface** è possibile una classe astratta al 100%. Questo significa che, usando **interface**, è possibile specificare cosa deve fare una classe, ma non come deve farlo. Le **interfacce** sono simili alle classi ma non hanno attributi. Una volta definita un'interfaccia, qualsiasi numero di classi potrà **implementarla**. Inoltre, una classe può implementare un numero qualsiasi di interfacce.

Per definire un'interfaccia si utilizza la sintassi:

```
<interface_declaration> ::=
<modifier> interface <name>
[extends <interface> [, <interface> *]] {
<interface_body>
}
```

Una interfaccia può anche dichiarare costanti:

```
1 public static final int COSTANTE = 8;
```



Le variabili dichiarate nelle interfacce sono implicitamente **public**, **static**, **final**. Pertanto la dichiarazione precedente poteva anche essere scritta nel modo seguente:

```
1 int COSTANTE = 8;
```



Per implementare un'interfaccia si usa la keyword **implements**. La sintassi generale è la seguente:

```
<class_declaration> ::= <modifier> class <name> [extends <superclass>]
[implements <interface> [, <interface> *]] {
<class_body>
}
```

Una classe deve fornire l'insieme completo dei metodi richiesti dall'interfaccia. Si consideri il codice seguente:

```
1 public interface A{
2     void Callback(int param);
3 }
4
5 class B implements A{
6 }
```

Codice 4.50: Definizione di una interfaccia e di una classe che la implementa

In questo caso la classe B, nonostante indichi di star implementando l'interfaccia A non offre una definizione del metodo **Callback(int)**. Davanti ad errori del genere la Java Virtual Machine fornisce un warning e considererà la classe B di tipo astratto fino a quando non si ottiene un'implementazione del metodo (che sia nella classe B o in sue sottoclassi che la estendono).

⁵Con visibilità **protected** per garantire l'accesso solo alle classi che ereditano dalla superclasse astratta.

4.13.2 Vantaggi delle interfacce

Le interfacce⁶ sono spesso considerate una alternativa all’ereditarietà multipla, anche se esse forniscono diverse funzionalità. Esse sono utili:

- Per dichiarare metodi che una o più classi ci si aspetta dovrà implementare;
- Per rivelare solo una interfaccia, senza rilevare il corpo vero di una classe;
- Per catturare similarità tra classi non correlate, senza forzare una relazione tra classi;
- Per simulare l’ereditarietà multipla, dichiarando una classe che implementi varie interfacce.

4.14

LE ECCEZIONI



Spesso vi sono istruzioni “critiche”, che in certi casi possono produrre errori. L’approccio classico consiste nell’inserire blocchi `if ...else` per cercare di intercettare a priori tali situazioni ma spesso è un modo di procedere insoddisfacente in quanto non è facile prevedere tutte le situazioni che potrebbero produrre l’errore e l’interruzione dell’esecuzione del programma. Per questo motivo sono state introdotte le **eccezioni**.

Anziché tentare di prevedere le situazioni di errore, si tenta di eseguire l’operazione in un blocco controllato. Se si genera un errore, l’esecuzione lancia un’eccezione che viene catturata dal blocco entro cui l’operazione è eseguita per essere poi gestita nel modo più appropriato.

Un’eccezione indica che si è verificato un problema durante l’esecuzione del programma, questi errori possono essere causati ad esempio da:

- Un file che contiene informazioni errate
- Il fallimento di una connessione alla rete
- Il fallimento di accesso ad un’unità di memoria di massa
- Uso di indici invalidi in un array
- L’uso di un riferimento ad un oggetto a cui non è stato assegnato un oggetto

Si consideri ad esempio la seguente porzione di codice:

```
1 import java.util.Scanner;
2 public class Main{
3     public static int quotient(int numeratore, int denominatore) {
4         return numeratore/denominatore;
5     }
6     public static void main(String[] args) {
7         Scanner scanner = new Scanner(System.in);
8         System.out.println("Inserire un numero intero:");
9         int numerator = scanner.nextInt();
10        System.out.println("Inserire un numero intero:");
11        int denominator = scanner.nextInt();
12        int result = quotient(numerator, denominator);
13        System.out.println("Il risultato della divisione tra "+numerator+" e "+ denominator+" e': "+result);
14    }
15 }
```

Codice 4.51: DivideByZeroNoExceptionHandling.java

Output:

```
Inserire un numero intero:
100
Inserire un numero intero:
4
Il risultato della divisione tra 100 e 4 è: 25
```

```
Inserire un numero intero:
3
Inserire un numero intero:
0
```

⁶Il concetto di interfaccia è preso in prestito dal Objective-C, in cui essa è chiamata **protocollo**.

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
at main.Main.quotient(Main.java:5)
at main.Main.main(Main.java:13)
```

Inserire un numero intero:

100

Inserire un numero intero:

hello

```
Exception in thread "main" java.util.InputMismatchException
at java.base/java.util.Scanner.throwFor(Scanner.java:943)
at java.base/java.util.Scanner.next(Scanner.java:1598)
at java.base/java.util.Scanner.nextInt(Scanner.java:2263)
at java.base/java.util.Scanner.nextInt(Scanner.java:2217)
at main.Main.main(Main.java:12)
```

La prima delle tre esecuzioni dell'esempio mostra una divisione corretta. Nella seconda esecuzione, l'utente inserisce il valore 0 come denominatore. A questo punto vengono mostrate vengono mostrate molte linee di informazione in risposta a questo valore non valido dell'input. Queste informazioni sono conosciute come **traccia dello stack (stack trace)** e includono il nome dell'eccezione (**java.lang.ArithmeticException**) in un messaggio descrittivo che indica il problema che si è verificato e lo *stack* completo delle chiamate ai metodi (cioè la catena delle chiamate) nell'istante in cui si è verificata l'eccezione.

La prima riga dello *stack trace* specifica che si è verificata una **ArithmeticException**. Java, infatti, non consente la divisione per zero tra numeri interi. Le **ArithmeticException** possono derivare da molteplici problemi, per cui il dato extra (" / by zero") fornisce maggiori informazioni in merito alla specifica eccezione.

Nella terza esecuzione, l'utente inserisce la stringa "hello" come denominatore. Si noti di nuovo che viene visualizzato uno *stack trace* che ci informa che si è verificata una **InputMismatchException**. Può capitare spesso che l'utente, al posto di inserire un valore del tipo aspettato, ne inserisca uno del tutto incompatibile. Una **InputMismatchException** si verifica quando il metodo **nextInt** di **Scanner** riceve una stringa che non rappresenta un intero valido.

4.14.1 Cosa sono le eccezioni?

Una eccezione è un oggetto della classe **Throwable** o di una sua sottoclasse:

- **Exception**: indica situazioni recuperabili che andranno "catturate" e gestite.
- **Error**: indica problemi relativi al funzionamento della Java Virtual Machine e viene considerato irrecuperabile.

La figura 4.10 mostra i tipi di eccezioni predefinite esistenti in Java. Le eccezioni in Java si dividono in due categorie, le eccezioni **checked** e quelle **unchecked**. Mentre le prime sono delle eccezioni che si verificano a tempo di compilazione e devono essere obbligatoriamente essere gestite, le seconde avvengono a tempo di esecuzione e la loro gestione risulta opzionale. È comunque buona norma gestirle: una eccezione non controllata può infatti propagarsi di blocco in blocco. Se una eccezione raggiunge il main senza essere stata gestita il programma abortisce.

Oltre a quelle già esistenti è possibile definire le proprie eccezioni estendendo la classe **Throwable** o **Exception**. Le eccezioni predefinite sono in realtà sufficienti a coprire una vasta casistica; il motivo che spinge alla creazione di nuove eccezioni è più che altro l'esigenza di fornire informazioni specifiche note al programmatore ed utili per il trattamento dell'eccezione stessa.

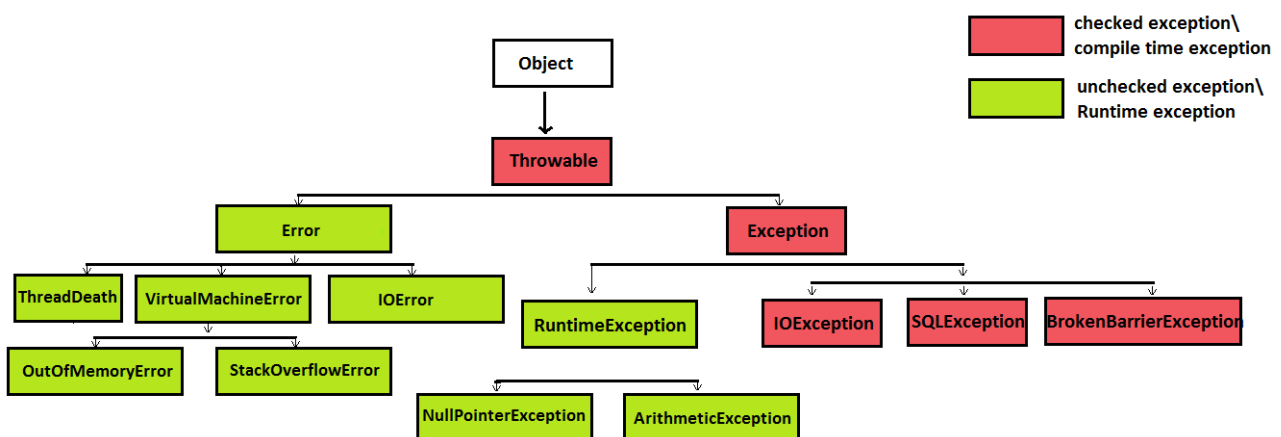


Figura 4.10: Tipi di eccezioni

Una eccezione quindi può essere lanciata usando la parola riservata **throw**:

```
1 throw new Exception();
```



e tutto ciò che segue, nello stesso blocco di istruzioni, il lancio dell'eccezione non verrà eseguito.

La classe Error

La classe **Error** e le sue sottoclassi rappresentano situazioni insolite che non sono causate da errori di programmazione o da ciò che normalmente succede durante l'esecuzione del programma. Per esempio, la JVM ha esaurito la memoria oppure qualche altra risorsa non è disponibile.

In genere, una applicazione non deve essere capace di riprendersi da una situazione di errore. Pertanto, un programma non è obbligato a gestire gli oggetti **Error**: esso compila senza problemi.

4.14.2 ■ Il costrutto try...catch

Il costrutto **try-catch** è fondamentale per la **gestione delle eccezioni** dato che rende possibile intercettare le eccezioni lanciate dalla JVM.

La sintassi generale di codifica di un **blocco try-catch** è mostrata nel Listato 4.52.

```
1 try{
2     //Codice
3 }catch (TipoEccezione1 e1){
4     //Codice che gestisce l'eccezione 1 (piu' specifica)
5 }catch (TipoEccezione2 e2){
6     //Codice che gestisce l'eccezione 2
7 }
8 // L'ultima è quella che gestisce il caso più generico
9 catch(Exception e){
10 }
```

Codice 4.52: Blocco try/catch

Dove il codice che potrebbe lanciare una eccezione è inglobato in un blocco **try** mentre il codice che assume la responsabilità di fare qualcosa in conseguenza di una eccezione si chiama **manipolatore** (*exception handler*) e va inglobato in una clausola **catch**.

Quando si arriva all'esecuzione del codice delimitato dalla parola chiave **try**, se si verifica una eccezione questa viene confrontata con quelle gestite nei blocchi **catch** e, se uno dei blocchi **catch** è in grado di gestirne il tipo, la intercetta e viene eseguito il codice contenuto nel blocco. Fatto questo, il programma prosegue con l'esecuzione della prima istruzione successiva al blocco **try-catch**.

Se esistono una o più clausole **catch** esse devono seguire immediatamente il blocco **try** ed è significativo l'**ordine** in cui si succedono le varie clausole **catch**.

```
1 import java.util.Scanner;
2 public class Main{
3     public static int quotient(int numeratore, int denominatore) { return numeratore/denominatore; }
4
5     public static void main(String[] args) {
6         Scanner scanner = new Scanner(System.in);
7         try{
8             System.out.println("Inserire un numero intero:");
9             int numerator = scanner.nextInt();
10            System.out.println("Inserire un numero intero:");
11            int denominator = scanner.nextInt();
12            int result = quotient(numerator, denominator);
13            System.out.println("Il risultato della divisione tra "+numerator+" e "+ denominator+" e': "+result);
14        }catch(ArithmeticException e1){
15            System.out.println("Hai inserito un denominatore uguale a zero!");
16        }
17        System.out.println("Il programma non si e' interrotto");
18    }
19 }
```

Codice 4.53: Esempio di blocco try-catch

Output:

```
Inserire un numero intero:
10
Inserire un numero intero:
2
Il risultato della divisione tra 10 e 2 e': 5
Il programma non si è interrotto
```

```
Inserire un numero intero:
10
Inserire un numero intero:
0
Hai inserito un denominatore uguale a zero!
Il programma non si è interrotto
```

In generale, quando il codice contenuto in un blocco **try** genera un'eccezione i blocchi **catch** vengono verificati in sequenza: il primo in grado di gestire il tipo di eccezione sollevata la intercetterà ed eseguirà il proprio codice. Se nessun blocco **catch** è in grado di eseguire il tipo dell'eccezione questa verrà presa in carico dal gestore di default.

Il blocco finally

Un blocco opzionale marcato **finally** è un blocco che viene sempre eseguito, anche dopo il lancio e la manipolazione (eventuale) dell'eccezione. Il blocco **finally** viene eseguito perfino dopo una eventuale istruzione **return** presente nei blocchi **try** o **catch**.

Gli unici casi in cui un blocco **finally** potrebbe non essere eseguito o non completare l'esecuzione sono i crash totali di sistema oppure una invocazione di `System.exit(int status)`.

4.14.3 Sollevamento delle eccezioni

Le eccezioni vengono sempre lanciate dalla Java Virtual Machine quando incontra delle anomalie. Queste possono essere:

- Causate da un'istruzione sbagliata: in tal caso l'istruzione viene troncata. Se, ad esempio, è un'espressione e l'eccezione è innescata dall'operando sinistro, il destro non sarà valutato.
- Generate esplicitamente con l'istruzione **throw <expr>** che deve restituire un riferimento ad un oggetto Throwable:
- Essere causate da un errore interno della Java Virtual Machine: in questo caso, nulla in genere si può fare e si parla di **eccezione asincrona**.

Al verificarsi dell'eccezione si possono seguire due strade:

- Gestire l'eccezione con un blocco **try/catch**;
- Rilanciarla esplicitamente all'esterno del metodo⁷ che contiene le istruzioni che hanno provocato l'eccezione, delegandone in pratica la gestione ad altri (unica eccezione: il **main**). Se si sceglie questa seconda strada il metodo deve indicare quali eccezioni possono "scaturire" da esso con la clausola **throws**.

In assenza di indicazioni da parte dello sviluppatore viene invocato il **gestore di default** il quale mostra una stringa che descrive l'eccezione, traccia il punto del programma in cui l'eccezione si è verificata e termina il programma.

```
1 import java.util.Scanner;
2 public class TestException{
3     public static void main(String[] args){
4         int x;
5         Scanner scanner = new Scanner(System.in);
6         x = scanner.nextInt();
7
8         // Verifica che l'utente non abbia inserito un valore troppo grande e nel caso lancia una eccezione
           manuale
9         if (x>100)
10            throw new Exception("x e' troppo grande");
11     }
12 }
```

Codice 4.54: Lancio di eccezioni tramite throw

Consideriamo nuovamente il metodo **quotient** visto nel Listato 4.51. Il metodo contiene una porzione di codice poco sicura in quanto non gestisce il caso in cui il denominatore possa essere uguale a zero determinando l'interruzione del programma. In

⁷Si "immerge" nel record di attivazione che ha invocato il metodo e così via finché, eventualmente, non arriva al record di attivazione del **main**, dal quale, se non catturata, "esplode".

questo caso il metodo lancia una eccezione di tipo `ArithmeticException` che deve essere definita nella firma del metodo (vedi il Listato 4.55).

```
1 public static int quotient(int numeratore, int denominatore) throws ArithmeticException {
2     return numeratore/denominatore;
3 }
```

Codice 4.55: La parola chiave `throws`

4.14.4 ■ La cattura delle eccezioni

Una clausola `catch` cattura ogni oggetto-eccezione il cui tipo può essere ricondotto mediante una conversione automatica al tipo specificato nella clausola.

Esempio

Ad esempio la classe `IndexOutOfBoundsException` ha due sottoclassi:

- `ArrayIndexOutOfBoundsException`
- `StringIndexOutOfBoundsException`

e si può scrivere un'unica clausola che catturi una qualunque di queste due eccezioni:

```
1 try{
2     // Codice che potrebbe lanciare una eccezione IndexOutOfBoundsException
3     // oppure ArrayIndexOutOfBoundsException oppure StringIndexOutOfBoundsException
4 }catch (IndexOutOfBoundsException e){
5     e.printStackTrace();
6 }
```



Per questo motivo l'ordine delle clausole `catch` è molto importante. Nel listato precedente, se avessimo scritto:

```
1 try{
2     // Codice che potrebbe lanciare una eccezione IndexOutOfBoundsException
3     // oppure ArrayIndexOutOfBoundsException oppure StringIndexOutOfBoundsException
4 }catch (IndexOutOfBoundsException e){
5     // tratta l'eccezione nel modo A
6 }catch (ArrayIndexOutOfBoundsException e){
7     // tratta l'eccezione nel modo B
8 }
```



il codice non sarebbe stato compilato.

L'ordine delle eccezioni da inserire nelle clausole `catch` deve andare da quella più **specifica** a quella più **universale**⁸

4.15

LE CLASSI INTERNE



Sostanzialmente, le **classi interne** sono classi definite all'interno del corpo di altre classi. Possono essere poste in qualsiasi blocco, incluso i blocchi dei metodi. Le classi interne si dividono in quattro differenti gruppi:

1. Classi membro
2. Classi (ed interfacce) statiche innestate
3. Classi locali
4. Classi anonime

4.15.1 ■ Le classi membro

Si consideri come esempio il Listato 4.56.

```
1 public class Persona{
2     public class NomeAnagrafico {
3         private String nome;
```

⁸Inserire una clausola `catch(Exception e)` permette infatti di gestire qualsiasi tipo di eccezione e in quanto tale andrebbe inserita come ultima tra le clausole.

```

4     private String cognome;
5
6     public NomeAnagrafico(String nome, String cognome) {
7         this.nome = nome;
8         this.cognome = cognome;
9     }
10
11     public String getNome() {return nome;}
12     public String getCognome() {return cognome;}
13 }
14
15 private NomeAnagrafico nomeAnagrafico;
16
17 public Persona(String nome, String cognome) {
18     this.nomeAnagrafico = new NomeAnagrafico(nome, cognome);
19 }
20
21 public NomeAnagrafico getNomeAnagrafico() {
22     return nomeAnagrafico;
23 }
24 }

```

Codice 4.56: Classi membro

Il nome della classe includente diventa parte del nome della classe inclusa. In questo caso i nomi completi delle due classi sono **Persona** e **Persona.NomeAnagrafico**. Questo formato è reminiscnte del modo in cui una classe è nominata all'interno di un pacchetto. Infatti, una classe interna appartiene alla propria classe includente allo stesso modo con cui una classe appartiene al proprio pacchetto. Dato che non è possibile che una classe e un pacchetto abbiano lo stesso nome non è possibile avere ambiguità di interpretazione.

Osservazione

La rappresentazione puntata vale solo all'interno del sorgente Java; essa non riflette il nome del file delle singole classi. Sul disco la classe interna si chiama **Esterno\$Interno**.

Quando si deve costruire una istanza di una classe membro, in genere, deve esistere già una istanza della classe esterna che agisca da contesto. Lo scope di una classe membro infatti è determinato dall'ambiente della sua classe esterna. L'oggetto interno può accedere quindi a tutte le componenti dell'oggetto esterno, indipendentemente dalla loro visibilità, attraverso un riferimento implicito.

Nel caso in cui non esista l'oggetto esterno, per esempio perché la JVM sta eseguendo un metodo statico, allora bisogna costruirne uno "al volo":

```

1 Esterna.Interna i = new Esterna().new Interna();
2 i.metodoInterno();

```



Esempio

Si consideri la seguente porzione di codice:

```
1 public class Esterna{
2     private int x;
3
4     public class Interna{
5         private int x;
6
7         public void fff (int x){
8             // parametro locale
9             x++;
10            // attributo interno
11            this.x++;
12            // attributo esterno
13            Esterna.this.x++;
14        }
15    }
16 }
```



In questo esempio, abbiamo una classe esterna chiamata **Esterna** che contiene una classe interna chiamata **Interna**. La classe interna ha un metodo chiamato **fff** che dimostra l'accesso e l'utilizzo di variabili nelle tre "scope" distinti:

1. **Parametro Locale (x)**: È il parametro del metodo **fff** e rappresenta una variabile locale al metodo stesso.
2. **Attributo Interno (this.x)**: È l'attributo **x** della classe interna. L'uso di **this.x** specifica chiaramente che si sta facendo riferimento all'attributo interno.
3. **Attributo Esterno (Esterna.this.x)**: È l'attributo **x** della classe esterna. Utilizzando **Esterna.this.x**, si fa riferimento esplicitamente all'attributo della classe esterna.

4.15.2 ■ Classi membro statiche

Le classi statiche innestate sono definite all'interno del corpo di una classe precedute da uno specificatore di accesso e dalla parola chiave **static**. Qualcosa come mostrato nel Listato 4.57. La classe **NomeAnagrafico** è interna, definita nel corpo della classe di primo livello **Persona**.

La struttura è usata per mantenere salda la coppia di dati **nome** e **cognome**. La compilazione del sorgente **Persona.java** porta alla nascita di due distinti bytecode: **Persona.class**, come di consueto, e **Persona\$NomeAnagrafico.class**. La norma che fa corrispondere un bytecode ad ogni classe o interfaccia, dunque non è infranta. In ogni caso, **NomeAnagrafico** è una classe speciale poiché statica ed interna a **Persona**.

Il codice presente nel corpo della classe **Persona** (Listato 4.57) può creare un oggetto di tipo **NomeAnagrafico** come di consueto:

```
NomeAnagrafico na = new NomeAnagrafico(...);
```

```
1 public class Persona{
2     public static class NomeAnagrafico {
3         private String nome;
4         private String cognome;
5
6         public NomeAnagrafico(String nome, String cognome) {
7             this.nome = nome;
8             this.cognome = cognome;
9         }
10
11        public String getNome() {return nome;}
12        public String getCognome() {return cognome;}
13        public String display() {return nome + " " + cognome;}
14    }
15
16    private NomeAnagrafico nomeAnagrafico;
17
18    public Persona(String nome, String cognome) {
19        nomeAnagrafico = new NomeAnagrafico(nome,cognome);
20    }
21
22    public NomeAnagrafico getNomeAnagrafico() {
23        return nomeAnagrafico;
24    }
```


4.15.3 ■ Classi locali

Le **classi locali** sono classi definite all'interno di un metodo:

1. Come tutte le altre variabili del metodo, sono locali in quel metodo e non possono essere marcate con nessun modificatore tranne che con `abstract` o `final`
2. Esse non sono accessibili in alcun modo all'esterno del metodo come tutte le altre variabili locali
3. Hanno accesso limitato alle altre variabili locali del metodo

La regola di accesso è semplice:

Ogni variabile locale (anche un parametro) del metodo includente non può essere utilizzata dalla classe interna, a meno che quella variabile non sia marcata `final`.

In realtà un oggetto (sullo heap) non dovrebbe accedere affatto alle variabili dei metodi (di stack) poiché gli oggetti sullo heap possono sovrascrivere ai record sullo stack. Quindi in che modo la JVM permette l'accesso limitatamente alle variabili `final`?

Va detto che una variabile `final` è una costante del metodo. L'oggetto che deve accedervi ne possiede semplicemente una copia (ad esempio generata durante la costruzione dell'oggetto). In tal modo, alla terminazione del metodo, la costante sarà ancora utilizzabile, anche se non esiste più il record di attivazione che la contiene.

```

1  public class ClasseEsterna {
2
3      private int variabileEsterna = 10;
4
5      public void metodoContenitore() {
6          // Definizione di una classe locale
7          class ClasseLocale {
8              void stampa() {
9                  System.out.println("Variabile esterna: " + variabileEsterna);
10             }
11         }
12
13         // Creazione di un'istanza della classe locale
14         ClasseLocale classeLocale = new ClasseLocale();
15
16         // Chiamata al metodo della classe locale
17         classeLocale.stampa();
18     }
19
20     public static void main(String[] args) {
21         ClasseEsterna esterna = new ClasseEsterna();
22         esterna.metodoContenitore();
23     }
24 }
```

Codice 4.58: Esempio di classe locale

4.15.4 ■ Classi anonime

Le **classi anonime** è una classe locale priva di nome. Le classi anonime rappresentano una scorciatoia per dichiarare e contemporaneamente istanziare una classe che dovrà essere usata per generare un solo oggetto. Le classi anonime usano il concetto di ereditarietà. Possono essere usate per estendere un'altra classe oppure, in alternativa, implementare una **singola interfaccia**. La sintassi non concede di fare entrambe le cose, né di implementare più di una interfaccia: se si dichiara una classe che implementa una singola interfaccia, allora la classe è sottoclasse diretta di `java.lang.Object`.

Poiché non si conosce il nome di tale classe, non si può usare `new` nel modo solito per crearne una istanza. Infatti, la definizione, la costruzione e il primo uso (spesso in assegnazione) avvengono nello stesso enunciato. L'utilità sta nella possibilità di sovrapporre qualche metodo della superclasse (o di implementare metodi di una interfaccia) senza la necessità di scrivere una vera classe che lo faccia.

Si consideri il seguente esempio:

```

1  class A{
```

```

2   public void f(){
3       System.out.println("A");
4   }
5   }
6
7   class B {
8       A a = new A(){
9           public void f() {
10              System.out.println("B");
11          }
12      }; // notare il ';'
13  }

```



Qui la variabile **a** si riferisce non ad una istanza della classe **A**, ma ad una istanza di una sottoclasse anonima di **A**. La linea 8 comincia con una dichiarazione di un riferimento ad **A** ma invece di essere:

```
A a = new A();
```

c'è una parentesi graffa aperta alla fine che si chiude alla linea 12. Quello che c'è nel mezzo si può parafrasare come “Dichiarare una variabile di tipo **A**; poi dichiara una nuova classe, senza nome, sottoclasse di **A**, che contenga una sovrapposizione del metodo **f()**; infine, costruisci una istanza di tale sottoclasse ed assegna il suo riferimento ad **a**”.

Dietro alle classi anonime si nasconde il polimorfismo: si usa un riferimento ad una superclasse per riferirsi ad una istanza di una sottoclasse. Ciò implica:

- Non si avrà mai la possibilità di accedere a ciò che si aggiunge nella definizione della sottoclasse anonima;
- L'uso di una classe anonima interna è possibile solo per quanto riguarda i metodi che essa sovrappone, e che sono citabili solo perché presenti nella superclasse;
- Essa non può avere un costruttore esplicito.

```

1   class A{
2       public void f(){
3           System.out.println("A");
4       }
5   }
6
7   class B {
8       A a = new A(){
9           public void g() { ... }
10          public void f() { ... }
11      };
12
13      public void usa(){
14          a.f(); // OK
15          a.g(); // Illegale
16      }
17  }

```



IL PARADIGMA FUNZIONALE: IL LINGUAGGIO ML

5.1

IL PARADIGMA FUNZIONALE



La **programmazione funzionale** è un paradigma di programmazione che si basa sul concetto di trattare le computazioni come valori di funzioni matematiche. Invece di modificare lo stato dei dati attraverso istruzioni imperative, nella programmazione funzionale si *definiscono funzioni pure* che trasformano i dati di input in dati di output senza causare effetti collaterali. Questo perché gli ambienti *mappano gli identificatori direttamente sul loro valore* (immutabile) piuttosto che sulla locazione di memoria.

Senza assegnamenti non ci possono essere strutture di controllo come i cicli **for/while** e di conseguenza:

- Si fa uso della **ricorsione** al posto dei cicli
- Si hanno **modifiche all'ambiente** anziché degli assegnamenti
- Con la creazione di nuovi identificatori con lo stesso nome si ottiene lo **shadowing** (come nello scoping statico).

Il linguaggio di programmazione ML ha le sue radici come meta-linguaggio per definire tattiche di dimostrazione all'interno di programmi di matematica interattivi. Nel corso degli anni, il linguaggio si è evoluto in un linguaggio di programmazione completo, con caratteristiche eccellenti per la programmazione su piccola e grande scala. Diverse proprietà rendono ML un linguaggio interessante:

- ML è un linguaggio **fortemente tipato**: il controllo dei tipi avviene interamente a tempo di compilazione. Ogni espressione del linguaggio ha un tipo che descrive i valori a cui può essere valutata, e il controllo del tipo al momento della compilazione assicura che vengano eseguite solo operazioni compatibili. Oltre a questo non richiede di dichiarare il tipo degli identificatori in quanto spesso lo capisce da solo (*type inference*);
- Usa sia la *structural equivalence* che la *name equivalence*;
- Permette di definire tipi ricorsivi come liste e alberi;
- Supporta il *polimorfismo parametrico* (come i template);
- Ha il garbage collector;
- Supporta l'incapsulamento (tipi di dato astratto) ma non è un linguaggio ad oggetti in quanto manca la gerarchia dei tipi e di conseguenza l'ereditarietà;

ML può essere usato in due modi:

1. Interagendo con l'interprete **sml** inserendo definizioni ed espressioni una per una (ottenendo una risposta ad ogni passo) oppure caricando programmi da file digitando nella shell dell'interprete il comando:

```
use "nome del file";
```

Per lanciare l'interprete SML dal terminale basta digitare il comando **sml** e premere invio. Per uscire si premiono i tasti CTRL+D. Per interrompere l'interprete e ripristinare una nuova linea di comando si preme CTRL+C.

2. Compilando un programma in codice oggetto direttamente eseguibile mediante il compilatore **mlton** per standard ML:

```
mlton "nome del file.sml"
```

Questo comando produce un file eseguibile con lo stesso nome (ma senza l'estensione **.sml**)



I tipi di dato primitivi in ML sono:

- **Numerali:** le costanti come 1, 2, 3 vengono letti come degli **interi** (**int**): 0, 1, 2, 3, 4, -1, -4, Notare il fatto che si usa ~ invece del segno meno per i numeri negativi. Gli interi senza segno vengono chiamati **word** e sono specificati dal letterale 0w: 0x44, 0wxff, ×. I valori **real** sono del tipo: 3.14, 2.0, 0.1E6,....
- **Letterali:** le variabili di tipo **string** sono caratterizzate dal doppio apice: "abc", ``123'', mentre le variabili **char** sono del tipo #"a", #"123",....
- **Booleani:** le variabili **bool** sono true e false.

```
Standard ML of New Jersey (64-bit) v110.99.4 [built: Tue Aug 08 11:25:21 2023]
-
- 3;
val it = 3 : int
- 0w7;
val it = 0wx7 : word
- 0wxff;
val it = 0wxFF : word
- ~4;
val it = ~4 : int
- "Hello";
val it = "Hello" : string
- #"a";
val it = #"a" : char
- 3.14;
val it = 3.14 : real
- 3.2+2;
stdIn:9.1-9.6 Error: operator and operand do not agree [overload - bad instantiation]
  operator domain: real * real
  operand:         real * 'Z[INT]
  in expression:
    3.2 + 2
- 3.2+real(2);
val it = 5.2 : real
```

Figura 5.1: Esempio di interazione con interprete SML

Esempio

La Figura 5.1 mostra vari esempi di interazione con l'interprete. `it` si riferisce all'espressione data; l'interprete ne calcola sia il valore che il tipo. Si noti che, quando si è tentato di eseguire l'istruzione `3.2+2`, l'interprete ha lanciato un'errore. Infatti non è possibile alcuna conversione automatica tra i tipi numerici. Vanno usati cast espliciti e le librerie **basis library** (c'è una basis library per ogni tipo primitivo (Int, Word, Real, ...) con funzioni per conversioni, parsing e altre utilità):

```
1 val r = 3.0 + real(2);
2 val i = 1 + 0w1;
3 val s = 1 + Word.toInt(0w1);
```



Esempio

Il tipo `Real` non supporta l'uguaglianza. Per questo motivo va usato l'operatore `==` definito nella `basis library Real`:

```
1 - val x = 1.0; val y = 2.0;
2 val x = 1.0 : real
3 val y = 2.0 : real
4 - x = y;
5 Error: operator and operand don't agree [equality type required]
6 -Real.==(x,y);
7 val it = false : bool
```



Questo perché lo standard IEEE prevede valori che risultano da operazioni non definite, denominati `NaN`. Un `NaN` non è confrontabile con nessun altro numero, nemmeno con sé stessi:

```
1 - val e = Math.sqrt(~2.0);
2 val e = nan : real
3 - Real.==(e,e);
4 val it = false : bool
```



5.3

DICHIARAZIONI E SCOPING IN ML



5.3.1 Dichiarazione di identificatori

Un valore viene legato ad un identificatore usando la sintassi:

```
1 val id = exp;
```



Con `val` si aggiunge un nuovo identificatore all'ambiente e gli si associa un valore. Si noti la differenza tra una dichiarazione (che lega un identificatore ad una espressione) e una espressione (che valutano valori). Gli identificatori che sono stati legati ad un valore possono essere usati successivamente all'interno delle espressioni per riferirsi a tali valori:

```
1 val x = 1;
2 val pi = 3.141592;
3 val y = if (3>4) then 0 else 1;
4 x + y;
5 if (x = 0) then pi else 1.0;
```



Se si vincola un valore a un identificatore al quale è già stato legato un valore, il nuovo legame **oscurerà il vecchio legame** e sarà usato da quel momento in poi. Come caso particolare, ogni volta che un'espressione è valutata a livello `oplevel` e non è esplicitamente legata a qualche identificatore, viene automaticamente legata all'identificatore stesso. Pertanto, è sempre possibile accedere al valore dell'ultima espressione valutata. Ad esempio:

```
1 - 3+4;
2 val it = 7 : int
3 - it * 2;
4 val it = 14 : int
```



5.3.2 Dichiarare una funzione

Ci sono diversi modi di definire e chiamare nuove funzioni in ML. Il metodo più tradizionale consiste nell'usare la keyword `fun` seguendo la seguente sintassi:

```
1 fun <fun_name> <parameter> = <expression>
```



Esempio

Ad esempio:

```
1 - fun double x = 2*x;  
2 - fun inc x = x+1;  
3 - fun adda s = s ^ "a";
```



Per eseguire una funzione basterà digitare il nome della funzione seguita dal parametro attuale:

```
1 - double 6;  
2 val it = 12 : int  
3 - inc 100;  
4 val it = 101 : int  
5 - adda "cub";  
6 val it = "cuba" : string
```



Esempio

È possibile anche definire funzioni ricorsive:

```
1 - fun fatt x = if x = 0 then 1 else x*fatt(x-1);  
2 val fatt = fn : int → int  
3 - fatt;  
4 val it = fn : int → int
```



Con la keyword **fun** si dichiara quindi una funzione. La keyword **fun** aggiunge all'ambiente l'identificatore **fatt** e lo associa alla funzione da interi a interi. Per vedere a cosa è associato l'identificatore **fatt** basterà indicare il suo nome senza argomenti al compilatore il quale mostrerà solo il tipo della funzione associata a tale nome.

5.3.3 Blocchi e scope locale

È possibile dichiarare *vincoli locali* (legami che sono validi solo all'interno del calcolo di una data espressione). Questi sono l'equivalente dei blocchi visti nei capitoli precedenti. Lo scoping in questo caso è **statico** e la sintassi è la seguente:

```
1 let  
2 <dichiarazioni>  
3 in  
4 <espressione>  
5 end
```

Codice 5.1: Equivalente dei blocchi in ML

Quindi una espressione come

```
1 let  
2 val x=2  
3 in  
4 3*x  
5 end;
```



calcola l'espressione **3*x** sotto i legami definiti dopo **let**. Dopo aver calcolato l'espressione i vari legami sono dimenticati.

Esempio

Si faccia attenzione ai diversi valori dell'identificatore **a** nel codice seguente:

```
1 - val a = 10;  
2 val a = 10 : int  
3 - a;  
4 val it = 10 : int  
5 - let val a = 30  
6 in a + 1 end;  
7 val it = 31 : int  
8 - a;  
9 val it = 10 : int
```



Esempio

Dato lo scoping di tipo statico (vedi 3.2) in caso di dichiarazioni annidate che usano lo stesso identificatore si ottiene l'effetto collaterale del mascheramento degli identificatori precedenti:

```
1 - let val x=2 in  
2 let val x=3 in (* questa def. maschera la precedente *)  
3 3*x  
4 end  
5 end;  
6 val it = 9 : int
```



Esempio

Per quanto riguarda le funzioni si ricordi che l'ambiente non locale è propagato dall'ambiente che la contiene sintatticamente:

```
1 - val x = 0;  
2 val x = 0 : int  
3 - let val x = 1 in let fun f(y) = x+y in f(0) end end;  
4 val it = 1 : int  
5 - x;  
6 val it = 0 : int
```



5.3.4 Dichiarazioni locali

Le dichiarazioni locali sono simili alle espressioni **let** ma dopo **in** c'è una dichiarazione invece di una espressione da valutare. Permettono la dichiarazione di valori che possono essere usati da altre dichiarazioni senza che questi siano rilevati. Come nel caso dei tipi astratti, il sistema dei moduli (attraverso l'assottigliamento delle firme) fornisce un modo più flessibile di gestire le dichiarazioni locali nel caso di programmi di grandi dimensioni.

Esempio

Ad esempio:

```
1 - local
2   val a = 3
3   val b = 10
4 in
5   val x = a + b
6   val y = a * b
7 end;
8
9 val x = 13 : int
10 val y = 30 : int
11 - x;
12
13 val it = 13 : int
14
15 - a;
16 stdIn:139.1 Error: unbound variable or constructor: a
```



5.4

TIPI STRUTTURATI IN ML



5.4.1 ■ Prodotti cartesiani

In ML, le tuple sono strutture dati immutabili che consentono di raggruppare diversi valori di tipi diversi in un'unica entità. Una tupla può contenere un numero arbitrario di elementi, e ogni elemento può avere un tipo diverso dagli altri.

La dichiarazione di una tupla in SML/NJ segue la seguente sintassi:

```
1 val nomeTupla = (valore1, valore2, ..., valoreN)
```

Codice 5.2: Dichiarazione di una tupla in ML

Una tupla viene interpretata come un **prodotto cartesiano** tra i tipi dei valori che appartengono alla tupla. Il prodotto cartesiano tra tipi, indicato con $*$, è un concetto utilizzato nella teoria dei tipi per combinare N tipi per formare un nuovo tipo che rappresenta tutte le possibili tuple ordinate di valori di quei tipi. Questo tipo di rappresentazione è utile per rappresentare strutture di dati più complesse, come record o strutture nidificate. È spesso utilizzato nei linguaggi di programmazione funzionali per gestire dati strutturati in modo elegante e tipizzato.

Per estrarre l' i -esimo elemento da una tupla si usa la notazione $\#i$:

```
1 - (1+1, "A");
2 val it = (2,"A") : int * string
3 - val miatupla = ("A", 2);
4 val miatupla = ("A",2) : string * int
5 - #1(miatupla);
6 val it = "A" : string
```



5.4.2 ■ Record

In ML i record sono strutture dati che consentono di raggruppare diversi valori correlati in una singola entità. I record sono simili alle tuple, ma a differenza delle tuple, i record permettono di accedere agli elementi utilizzando etichette o nomi di campo anziché indici.

La dichiarazione di un record in ML segue la seguente sintassi:

```
1 datatype nomeRecord = {campo1: tipo1, campo2: tipo2, ..., campoN: tipoN}
```

Codice 5.3: Dichiarazione di un record in ML

Esempio

Un record è quindi un insieme di espressioni `<nome>=<valore>`. Notare come viene rappresentato il tipo:

```
1 - val r = {nome = "giorgio", nato = 1999};
2 val r = {nato=1999,nome="giorgio"} : {nato:int, nome:string}
```



Il valore associato al nome N si estrae con la notazione `#N`:

```
1 - #nome(r);
2 val it = "giorgio" : string
3 - #nato(r);
4 val it = 1999 : int
```



5.4.3 Sinonimi di tipo

La keyword `type` in SML/NJ viene utilizzata per definire sinonimi di tipo, cioè per creare un nome alternativo per un tipo esistente. Questo può rendere il codice più leggibile e semantico, e può essere utile per fornire una documentazione chiara dei tipi. La sintassi per definire un sinonimo di tipo utilizzando `type` è la seguente:

```
type nomeSinonimo = tipoOriginale
```

Esempio

La keyword `type` è simile ai `typedef` del C:

```
1 - type coord = real*real;
2 type coord = real * real
3 - val x = (3.0, 4.0);
4 val x = (3.0,4.0) : real * real
5 - val x:coord = (3.0,4.0);
6 val x = (3.0,4.0) : coord
```

Codice 5.4: Sinonimi di tipo: la keyword `type`

I tipi `coord` e `(real * real)` sono compatibili tra loro (*structural equivalence*) quindi sarà possibile passare una espressione di tipo `coord` ad un parametro di tipo `(real*real)` e viceversa, proprio grazie al fatto che `coord` non è altro che un sinonimo per `real * real` (`coord` sarà compatibile con ogni altro tipo definito come `(real*real)`).

5.4.4 Datatypes e costruttori

In SML/NJ, è possibile definire nuovi tipi utilizzando la keyword `datatype`. Questa sintassi consente di creare nuovi tipi di dati astratti. La sintassi generale per la definizione di un **costruttore** per creare data objects è la seguente:

```
1 datatype nomeTipo = Costruttore1 of tipo1 |
2 Costruttore2 of tipo2 | ... | CostruttoreN of tipoN
```

Codice 5.5: Dichiarazione di un costruttore

In questa sintassi, `nomeTipo` è il nome del nuovo tipo che si sta definendo mentre `Costruttore1`, `Costruttore2`, ecc. definiscono i possibili valori del tipo appena definito e vengono chiamati **costruttori**.

Esempio

La sintassi, solo apparente, è simile a quella delle `enum` del C. Infatti, le enumerazioni in C non sono altro che delle codifiche per gli interi. Invece i `datatype` di ML definiscono tipi genuinamente nuovi che non hanno nulla a che fare con gli `int`. Per questo motivo ogni tipo definito con `datatype` è incompatibile con tutti gli altri tipi (*name equivalence*).

```
1 - datatype color = red | green | blue;
2 datatype color = blue | green | red;
3
4 -val c = red;
5 val c = red : color
6
7 - val c:color = 10;
8 Error: pattern and expression in val dec don't agree
```



Esempio

Ad esempio, per definire un nuovo tipo chiamato `Giorno` che rappresenta i giorni della settimana, si può scrivere:

```
1 - datatype Giorno = Lunedì | Martedì | Mercoledì;
2 datatype Giorno = Lunedì | Martedì | Mercoledì
3 - val oggi : Giorno = Mercoledì;
4 val oggi = Mercoledì : Giorno
```

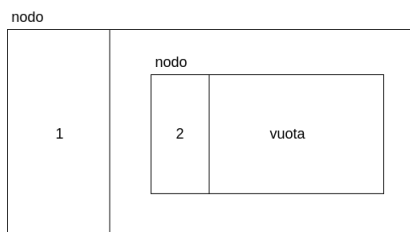


Figura 5.2: Rappresentazione della lista [1, 2]

Esempio

Definiamo una lista concatenata di interi utilizzando la definizione ricorsiva. Una lista di interi è:

- la lista vuota (caso base)
- un nodo che contiene un intero e una lista di interi (passo induttivo)

Per questo motivo, nel definire un nuovo tipo chiamato `listaInt` saranno necessari due costruttori: uno per la lista vuota e uno per i nodi. Attraverso l'operatore `of` è possibile specificare il tipo di una espressione o di una variabile.

```
1 - datatype listaInt = vuota | nodo of int * listaInt;
2 datatype listaInt = nodo of int * listaInt | vuota
3
4 - val L = nodo (1, nodo(2,vuota));
5 val L = nodo (1,nodo (2,vuota)) : listaInt
```



In questo caso il tipo di un nodo è dato dal prodotto cartesiano di un tipo intero e del tipo appena definito. La lista `L` (si veda la Figura 5.2) viene così definita mediante il costruttore di un `nodo` che a sua volta al suo interno vede definita una lista con un singolo nodo.

Esempio

Si definisca un albero binario con i nodi etichettati da interi. La definizione ricorsiva è data da:

- Un albero vuoto;
- Un nodo che contiene un intero e due alberi dello stesso tipo.

Quindi:

```
1 datatype albero = vuoto | nodoAlb of int * albero * albero;
```



In questo esempio costruiamo un albero con radice 1, figlio sinistro 2 (che è una foglia), mentre il figlio destro manca:

```
1 nodoAlb (1, nodoAlb (2, vuoto, vuoto), vuoto)
```



5.5

PATTERN MATCHING



A differenza di altri linguaggi ML permette di inserire una struttura come argomento a sinistra di una assegnazione. Grazie alla **type inference**, infatti, ML è in grado di osservare la struttura e assegnare i corretti legami di tipo. Questo meccanismo prende il nome di **pattern matching**.

Esempio

Quando si vuole scandire una lista bisogna controllare innanzitutto se essa è vuota:

```
1 - val L = nodo (1, nodo(2, vuota));  
2 val L = nodo (1, nodo(2,vuota)) : listaInt  
3  
4 - L = vuota;  
5 val it = false : bool
```



Se la lista risulta non vuota potrebbe servirci il primo elemento. Questo può essere estratto con **pattern matching**:

```
1 - val nodo(p,_) = L; (* assegna a p il primo elemento di L*)  
2 val p = 1
```



Il costrutto `nodo(p,_)` viene chiamato **pattern** e serve ad assegnare alla variabile `p` il primo elemento di `L`. Il simbolo “`_`” è una wildcard. Per ottenere il resto della lista:

```
1 - val nodo(_,r) = L; (* assegna a r il resto *)  
2 val r = nodo (2,vuota) : listaInt
```



Esempio

Una funzione che conti gli elementi della lista scritta in paradigma funzionale deve ovviamente essere ricorsiva. Questa quindi verifica innanzitutto che la lista non sia vuota in testa, in caso contrario scandisce la sottolista fino a quando questa non è vuota, una volta trovato il caso base restituisce un incremento che viene sommato al risultato finale:

```
1 -fun conta x = if x = vuota then 0
2 else let val nodo(_,p) = x in conta(r)+1 end;
3 val conta = fn : listaInt → int
4
5 - conta L;
6 val it = 2 : int
```

Codice 5.6: Contare gli elementi di una lista

Notare che il compilatore ha *dedotto* il tipo della funzione. Infatti:

1. `x` viene confrontato con “vuota”, che è di tipo `listaInt`, quindi anche `x` è di tipo `listaInt`. Quindi l’input di `conta` è di tipo `listaInt`.
 2. Il costrutto `then` restituisce 0, che è un intero: quindi l’output di `conta` è di tipo intero.
- Oltre a ciò, il compilatore controlla che anche il resto della funzione sia compatibile con questi tipi:
1. `r` corrisponde al secondo argomento del nodo, che è di tipo `listaInt`, allora è corretto passarlo a `conta` che restituisce un intero;
 2. Quindi anche l’`else` restituisce un intero. Tutto torna.
- Questo meccanismo prende il nome di **type inference**.

Esempio

È possibile estrarre tutti gli elementi di un costruttore in un colpo solo:

```
1 -val nodo(p,r) = L
2 val p = 1 : int
3 val r = nodo (2, vuota) : listaInt
```



Si dichiarano cioè due identificatori (`p` e `r`) in un colpo solo. In questo modo è possibile *definire una funzione per casi*:

```
1 - fun conta(vuota) = 0
2 | conta(nodo(_,r)) = conta(r)+1
```



mettendo direttamente i pattern al posto dei parametri formali. Da notare l’eleganza e la concisione di una definizione del genere.

5.6

LISTE



Le liste sono tra le strutture dati più usate in programmazione funzionale perché in qualche misura sostituiscono i vettori (che sono la diretta espressione del paradigma imperativo). Per questo motivo il linguaggio ML le fornisce come tipo di dato predefinito tramite i costruttori `nil` e `::`.

```
1 nil (*lista vuota*)
2 1 :: 2 :: 3 :: nil (*lista che contiene 1,2,3*)
3
4 (*formato equivalente basato su parentesi quadre*)
5 []
6 [1,2,3]
7
8 (*sono veramente equivalenti*)
9 - [] = nil;
10 val it = true:bool
11
12 -1::2::3::nil = [1,2,3];
13 val it = true:bool
```

Codice 5.7: Dichiarazione di liste in ML

5.6.1 Principali operatori sulle liste in ML

La struttura `List` fornisce una collezione di molte funzioni utili per manipolare le liste. La loro documentazione è disponibile sul sito <https://smlfamily.github.io/Basis/list.html>.

Funzione	Descrizione
<code>length</code>	Restituisce la lunghezza di una stringa
<code>null</code>	Restituisce <code>true</code> se la stringa è vuota
<code>hd</code>	Restituisce il primo elemento di una lista
<code>tl</code>	Restituisce il resto di una lista
<code>List.nth(L,i)</code>	Restituisce l' <i>i</i> -esimo elemento di <i>L</i>
<code>List.last(L)</code>	Restituisce l'ultimo elemento di <i>L</i>

Tabella 5.1: Alcune funzioni disponibili sulle liste in ML

```
1 -val L = [1,2,3];
2 val L = [1,2,3] : int list
3
4 -hd L;
5 val it = 1 : int
6
7 -tl L;
8 val it = [2,3] : int list
```



5.7

CURRYING

Il linguaggio ML sceglie il tipo più generale (il meno restrittivo) possibile per le funzioni definite dall'utente. Per questo motivo ogni funzione riceve sempre un solo argomento. Ad esempio, la definizione:

```
1 fun f(x,y) = ... (* l'argomento e' una singola coppia *)
```



Riceve un singolo argomento, ovvero una coppia ordinata del tipo (x,y) . Consideriamo invece il seguente listato:

```
1 fun f x y = ... (* l'argomento e' x *)
```



In questo caso, si definisce una funzione `f` (di `x`) che restituisce una funzione (di `y`).

Esempio

Si osservi il listato seguente:

```
1 -fun f (x,y) = x+y;
2 val f = fn : int * int -> int
3 -fun f' x y = x+y;
4 val f' = fn : int -> int -> int
```



Il tipo `int->int->int` va inteso come `int->(int->int)`

La riduzione da funzioni con molti argomenti ad una sequenza di funzioni ad una variabile prende il nome di **currying**¹.

¹Prende il nome dal matematico Haskell Curry che aveva sviluppato questa tipologia di analisi delle funzioni.

Esempio

```
1 - f(3,2);
2 val it = 5 : int
3 - f' 3 2; (*viene interpretato come f'3(2)*)
4 val it = 5 : int
5 - f 3 2;
6 Error: operator and operand don't agree
7 - f'(3,2);
8 Error: operator and operand don't agree
9 - val g = f' 3;
10 val g = fn : int → int
11 - g 1;
12 val it = 4 : int
```



Esempio

Quindi:

```
1 - fun f x y = x :: [y];
2 val f = fn : 'a → 'a → 'a list
```



definisce una funzione che prende un parametro `x` di tipo `'a` e restituisce una funzione (di `y`, il cui tipo è `'a`) che restituisce una lista di tipo `'a`. Confrontiamola con la funzione:

```
1 - fun g(x,y) = x :: [y];
2 val g = fn : 'a * 'a → 'a list
```



Qui la funzione `g` prende in input una coppia di argomenti (entrambi di tipo `'a`) e restituisce una lista di tipo `'a`. Il vantaggio del currying sta nel fatto che possiamo vincolare una funzione ad un valore e lasciare il secondo parametro variabile.

Ad esempio:

```
1 - f(1);
```



è una chiamata lecita in quanto restituisce una funzione del tipo:

```
1 fn : int → int list
```



La funzione restituita è equivalente alla funzione:

```
1 - fun h b = 1 :: [b];
2 val h = fn : int → int list
```



5.8

FUNZIONI DI ORDINE SUPERIORE



La maggior parte delle funzioni ricorsive che operano su liste, alberi e simili *hanno la stessa struttura*. Cambia solo l'*operazione che si applica ai nodi*. Quindi basta scrivere una volta per tutte le volte che si scandisce la struttura dati (che fa la funzione del ciclo) e passargli la funzione da applicare ai nodi. Le funzioni che hanno altre funzioni come parametri sono dette di **ordine superiore**.

Le tipologie di funzioni/ciclo più comuni sono tre:

- Filter
- Map
- Reduce

5.8.1 La funzione Filter

La funzione FILTER prende una funzione booleana f e una lista L e seleziona gli elementi di L per cui f è vera:

```
1 fun filter f [] = []  
2 | filter f (x::y) = if f(x) then x::(filter f y)  
3 else filter f y
```

Codice 5.8: La funzione filter

Esempio

Selezioniamo gli elementi negativi da una lista:

```
1 - let fun neg x = x < 0  
2 in filter neg [0,~1,3,~3] end;  
3 val it = [~1,~2] : int list
```



Esempio

Selezioniamo gli elementi positivi da una lista:

```
1 - let fun pos x = x > '  
2 in filter pos [0,~1,3,~2] end;  
3 val it = [3] : int list
```



5.8.2 La funzione Map

La funzione MAP prende una funzione booleana f e una lista L applica f a tutti gli elementi della lista:

```
1 fun map f [] = []  
2 | map f (x::y) = f(x)::(map f y)
```

Codice 5.9: La funzione map

Esempio

Conversione in lista di reali:

```
1 - map real [1,2,3]  
2 val it = [1.0,2.0,3.0] : real list
```



Esempio

Conversione in lista di stringhe:

```
1 - map Int.toString [1,2,3];  
2 val it = ["1","2","3"] : string list
```



5.8.3 La funzione Reduce

La funzione REDUCE serve per calcolare aggregati di una lista come ad esempio **min**, **max**, **somma**, **media**...

La funzione prende in input una funzione a 2 argomenti f , un valore finale t ed una lista L ed effettua questo calcolo:

$$reduce\ f\ t\ [x_1, x_2, x_3, \dots, x_n] = f(x_1, f(x_2, \dots, f(x_n, t)))$$

```

1 fun reduce f t [] = e
2 | reduce f t (x::y) = f(x, reduce f t y)

```

Codice 5.10: La funzione reduce

Esempio

Se $f +$ rappresenta la somma e $t = 0$ allora REDUCE calcola la somma degli elementi della lista:

```

1 - reduce (op+) 0 [1,2,3]
2 val it = 6 : int

```



Esempio

Calcolare la media di $[x_1, \dots, x_n]$ ovvero $m = \frac{\sum_{i=1}^n x_i}{n}$:

```

1 - let
2 fun f L (elem, accum) = elem / real(lenght L) + accum
3 val lista = [1.0, 2.0, 3.0]
4 in
5 reduce (f lista) 0.0 lista
6 end;
7 val it = 2.0 : real

```



5.8.4 Funzioni anonime

Quando utilizziamo funzioni di ordine superiori come FILTER, MAP e REDUCE, può far comodo passargli funzioni semplici, specificate lì per lì senza dover dare loro un nome (nè usare un blocco `let`). Queste funzioni prendono il nome di **funzioni anonime** e si specificano con la keyword `fn`:

```

1 - fn x => x+1;
2 val it = fn : int -> int
3
4 - map (fn x => x+1) [1,2,3];
5 val it = [2,3,4] : int list

```



In Lisp e Scheme l'equivalente di `fn` è la keyword `lambda`. Storicamente deriva dal *lambda calcolo*, un modello di calcolo matematico basato su funzioni di ordine superiore a cui tutti i linguaggi funzionali si sono ispirati. Nel lambda calcolo l'operatore λ è l'analogo di `fn`.

fun o val?

Consideriamo le seguenti funzioni definite rispettivamente con le parole chiave `fun` e `val`:

```

1 - fun f x = x + 1;
2 val f = fn : int -> int
3
4 - val f = fn x => x+1;
5 val f = fn : int -> int

```



Dalle risposte dell'interprete possiamo osservare che entrambe le definizioni della funzione $f(x) = x + 1$ portano allo stesso risultato. Infatti `fun` altro non è che **zucchero sintattico**, una utile abbreviazione per ciò che si potrebbe fare benissimo con la keyword `val`.

5.8.5 Ancora sul currying

Adesso possiamo mostrare più esplicitamente la natura del currying. Riprendiamo l'esempio:

```

1 - fun f' x y = x+y;

```



In effetti f' può essere definita equivalentemente come:

```
1 fun f' x = fn y => x+y;
2 val f' = fn : int -> int -> int
```



L'esempio della chiamata di f' con un solo parametro può essere spiegata così:

```
1 - val g = f' 3;      (* come fosse val g = fn y => 3+y *)
2 val g = fn : int -> int
3
4 - g 1;
5 val it = 4 : int
```



È anche possibile definire funzioni che effettuano il currying e la sua trasformazione inversa per una data funzione del tipo giusto:

```
1 (* f deve accettare una coppia (x,y) *)
2 val curry_2args = fn f => fn x => fn y => f (x,y)
3
4 (* f deve essere del tipo f x y *)
5 val uncurry_2args = fn f => fn (x,y) => f x y
```



Esempio

Si ha:

```
1 fun f (x,y) = x+y;
2 val f' = curry_2args f; (* equivalente a f' x y = x+y *)
```



oppure:

```
1 fun f' x y = x+y;
2 val f = uncurry_2args f'; (* equivalente a f(x,y) = x+y *)
```



5.9

POLIMORFISMO PARAMETRICO



Il linguaggio ML supporta l'analogo dei template di C++ e Java, ovvero i **tipi parametrici**. In realtà questa tipologia di polimorfismo si è vista fin dal momento in cui sono state introdotte le liste. Infatti, il costruttore `::` può essere applicato a qualunque tipo e, analogamente, la funzione `length` prende in input una lista di elementi il cui tipo `'a` non è specificato². Anche la funzione di ordine superiore `Map` è una funzione parametrica:

```
1 - map;
2 val it = fn : ('a -> 'b) -> 'a list -> 'b list
```



5.9.1 Tipi parametrici

Per definire tipi parametrici come le liste si usa l'apice prima del nome del parametro per indicare che quella è una variabile di tipo:

```
1 (* generalizzazione delle liste *)
2 - datatype 'a lista = vuota | nodo of ('a * 'a lista);
3 datatype 'a lista = nodo of 'a * 'a lista | vuota;
4
5 (* alberi binari con etichette parametriche *)
6 - datatype 'a bt = emptybt | btnode of ('a * 'a bt * 'a bt);
```

²Infatti non ha bisogno di saperne il tipo, deve solo contarne i nodi

```
7 datatype 'a bt = bnode of 'a * 'a bt * 'a bt | emptybt
```



Per quanto riguarda invece le funzioni non bisogna specificare alcunché in quanto sarà il compilatore mediante la **type inference** a determinare il tipo:

```
1 - fun conta(vuota) = 0
2   | conta (nodo (x,l)) = conta(l)+1;
3 val conta = fn : 'a lista → int
```



Osservazione

Quando si vuole che il tipo supporti l'uguaglianza bisogna mettere un doppio apice. Ogni tanto la type inference se ne accorge da sola.

Esempio

Si consideri il seguente codice:

```
1 - fun diag (x,y) = x=y;
2 val diag = fn : 'a * 'a → bool
3
4 - diag(1.0,1.0);
5 Error: operator and operand don't agree [equality type required]
6   operator domain: 'Z * 'Z
7   operand:  real * real
```



Infatti i reali non supportano l'uguaglianza...

5.10

ENCAPSULATION E INTERFACCE



5.10.1 Signatures

Le **signatures** sono il costrutto mediante il quale è possibile definire le interfacce in ML. Mediante quest'ultime è possibile quindi definire tipi e funzioni senza specificare come questi devono essere implementati.

Esempio: lo stack

Proviamo a definire una struttura dati astratta e le sue operazioni di base:

```
1 signature STACK =
2 sig
3   type 'a stack  (* dichiara un tipo parametrico 'a stack senza definire come è definito *)
4   val empty : 'a stack  (* una funzione per costruire uno stack vuoto *)
5   val push : ('a * 'a stack) → 'a stack
6   val pop : 'a stack → ('a * 'a stack)
7 end;
```

Codice 5.11: Definizione di uno stack con le signatures

5.10.2 Structures

Le **structures**, come le classi, definiscono i tipi di dato astratto.

Esempio

Riprendendo l'esempio precedente, è possibile implementare uno stack mediante le liste concatenate:

```
1 structure Stack :> STACK =  
2 struct  
3   type 'a stack = 'a list;  
4   val empty = [];  
5   fun push (x,s) = x :: s;  
6   fun pop (x::s) = (x,s);
```



Con l'espressione `Stack :> STACK` si dichiara che la struttura `Stack` deve implementare tutti gli identificatori dichiarati nella signature `STACK`. Inoltre, i tipi di dato dichiarati in `Stack` possono essere utilizzati solo con le operazioni dichiarate nella struttura. Si ottiene in questo modo l'**incapsulamento**.

Osservazione



Anche se `Stack` è definita con le liste non è possibile utilizzare le operazioni delle liste. Questo perché la struttura `Stack` non mette a disposizione alcuna funzione, come `length` sul tipo `stack` e ne nasconde l'implementazione.

5.10.3 Functors

Per definire strutture parametriche³ si usa la keyword `functor`.

Esempio

Ecco un esempio di definizione di immagini parametrica rispetto alla codifica del colore. Supponiamo di avere due strutture `RGB` e `CMYK` per gli omonimi modelli di colore e che entrambe le strutture implementino la signature `COLOR`. La struttura parametrica si può dichiarare cos':

```
1 functor Image (X : color) =  
2 struct  
3   (* qui si puo' usare X come un tipo *)  
4   (* con le operazioni definite da COLOR *)  
5 end;  
6  
7 structure Image_RGB = Image(RGB);  
8 structure Image_CMYK = Image(CMYK);
```



5.11

ECCEZIONI E INTEGRAZIONE CON TYPE CHECKING



5.11.1 Eccezioni di default e meccanismo

Come nel linguaggio Java, anche il linguaggio ML ha le sue eccezioni predefinite:

```
1 -3 div 0;  
2 uncaught exception Div [divide by zero]
```



In ML la gestione delle eccezioni è integrata con il type checking:

³Analogo dei template o dei Generics che fanno uso della notazione diamond `<T>`

Esempio

Si consideri la seguente porzione di codice:

```
1 - fun pop (x::s) = (x,s);
2 Warning : match nonexhaustive
3 x :: a => ...
4
5 - pop [];
6 uncaught exception Match [nonexhaustive match failure]
```



In questo caso:

1. La type inference capisce che l'input della funzione `pop` è una lista;
2. Il datatype di lista ha due costruttori: `::` e `[]`;
3. La definizione per casi di `pop` ha un caso solo per il costruttore `::` (da cui il warning);
4. Il compilatore inserisce automaticamente una eccezione **Match** nei casi mancanti.

5.11.2 ■ Eccezioni personalizzate

Il programmatore può definire le proprie eccezioni mediante la keyword **exception**. Per segnalare al compilatore che una funzione può sollevare una determinata eccezione si usa invece il costrutto **raise**.

Esempio

```
1 exception EmptyStack;    (* dichiara una nuova eccezione *)
2
3 fun pop(x::s) = (x,s)
4 | pop[] = raise EmptyStack;
```



Il risultato in caso di errore è più esplicativo dell'eccezione "automatica" **Match**:

```
1 - pop[];
2 uncaught exception EmptyStack
```



5.11.3 ■ La gestione delle eccezioni

Le eccezioni possono essere catturate e gestite mediante il costrutto **handle**.

```
1 pop x
2 handle EmptyStack =>
3   ( print "messaggio di errore personalizzato";
4     raise EmptyStack );
```



Il costrutto **handle** può essere messo dopo qualunque espressioni che può generare una eccezione e gestire anche diverse eccezioni. Ogni ordine è buono in quanto non c'è una sovrapposizione tra le varie eccezioni come invece accade all'interno del linguaggio Java.

Per usare il costrutto **handle** in ML funzionale puro esistono due modi:

1. Fare qualcosa come stampare un messaggio e rilanciare l'eccezione
2. "Aggiustare" l'errore restituendo un valore dello stesso tipo dell'espressione che ha sollevato l'eccezione.

IL PARADIGMA LOGICO: IL LINGUAGGIO PROLOG

6.1

INTRODUZIONE



La programmazione logica nacque agli inizi degli anni 70', grazie soprattutto alle ricerche di due studiosi:

- **Kowalski** ne elaborò i fondamenti teorici.
- **Colmerauer** fu il primo a progettare e implementare un interprete per un linguaggio logico: il Prolog.

Col passare del tempo l'interesse per questo nuovo tipo di programmazione si è allargato, uscendo dall'ambito accademico ed approdando nel mondo industriale. Nel 1981 i giapponesi, in un rapporto, elessero il Prolog (PROgramming LOGic) come linguaggio sul quale fondare la progettazione della V generazione di computer, caratterizzata da un elevato parallelismo. Attualmente il Prolog è il linguaggio di programmazione logica per eccellenza, e trova impiego nelle più svariate aree applicative dell'intelligenza artificiale e dell'ingegneria della conoscenza.

Idea alla base

Nel paradigma logico un problema viene descritto con un insieme di formule della logica, dunque in forma *dichiarativa*. Vengono fatte delle asserzioni logiche, cioè **dichiarazioni** che forniscono una definizione del problema stabilendo i fatti conosciuti. Le dichiarazioni sono costruite attraverso le **relazioni**.

Per imparare il linguaggio Prolog useremo il sistema SWI Prolog che nelle macchine Unix è installato come **swipl**:

```
Welcome to SWI-Prolog (threaded, 64 bits, version 9.0.4)
```

```
For online help and background, visit https://www.swi-prolog.org
```

```
For built-in help, use ?- help(Topic). or ?- apropos(Word).
```

```
?-
```

Dopo aver lanciato Prolog, è possibile caricare un programma inserendo il nome del file (la cui estensione sarà **.pl**) racchiuso tra parentesi quadre:

```
1 ? - [mioprolog.pl]. oppure
2 ? - consult(mioprolog.pl). quando si carica la prima volta;
3 ? - reconsult(mioprolog.pl). quando si ricarica dopo una correzione
```



Alternativamente, un programma può essere lanciato direttamente dal terminale **bash** usando il comando **swipl programma.pl** a patto che il codice sorgente sia presente nella working directory. La working directory solitamente è la cartella all'interno della quale è stato eseguito **swipl**.

6.2

COSTRUTTI DI BASE



I costrutti base della programmazione logica derivano dalla logica. Esistono tre tipologie di statement di base: fatti, regole e queries.

6.2.1 ■ Facts

Il tipo più semplice di statement è chiamato **fatto**. I fatti sono un mezzo attraverso il quale dichiarare una relazione tra oggetti.

Esempio

Un esempio può essere:

```
1 father(abraham, isaac)
```

Codice 6.1: Facts

father, oltre che relazione, è chiamato anche **predicato**. Questo statement dichiara che Abramo è padre di Isacco, o che la relazione **father** è tenuta tra gli individui chiamati **abraham** e **isaac**. I nomi degli individui sono chiamati anche **atomi**. I nomi dei predicati devono iniziare per lettera minuscola. Gli argomenti **abraham** e **isaac** iniziano con lettera minuscola perché costanti, come i numeri. Con i fatti è possibile definire ad esempio un database (vedi Listato 6.2). Ogni predicato corrisponde ad una *tabella relazionale*:

```
1 father(terach,abraham).
2 father(terach,nachor).
3 father(terach,haran).
4 father(abraham,isaac).
5 father(haran,lot).
6 father(haran,milcah).
7 father(haran,yiscah).
8 mother(sarah,isaac).
9 female(sarah).
10 female(milcah).
11 female(yiscah).
12 male(terach).
13 male(abraham).
14 male(nachor).
15 male(haran).
16 male(isaac).
17 male(lot).
```

Codice 6.2: Un database della famiglia biblica

6.2.2 ■ Queries

I programmi logici sono fatti per rispondere a delle interrogazioni (*queries* o *goals*). Se il programma caricato è quello mostrato nel Listato 6.2 allora:

```
1 ?- father(abraham,milcah).
2 false.
```



Le query hanno la stessa forma dei dati. Sono entrambi dei cosiddetti **atomi logici**.

6.2.3 ■ Variabili logiche nelle query

È utile chiedere cose come: *quali sono i figli di abraham?*. Si può fare con le **variabili logiche** che sono caratterizzate dalla lettere maiuscola:

```
1 ?- father(terach,X).
2 X = abraham ;
3 X = nachor ;
4 X = haran.
```



La macchina virtuale cerca i valori che, sostituiti a X, rendono la query uguale a uno dei fatti nel programma.¹ A differenza delle variabili degli altri paradigmi:

- Le variabili logiche rappresentano oggetti qualsiasi, non specificati;
- Le variabili dei linguaggi imperativi sono locazioni di memoria;

¹Questo vale per i programmi di soli fatti.

- Gli identificatori dei linguaggi funzionali denotano valori immutabili.

Le variabili logiche si possono usare anche nei fatti.

Query	Facts
Esistenzialmente quantificate	Universalmente quantificate

Tabella 6.1: Differenza tra le variabili nei fatti e nelle query

6.2.4 Termini

I **termini** sono l'unica struttura dati in Prolog². Si distinguono in: costanti, variabili logiche e **funtori** i quali hanno un ruolo simile ai costruttori di ML e sono caratterizzati da un nome e da una *arietà* (il numero di argomenti).

```
1 <term> ::= <constant> | <variable> | <functor name>('['<term>['<term>']*]')
```



Esempio

Ad esempio:

```
1 successor(Int)
2 date(25,april,2020)
3 color(rgb,0,0,1)
```



Gli argomenti di un funtore³ possono essere termini qualsiasi. Quindi la sintassi generale dei fatti è:

```
1 < fact > ::= < atomic formula > ' . '
2 < atomic formula > ::=
3 < predicate > '(' [ < term > [ ' , ' < term > ] * ] ')'
```



Quando la stessa variabile X compere più volte nello stesso fatto o nella stessa query significa che i termini nelle posizioni dove si trova X devono essere uguali tra loro.

Termini ground

Un termine è **ground** se non contiene variabili, altrimenti è **nonground**. Gli stessi aggettivi e gli stessi criteri si applicano ai fatti, alle query e anche alle regole.

6.2.5 Gli alberi

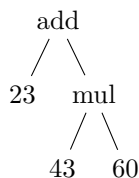
La struttura di base degli oggetti che il Prolog sa manipolare, è la struttura di **albero**. Le relazioni e i loro argomenti, infatti, sono degli alberi. Questa è una struttura sufficientemente ricca per rappresentare delle informazioni complesse, organizzate gerarchicamente, ma anche abbastanza semplice da manipolare, sia dal punto di vista algebrico che da quello informatico.

²In Prolog tutto è un termine: variabili, costanti, strutture.

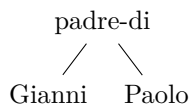
³Anche detta struttura

Esempio

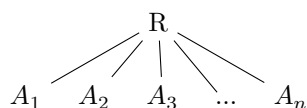
Ecco alcuni esempi di alberi. Per rappresentare l'espressione aritmetica $23 + 43 \cdot 60$ possiamo costruire il seguente albero:



Mentre, per la rappresentazione della relazione **padre-di**(Gianni,Paolo), possiamo costruire il seguente albero:



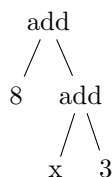
Più in generale, un albero A è costituito da un nodo R chiamato **radice** e da un insieme ordinato di elementi $A_1, \dots, A_n$ che sono essi stessi degli alberi che sono detti i figli di R . Le radici di $A_1, \dots, A_n$ sono i discendenti di R . Tutti i nodi senza discendenti sono detti **terminali** o **foglie**.



Spesso le strutture ad albero considerate non sono completamente note: comportano dei “buchi” rappresentati dalle variabili. Queste variabili possono ricevere un valore nel corso dell'esecuzione del programma, valore che è esso stesso una qualunque struttura ad albero.

Esempio

Si consideri il seguente albero:



Questo rappresenta l'infinità di alberi ottenuti rimpiazzando la variabile x con un albero qualunque. Una **informazione atomica** è la forma di albero più elementare; l'oggetto associato è una costante: identificatore, stringa di caratteri o intero.

Gli alberi sono la rappresentazione di quelli che abbiamo definito essere i **termini**. Giusto per ripetere, un termine è costruito mediante informazioni atomiche (costanti, nomi di funzioni, nomi di relazioni, ...), variabili, parentesi e virgole, che permettono di tradurre la struttura gerarchica dell'albero. Ad esempio, l'albero qui sopra è rappresentato dal termine:

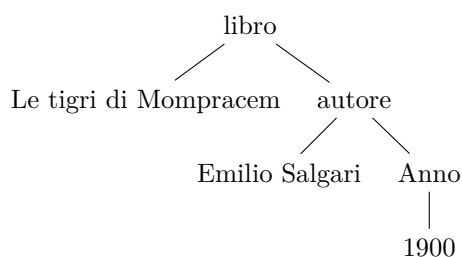
```
1 add(8,add(x,3));
```



Esempio

Si consideri il seguente termine:

```
1 libro('Le tigri di Mompracem',autore(Emilio Salgari,anno(1900))).
```



Questo termine si traduce nell'albero sintattico:

6.2.6 ■ Le regole

Regola

Una **regola** permette di definire nuove relazioni in termini di altre relazioni esistenti. In generale, la sintassi delle regole è:

```
<rule> ::= <atomic formula> [:- <goal list>]  
  
<goal list> ::= <atomic formula>[, <atomic formula>]*
```



dove la formula atomica prima di :- è detta **testa**, mentre quella successiva è detta **corpo**.

Osservazione



I fatti altro non sono che regole senza corpo.

Programma logico

Un **programma logico** è un insieme di regole.

6.3

STRUTTURA DI UN PROGRAMMA PROLOG



Come si è già detto, un programma Prolog è costituito da una serie di regole e fatti. Eseguire il programma consiste nel rispondere a una domanda riguardante queste relazioni. Da un punto di vista sintattico, una domanda è una successione di termini, o **obiettivi** (in inglese **goals**) che rappresenta una congiunzione di relazioni da soddisfare. Le risposte a questa domanda sono i **vincoli**, basati sulle variabili che compaiono nella successione di obiettivi e che soddisfano le relazioni considerate.

6.3.1 ■ Query semplici

Per rispondere ad una domanda, il Prolog utilizza il concetto di **pattern matching**. Se è presente un fatto che coincide con l'obiettivo o goal, allora la query ha successo e l'interprete risponde con “yes”. Se invece non sono presenti fatti corrispondenti allora la query fallisce e l'interprete restituisce “no”. Il pattern matching del linguaggio Prolog prende il nome di **unificazione**. Nel caso in cui l'insieme delle conoscenze contiene soli fatti, l'unificazione ha successo se avvengono queste tre condizioni:

- Il predicato usato nel goal e quello nel database hanno lo stesso nome;
- Entrambi i predicati hanno la stessa arietà;
- Tutti gli argomenti sono uguali.

Esempio

Si consideri il seguente listato:

```
1 room(kitchen).
2 room(office).
3
4 door(kitchen, office).
5
6 location(desk, office).
7 location(apple, kitchen).
8 location(broccoli, kitchen).
9 location(crackers, kitchen).
10 location(computer, office).
11
12 edible(apple).
13 edible(crackers).
14 here(kitchen).
```



Una prima query che è possibile pensare di fare è chiedere se l'ufficio sia una stanza:

```
1 ?-room(office).
2 yes
```



Allo stesso modo possiamo chiedere al programma se un determinato oggetto è presente in una stanza:

```
1 ?-location(apple,kitchen).
2 yes
```



Le variabili logiche aggiungono una nuova dimensione al meccanismo dell'unificazione. Come prima, i nomi dei predicati e l'arietà devono essere gli stessi. Tuttavia, quando gli argomenti corrispondenti sono a confronto, una variabile avrà sempre un esito affermativo. Dopo un'unificazione avvenuta con successo, la variabile logica assume il valore del termine col quale è stata accoppiata.

6.3.2 ■ Sostituzione

Sostituzione

Una **sostituzione** è un insieme finito di coppie:

$$\theta = \{X_1 = t_1, \dots, X_n = t_n\}$$

dove:

- Le X_i sono variabili e t_i termini.
- Le X_i sono tutte diverse tra loro.
- Nessuna delle X_i compare dentro i t_i .

L'applicazione di θ ad una espressione E (che potrebbe essere un termine, un fatto, una query o una regola) si denota con $E\theta$ consiste nella sostituzione di tutte le occorrenze delle variabili X_i in E con i rispettivi termini t_i .

Esempio

Consideriamo la query $E = \text{father}(\text{abraham}, X)$ allora, una possibile sostituzione è data da $\theta = \{X = \text{isaac}\}$. Si ottiene quindi:

$$E\theta = \text{father}(\text{abraham}, \text{isaac})$$

6.3.3 ■ Istanze

L'applicazione di una sostituzione θ a E crea un "caso particolare" di E in quanto le variabili di E vengono sostituite con valori specifici (termini ground) o parzialmente specificati (termini nonground).

E_1 è una **istanza** di E_2 se esiste θ tale che

$$E_1 = E_2\theta$$

Cioè se E_1 è un “caso particolare” di E_2 dove alcune variabili di E_2 sono *istanziate*, cioè legate ad un valore.

Esempio

Consideriamo la seguente espressione: $E = \text{hascolor}(o1, X)$ allora, data la sostituzione $\theta = \{X = \text{color}(\text{rgb}, Y, Y, Y)\}$ si ottiene l'istanza:

$$E\theta = \text{hascolor}(o1, \text{color}(\text{rgb}, Y, Y, Y))$$

E dice che l'oggetto $o1$ ha un colore non specificato mentre $E\theta$ offre una migliore specifica parziale: l'oggetto $o1$ ha un colore che è in formato RGB le cui componenti sono uguali (quindi una delle tre opzioni: RRR, GGG, BBB).

6.3.4 Vincoli sugli alberi

Come anticipato, il Prolog è un linguaggio che opera su strutture ad albero, dove tali alberi possono comprendere parti variabili. Uno degli elementi fondamentali di tutte le implementazioni di Prolog è l'algoritmo di unificazione, che si occupa di determinare se due alberi possono essere resi uguali. Nel caso affermativo, l'unificazione si realizza associando valori alle variabili presenti in uno dei due alberi dati.

Tuttavia, è possibile affrontare questa problematica da un punto di vista più ampio, considerando le **equazioni tra alberi**. La soluzione di un tale sistema, se esiste, consiste in un insieme di valori da assegnare alle variabili coinvolte, in modo che ciascuna equazione sia soddisfatta. Questi sistemi di equazioni sono comunemente denominati **vincoli**, e un vincolo rappresenta un insieme di vincoli elementari che devono essere soddisfatti simultaneamente.

Per risolvere questi sistemi di equazioni su alberi, l'interprete Prolog si avvale di un algoritmo di riduzione. Questo algoritmo procede attraverso semplificazioni successive del vincolo associato, cercando di trovare una soluzione compatibile che soddisfi l'uguaglianza tra alberi e, di conseguenza, assegni valori alle variabili coinvolte nel processo di unificazione. Questo approccio è fondamentale per il funzionamento del linguaggio Prolog, poiché consente di gestire in maniera efficiente e flessibile la manipolazione di dati strutturati rappresentati mediante alberi.

Un concetto chiave in questo contesto è il **Most General Unifier (MGU)**, che rappresenta la soluzione più generale per l'unificazione di termini o alberi. L'MGU è essenziale quando si trattano variabili, cercando di trovare l'associazione di valori alle variabili coinvolte nel modo più generale possibile durante il processo di unificazione.

Esempio

Dati due alberi $\mathbf{a}$ e $\mathbf{a}'$, si cerca se l'equazione $\mathbf{a}=\mathbf{a}'$ possiede una soluzione nel dominio degli alberi. Per questo si decompongono gli alberi $\mathbf{a}$ e $\mathbf{a}'$ nei loro rispettivi sottoalberi. Se le radici di $\mathbf{a}$ e $\mathbf{a}'$ sono identiche hanno lo stesso numero di discendenti si può scrivere:



Il vincolo iniziale $C_0 = \{a = a'\}$ è rimpiazzato dal vincolo:

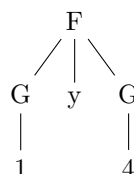
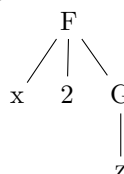
$$C_1 = \{a_1 = a'_1; a_2 = a'_2; \dots; a_n = a'_n\}$$

e si procede nello stesso modo su ognuno dei nuovi vincoli ottenuti in C_1 e ci si arresta quando il vincolo corrente è ridotto. Cioè:

- Contiene una uguaglianza tra due alberi che non hanno la stessa radice o le cui radici non hanno lo stesso numero di discendenti, nel qual caso il sistema iniziale non ha soluzione;
- Non contiene che uguaglianze i cui membri sono delle variabili tutte distinte, nel qual caso il vincolo corrente rappresenta la soluzione del sistema iniziale.

Esempio

Siano dati ad esempio gli alberi a e a' :



Scrivendo questi due alberi sotto forma di termini, l'uguaglianza $a=a'$ si esprime così:

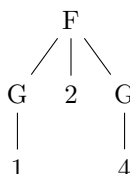
$$C_0 = \{F(x, 2, G(z)) = F(G(1), y, G(4))\}$$

L'algoritmo di riduzione dei vincoli si applica e produce in successione:

$$C_1 = \{x = G(1); y = 2; G(z) = G(4)\}$$

$$C_2 = \{x = G(1); y = 2; z = 4\}$$

che è la soluzione del sistema iniziale e da come forma comune di a e a' :



6.3.5 L'unificazione

Il matching tra query e fatti e la conseguente costruzione delle risposte avviene mediante il processo di **unificazione**. L'algoritmo di unificazione prende due espressioni E_1 ed E_2 e, se possibile, restituisce una sostituzione θ tale che $E_1\theta = E_2\theta$ detta **unificatore** (*unifier*).

L'unificatore costruito dall'algoritmo viene detto **most general unifier** (*mgu*) perché non sostituisce una variabile con un termine se non è necessario, cioè vincola il meno possibile il risultato $E_1\theta$ lasciando le variabili quando può. Tecnicamente, ogni altro unificatore θ' porta ad una *istanza* di $E_1\theta$, cioè esiste una sostituzione σ tale che: $(E_1\theta)\sigma = E_1\theta'$.

Nei programmi visti fino a questo momento (che consistono di soli fatti) le risposte ad una query: $q(t_1, \dots, t_n)$ vengono costruite così:

1. Si cerca nel programma il primo fatto $p(u_1, \dots, u_m)$ con lo stesso funtore (cioè $p = q$ e $m = n$). Se ne trova uno allora:
2. Si calcola $\theta = mgu(q(t_1, \dots, t_n), p(u_1, \dots, u_m))$
3. Se esiste θ allora:
 - Si restituisce θ come risposta (Se è vuota allora Prolog dice **true**)
 - Se poi l'utente non vuole altre soluzioni si termina
4. Altrimenti si cerca il prossimo fatto con lo stesso funtore. Se esiste si ripete da 2, altrimenti si termina.

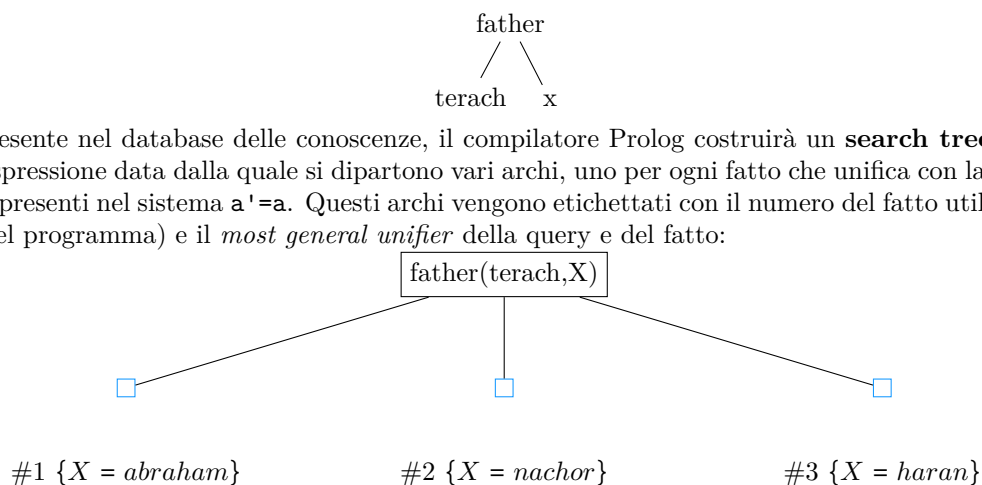
Prolog dice **false** quando nessun fatto unifica con la query.

Esempio

Consideriamo la query:

```
1 ?-father(terach,X).
```

Che è equivalente ad un albero della forma:



Per ogni fatto presente nel database delle conoscenze, il compilatore Prolog costruirà un **search tree** per la query che avrà in radice l'espressione data dalla quale si dipartono vari archi, uno per ogni fatto che unifica con la query, ovvero che soddisfa i vincoli presenti nel sistema $a' = a$. Questi archi vengono etichettati con il numero del fatto utilizzato (nell'ordine in cui compare nel programma) e il *most general unifier* della query e del fatto:

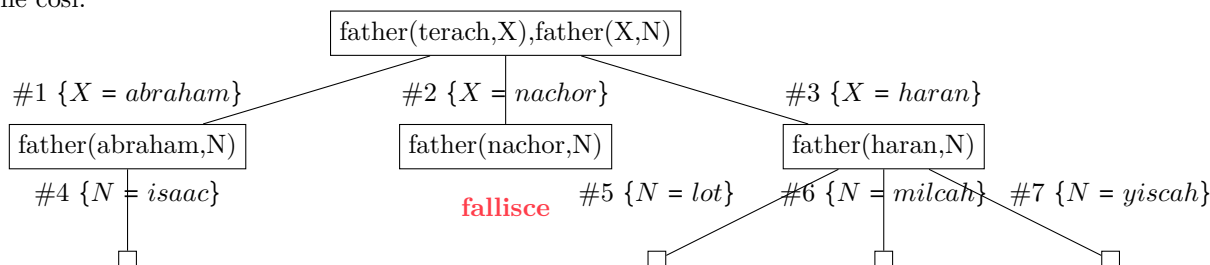
Esempio

Le query possono contenere più formule atomiche. Ad esempio, stando al Listato 6.2 si può chiedere chi sono i nipoti di terach:

```
1 father(terach,X),father(X,Nipote).
```

Codice 6.3: Conjunctive queries

dove la virgola rappresenta un AND logico (è come una join). In questo caso prendono il nome di **query congiuntive**. Si ottiene così:

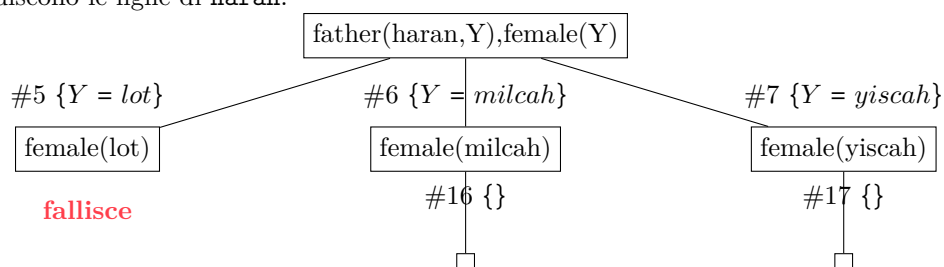


Esempio

Le figlie di haran possono essere trovate con una query congiuntiva:

```
1 ?-father(haran,Y), female(Y).
```

I valori di Y restituiscono le figlie di haran:



Esempio

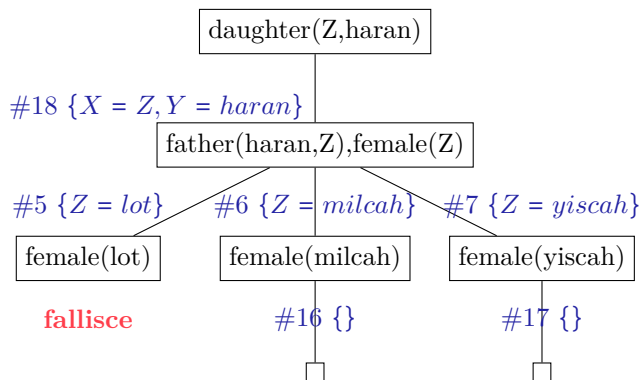
La relazione di parentela *padre-figlia* è esprimibile attraverso la seguente regola: “X è figlia di Y se Y è padre di X e X è femmina”:

```
1 daughter(X,Y) :- father(Y,X), female(X).
```

Le risposte alla query:

```
1 ? - daughter(Z,haran).
```

si troveranno quindi cercando una testa che unifica con la query. Si effettua poi una sostituzione nella query inserendo il corpo istanziato con il *mgu*:



Attenzione

Bisogna fare attenzione con le variabili: quelle della query devono essere sempre diverse da quelle della regola per non incorrere in problemi di shadowing. Per evitare questo problema, ogni volta che una regola viene usata la WAM^a ridenomina le sue variabili.

^aWarren Abstract Machine, introdotta da David H. D. Warren, è stata progettata per eseguire efficientemente i programmi scritti in Prolog. La macchina astratta WAM fornisce un ambiente di esecuzione per i programmi Prolog, consentendo di interpretare ed eseguire le istruzioni Prolog in modo più efficiente rispetto a un'interpretazione diretta del codice sorgente. La WAM è stata sviluppata per migliorare le prestazioni dell'esecuzione dei programmi Prolog, che altrimenti potrebbero essere lenti a causa della natura ricorsiva della logica e delle operazioni di unificazione.

6.3.6 Overloading e Wildcards

In Prolog, il termine **overloading** si riferisce alla capacità di definire più predicati con lo stesso nome ma con arietà diversa. L'arietà di un predicato è il numero di argomenti che accetta.

Esempio

Ecco un semplice esempio di overloading in Prolog:

```
1 % Predicato con arità 1
2 predicato_esempio(X) :-
3 % Implementazione del predicato per arità 1
4 write('Predicato con arità 1 chiamato con \\'argomento '), write(X), nl.
5
6 % Predicato con arità 2
7 predicato_esempio(X, Y) :-
8 % Implementazione del predicato per arità 2
9 write('Predicato con arità 2 chiamato con gli argomenti '), write(X), write(' e '), write(Y), nl.
```

In questo esempio, ci sono due predicati chiamati `predicato_esempio`, uno con arietà 1 e un altro con arietà 2. Prolog li distinguerà in base al numero di argomenti forniti quando il predicato viene chiamato.

Per quanto riguarda le **wildcards** in Prolog, Prolog utilizza l'underscore (`_`) come carattere jolly. Viene spesso utilizzato come segnaposto per le variabili quando si desidera ignorare o non importa di un particolare valore.



Finora abbiamo considerato solo semplici termini come argomenti per i nostri programmi. Tuttavia una struttura di dati molto comune in Prolog è la lista. La rappresentazione è simile a quella in ML: sono sempre racchiuse tra parentesi quadre e ciascun elemento è separato da una virgola.

Prolog fornisce vari costruttori per definire le proprie liste:

- costruttore di lista vuota: `[]`;
- costruttore nodi: `[elem|resto]`

Attraverso il simbolo `|` è possibile separare la testa della lista dal resto (la coda).

Esempio

Si ha quindi:

```
1 [first,second,third] = [A|B].
```

Si avrà quindi grazie al pattern matching: `A=first` e `B=[second,first]`.

Grazie alle variabili possiamo anche definire liste parzialmente specificate:

```
1 [a,b,c|X]. /* il resto della coda è variabile */
2 [a,X,b|Y]. /* alcuni elementi della lista e la coda sono variabili */
```

6.4.1 Predicati predefiniti sulle liste

Il predicato `member`

Grazie alla sintassi vista finora possiamo definire un predicato che possa cercare un elemento `X` in una lista `L`. L'obiettivo del predicato `member` siffatto deve essere quello di verificare la presenza di una variabile `X` in una lista `L`. Per definire tale predicato possiamo osservare che un elemento `X` è presente in `L` se:

- `X` è la testa di `L`
- `X` è presente nella coda di `L`

```
1 member(X,[X|_]). /* X e' la testa di L */
2 member(X,[_|Tail]) :- member(X,Tail). /* X e' presente nella coda */
```

Codice 6.4: Il predicato `member`

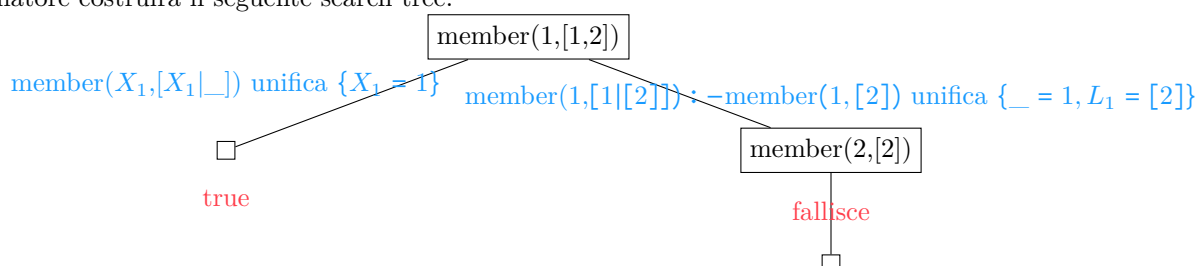
Chiaramente tale predicato risulta ricorsivo e fa uso delle wildcards. Infatti un elemento di una lista o è la sua testa oppure è la testa di una sottolista.

Esempio

Consideriamo la seguente query:

```
1 member(1,[1,2]).
```

Il compilatore costruirà il seguente search tree:



Il secondo ramo a destra non viene esplorato in quanto la query è ground.

In Prolog, i parametri “in” e “out” non sono concetti esplicitamente definiti come in alcuni altri linguaggi di programmazione, come ad esempio C o Java. Tuttavia, il modo in cui Prolog gestisce gli argomenti di un predicato e il flusso delle informazioni può essere spiegato in modo analogo.

Gli argomenti IN rappresentano dati forniti al predicato per essere utilizzati durante l'esecuzione. In Prolog, tutti gli argomenti di un predicato sono considerati argomenti IN di default. Gli argomenti OUT rappresentano dati che vengono restituiti dal predicato dopo l'esecuzione. In Prolog, ciò è spesso ottenuto mediante unificazione o attraverso il valore restituito del predicato. In Prolog, la comunicazione tra il chiamante e il predicato avviene attraverso l'**unificazione** e il **binding di variabili**. Pertanto, tutti gli argomenti possono essere considerati implicitamente bidirezionali, e la terminologia IN e OUT non è rigorosamente applicata come in altri linguaggi.

Calcolo della lunghezza di una lista: il predicato `length`

Cerchiamo di definire un predicato `length(L,N)` che calcoli la lunghezza di una lista. Il predicato riceve in input gli argomenti L ed N. Chiaramente la lunghezza di una lista sarà:

$$\begin{cases} 0 & \text{Se la lista è vuota} \\ 1 + \text{lunghezza}(\text{Coda}) & \text{altrimenti} \end{cases}$$

Si ha quindi il seguente programma:

```
1 length([],0).
2 length([_|Tail],N) :- length(Tail,N1), N is N1+1.
```



La concatenazione di liste: il predicato `append`

Scriviamo un predicato che effettui la concatenazione di due liste e restituisca la concatenazione nella terza lista passata come argomento:

```
1 append(Lista1, Lista2, Risultato)
```



Logicamente, definiamo la concatenazione nel seguente modo:

1. Se la prima lista è vuota e la seconda lista è L allora il risultato sarà L;
2. Se la prima lista non è vuota allora è possibile definirla come `Testa|Coda`. Possiamo quindi effettuare una concatenazione ricorsiva della coda con la seconda lista e salvare il risultato in una nuova lista del tipo `Testa|New List`.

Si ottiene quindi:

```
1 append([],L,L).
2 append([X|L1],L2,[X|L3]) :- append(L1,L2,L3).
```



Esempio

Possiamo usare il predicato `append` per la generazione di tutti i prefissi e i suffissi di una lista:

```
1 prefix(Prefix,List) :- append(Prefix,_,List).
2 suffix(Suffix,List) :- append(_,Suffix,List).
```



Oppure, con il predicato `member`:

```
1 member(X,L) :- append(_,[X|_],L).
```



Esempio

Con i predicati **prefix** e **suffix** si possono calcolare le sottoliste di una lista. Una lista **S** è una sottolista di **L** se valgono queste due condizioni:

- **S** è il prefisso di un suffisso di **L**;
- **S** è il suffisso di un prefisso di **L**.

```
1 sublist(Sub,L) :- prefix(Pre,List),suffix(Sub,Pre).
```



La cancellazione da una lista: il predicato delete

Supponiamo di avere una lista **L** ed un elemento **X** da eliminare. Bisogna considerare tre casi:

1. Se **X** è l'unico elemento, allora dopo la sua eliminazione bisogna restituire una lista vuota;
2. Se **X** è la testa di **L**, la lista risultante deve essere la coda;
3. Se **X** è all'interno della coda, allora sarà la testa di una sottolista.

Si ottiene quindi:

```
1 delete(X,[X],[]).
2 delete(X,[X|L1],L1).
3 delete(X,[Y|L2],[Y|L1]) :- list_delete(X,L2,L1).
```



Esempio

Definiti tali predicati, è possibile eseguire molteplici query:

```
1 ? - member(X,[1,2,3]), X>2.
2 X = 3
3
4 ? - append(X,[4,5],[1,2,3,4,5]).
5 X = [1,2,3]
6
7 ? - delete([1,2,3,3,1,4],1,X).
8 X = [2,3,4]
9
10 ? - delete([1,2,1,3,1,4],X,[2,3,4]).
11 X = 1
```

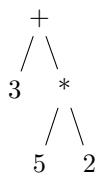


6.5

CALCOLO SIMBOLICO IN PROLOG



Grazie alla flessibilità del paradigma logico il linguaggio Prolog si può prestare ad alcune forme di calcolo simbolico. Infatti, come già visto in precedenza, è possibile tradurre una espressione aritmetica mediante una relazione gerarchica esprimibile mediante un albero sintattico. Ad esempio, l'espressione $3+5*2$ non è uguale a 13 bensì denota l'albero sintattico:



Per calcolare tale espressione si usa la keyword **is**:

```
1 Ris is 3+5*2.
```



Tale goal sarà vero se **Ris** è uguale al valore di $3+5*2$.

Il predicato is

In Prolog, il predicato **is** è utilizzato per valutare espressioni aritmetiche. Questo è particolarmente utile quando si desidera assegnare un valore numerico a una variabile o quando si desidera verificare se un'espressione aritmetica è vera. La forma generale di **is** è:

```
1 <Variabile> is <Expr>.
```

Codice 6.5: Sintassi della keyword **is**

È possibile usare il predicato **is** all'interno delle regole. Ad esempio:

```
1 increase(S,T) :- T is S+1.
```



6.6

PROGRAMMAZIONE NONDETERMINISTICA



La programmazione nondeterministica è una caratteristica chiave di Prolog che consente di esprimere soluzioni a problemi in modo dichiarativo. In Prolog, un programma può definire più possibili soluzioni per una query e il sistema Prolog può cercare tutte le soluzioni possibili attraverso un processo noto come **backtracking** (la visita nel search tree).

6.6.1 Il backtracking

Il backtracking è il meccanismo mediante il quale Prolog esplora tutte le possibili alternative quando cerca una soluzione a una query. Quando una regola fallisce durante l'esecuzione di una query, Prolog "torna indietro" (backtrack) alla scelta precedente e cerca ulteriori soluzioni. In sintesi, la programmazione nondeterministica in Prolog consente di esprimere relazioni logiche senza specificare l'ordine delle operazioni o come ottenere i risultati. Il backtracking è il meccanismo che consente a Prolog di esplorare tutte le possibili soluzioni in modo efficiente. Il suo schema generale può essere riassunto nelle specifiche **generate and test**:

```
1 solution(X) :- generate(X), test(X).
```



Prolog, infatti, genera via via tutte le risposte candidate e per ciascuna risposta verifica se è effettivamente una soluzione. Se sì, restituisce la soluzione; altrimenti fa **backtrack** e torna a generare il candidato successivo. Se si chiedono infine altre risposte, genera altre soluzioni.

ESERCIZI E APPROFONDIMENTI

7.1

IL PARADIGMA IMPERATIVO



7.1.1 ■ Le funzioni *mem* ed *env*

Esercizio

Denota le parti sinistra e destra dei seguenti assegnamenti:

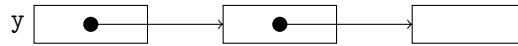
1. $x = y[3]$
2. A sinistra: $env(x) + 3$. A destra: $mem(mem(env(y)) + 1)$
3. $x = y[2]$
4. $x = *(z+i)$ dove z è un puntatore e i è una variabile di tipo `int`
5. $x[1] = *y$
6. $x[1] = \&y$
7. $*(x+1) = y[0]$, dove x è un puntatore.

1. Stiamo assegnando alla variabile x il valore contenuto nella terza cella del vettore y . Si avrà quindi a sinistra: $env(x)$, mentre a destra bisogna innanzitutto prendere l'indirizzo del vettore che è contenuto nella variabile y , aggiungere un offset di tre celle per raggiungere la cella desiderata e infine leggere il contenuto di tale cella mediante la funzione *mem*: $mem(env(y) + 3)$;
2. L'espressione $env(x) + 3$ sta a significare che ci si sta riferendo all'indirizzo della terza cella a partire dall'indirizzo di memoria contenuto nella variabile x . Questa è esprimibile quindi sia con la notazione vettoriale $x[3]$ oppure, mediante l'operatore $*$, come $*(x+3)$. A destra, invece, l'espressione $mem(mem(env(y)) + 1)$ denota il valore contenuto nella cella ottenuta incrementando di un offset l'indirizzo puntato dal puntatore y (ottenuto calcolando la funzione *mem* sull'indirizzo di y) ovvero $y[1]$. Si ha quindi $*(x+3) = y[1]$;
3. A sinistra basterà semplicemente applicare la funzione *env* all'identificatore x : $env(x)$. A destra si sta leggendo il valore memorizzato nella seconda cella del vettore y : sarà quindi necessario innanzitutto calcolare l'indirizzo di tale cella sommando un offset di due all'indirizzo della prima cella ($env(y) + 2$) e infine leggere mediante la funzione *mem*: $mem(env(y) + 2)$;
4. A sinistra: $env(x)$. A destra si sta leggendo il valore contenuto nella cella di memoria ottenuta accedendo alla cella puntata dal puntatore z e sommando l'offset espresso dalla variabile i . È necessario quindi ottenere l'indirizzo salvato in z ($mem(env(z))$), incrementare di un offset salvato nella variabile i ($mem(env(i))$), sommare le due quantità ($mem(env(z)) + mem(env(i))$) ed infine applicare la funzione *mem* all'indirizzo così ottenuto: $mem(mem(env(z)) + mem(env(i)))$;
5. A sinistra: $env(x) + 1$. A destra si sta calcolando il valore contenuto all'interno della cella puntata da y : per ottenere tale valore sarà quindi necessario leggere l'indirizzo contenuto in y e su tale indirizzo applicare la funzione *mem*. Si ha quindi a destra: $mem(mem(env(y)))$;
6. Questo è un esempio di assegnazione non corretta in quanto si sta provando ad assegnare un indirizzo ($\&y$) ad una variabile che non è un puntatore.
7. Nel caso in cui x sia un puntatore, l'espressione $*(x+1)$ esprime l'indirizzo della cella ottenuta incrementando di uno la cella puntata da x . Si ottiene così: $mem(env(x)) + 1$. A destra si ha invece il valore contenuto nella prima cella del vettore y : $mem(env(y))$.

Esercizio

Esprimere in termini di *mem* e *env* le parti sinistra e destra dell'assegnamento: $x = (*y)$, sapendo che *y* è un puntatore a puntatore.

A sinistra abbiamo semplicemente $env(x)$ mentre a destra si sta leggendo il valore della cella puntata dalla cella puntata da *y*, sarà quindi necessario innanzitutto ottenere l'indirizzo della cella puntata calcolando $mem(env(y))$, una volta ottenuto tale indirizzo è necessario applicare nuovamente la funzione *mem* per ottenere l'indirizzo puntato dalla seconda cella ed infine leggere il valore della terza cella: $mem(mem(mem(env(y))))$.



Esercizio

Scrivere un assegnamento le cui parti sinistra e destra corrispondono rispettivamente a

$$mem(env(a)) + 4 = mem(env(y) + 1)$$

Dagli esercizi precedenti abbiamo che $mem(env(a)) + 4$ si traduce nell'ivalore $*(a+4)$. A destra, invece, l'espressione denota la lettura della prima cella del vettore *y*, ovvero $y[1]$. Si ottiene quindi: $*(a+4)=y[1]$.

Esercizio

Esprimere in termini di *mem* ed *env* le parti sinistra e destra dell'assegnamento $*(p)=\&(y[4])$, sapendo che *y* è un vettore.

Supponendo che *p* sia un puntatore ad un puntatore, l'espressione $*(p)$ denota l'indirizzo della cella puntata dal puntatore puntato da *p*. È necessario quindi ottenere innanzitutto l'indirizzo di memoria salvato in *p* mediante la funzione $mem(env(p))$ e successivamente applicare nuovamente la funzione *mem* per ottenere l'indirizzo della cella a sua volta puntata dal secondo puntatore. Si ottiene così a sinistra $mem(mem(env(p)))$. Avendo *a* che fare con un puntatore a puntatore, è lecito salvare in $*(p)$ un indirizzo di memoria: l'espressione $\&(y[4])$ denota infatti l'indirizzo della quinta cella del vettore *y*. Tale indirizzo è facilmente ottenibile sommando all'indirizzo iniziale di *y* (ovvero al valore $env(y)$) un offset di 4: si ottiene così $env(y) + 4$.

Esercizio

Scrivere un assegnamento le cui parti sinistra e destra corrispondono rispettivamente a

$$mem(env(x)) = env(y) + mem(env(a))$$

Quando a sinistra si ha un'espressione in cui compare come funzione esterna la funzione *mem* si ha *a* che fare con dei puntatori. Quando si effettuare una assegnazione ad un puntatore si sta modificando l'indirizzo a cui sta puntando, tale valore si indica con $*x$. A destra, invece, bisogna ottenere un'espressione che esprima l'indirizzo da salvare nel puntatore *x*: tale indirizzo è ottenuto sommando all'indirizzo di *y* l'offset contenuto nella variabile *a*, ovvero: $\&y[a]$.

Esercizio

Esprimere in termini di *mem* ed *env* le parti sinistra e destra dell'assegnamento $*(a+4)=*(y)$, sapendo che *y* è un puntatore a puntatore.

Si ha a sinistra $mem(env(a)) + 4$ mentre a destra $mem(mem(mem(env(y))))$.

Esercizio

Scrivere un assegnamento le cui parti sinistra e destra corrispondono rispettivamente a

$$mem(env(a)) = mem(env(y) + 5)$$

Si ha l'assegnamento: $*a=y[5]$.

Esercizio

Esprimere in termini di *mem* e *env* le parti sinistra e destra dell'assegnamento $x[2]=y[a]$, sapendo che *x* è un vettore e *y* è un puntatore.

Sapendo che *x* è un vettore, per esprimere l'indirizzo della seconda cella di memoria basterà sommare un offset di due all'indirizzo della prima cella, ovvero $env(x) + 2$. A destra, essendo *y* un puntatore invece, accedere al valore $y[a]$ significa accedere all'indirizzo ottenuto sommando il valore contenuto nella variabile *a* all'indirizzo puntato da *y*: $mem(env(y) + mem(env(a)))$.

Esercizio

Scrivere un assegnamento le cui parti sinistra e destra corrispondono rispettivamente a

$$env(x) + mem(env(a)) = env(y) + mem(env(a))$$

Si ha l'assegnamento $*(x+a)=\&y[a]$.

Esercizio

Esprimere in termini di mem e env le parti sinistra e destra dell'assegnamento $*(a+4)=y[*p]$, sapendo che y è un vettore.

Si ha a sinistra l'espressione $mem(env(a)) + 4$ mentre a destra $mem(env(y) + mem(mem(env(p))))$.

Esercizio

Scrivere un assegnamento le cui parti sinistra e destra corrispondono rispettivamente a $env(x) = mem(mem(env(y)))$.

Si ha l'assegnamento $x=*y$.

Esercizio

Esprimere in termini di mem e env le parti sinistra e destra dell'assegnamento $*(p+z)=*y$, sapendo che y è un puntatore.

Tale assegnamento ha senso quando si ha a che fare con un puntatore ad un array di puntatori. In questo caso si sta modificando il valore della z -esima area di memoria puntata da un tale array. Per ottenere tale indirizzo bisognerà innanzitutto ottenere l'indirizzo della prima cella contenuto nel puntatore p , sommare il valore dell'offset contenuto nella variabile z ed infine calcolare l'indirizzo contenuto nel puntatore così ottenuto. Si ha allora:

$$mem(mem(env(p)) + mem(env(z))) = mem(mem(env(y)))$$

Esercizio

Scrivere un assegnamento le cui parti sinistra e destra corrispondono rispettivamente a $env(x)$ e $env(y)$.

Si ha $x=\&y$.

Esercizio

Esprimere in termini di mem ed env le parti sinistra e destra dell'assegnamento $*(x)=\&(y[*p])$, sapendo che x è un puntatore a puntatore e che y è un vettore.

Si ha: $mem(mem(env(x))) = env(y) + mem(mem(env(p)))$.

Esercizio

Scrivere un assegnamento le cui parti sinistra e destra corrispondono rispettivamente a

$$mem(env(x)) = mem(env(y))$$

Si ha $*x=y$.

Esercizio

Esprimere in termini di mem ed env le parti sinistra e destra dell'assegnamento $x[a]=\&(y[5])$, sapendo che x è un vettore e che y è un vettore.

Si ha: $env(x) + mem(env(a)) = env(y) + 5$.

Esercizio

Scrivere un assegnamento le cui parti sinistra e destra corrispondono rispettivamente a

$$env(x) + 1 = mem(mem(env(y)))$$

Si ha $x[1]=*y$.

Esercizio

Esprimere in termini di *mem* ed *env* le parti sinistra e destra dell'assegnamento $*x=y[a]$, sapendo che *x* è un puntatore e che *y* è un puntatore.

Si ha: $mem(env(x)) = mem(mem(env(y)) + mem(env(a)))$.

Esercizio

Scrivere un assegnamento le cui parti sinistra e destra corrispondono rispettivamente a

$$mem(env(a)) + 5 = mem(mem(env(y)))$$

Si ha $*(a+5)=*y$.

Esercizio

Esprimere in termini di *mem* e *env* le parti sinistra e destra dell'assegnamento $*x=*y$, sapendo che *x* è un puntatore e che *y* è un puntatore.

Si ha: $mem(env(x)) = mem(mem(env(y)))$.

Esercizio

Scrivere un assegnamento le cui parti sinistra e destra corrispondono rispettivamente a

$$mem(env(a)) + mem(env(z)) = env(y) + mem(env(a))$$

Si ha $*(a+z)=\&y[a]$.

Esercizio

Esprimere in termini di *mem* e *env* le parti sinistra e destra dell'assegnamento $*x=y$, sapendo che *x* è un puntatore.

Si ha $mem(env(x)) = mem(env(y))$.

Esercizio

Scrivere un assegnamento le cui parti sinistra e destra corrispondono rispettivamente a $mem(env(a)) = mem(env(y))$

Si ha $*a=y$.

Esercizio

Esprimere in termini di *mem* e *env* le parti sinistra e destra dell'assegnamento $*x=y[4]$, sapendo che *x* è un puntatore e che *y* è un vettore.

Usare un puntatore deferenziato a sinistra di un assegnamento significa modificare il valore all'interno della cella puntata, ciò significa avere a sinistra un'espressione del tipo $mem(env(x))$. A destra, invece, bisogna ottenere un'espressione che descriva il valore contenuto all'interno della quinta cella del vettore *y*, tale valore è ottenuto calcolando l'espressione $mem(env(y) + 4)$.

Esercizio

Esprimere in termini di *mem* e *env* le parti sinistra e destra dell'assegnamento $*x=\&y$, sapendo che *x* è un puntatore.

Si ha: $mem(env(x)) = env(y)$.

Esercizio

Scrivere un assegnamento le cui parti sinistra e destra corrispondono rispettivamente a $env(x) = mem(mem(env(y))+1)$.

Si ha: $x=*(y+1)$

Esercizio

Esprimere in termini di *mem* e *env* le parti sinistra e destra dell'assegnamento $*(x)=y$, sapendo che *x* è un puntatore.

Si ha: $mem(mem(env(x))) = mem(env(y))$.



7.2.1 Lo stack di attivazione

Esercizio

Si consideri il seguente programma:

```

1  program p1
2  int x; int y; int z; int t;
3  procedure p2([IN x copia] int y, [IN x copia] int x)
4  int z;
5  BEGIN
6  z=x;
7  x=4;
8  t=z-1;
9  write(x,y,z,t);
10 END
11
12 procedure p3([MODE] int x, [IN x copia] int t)
13 int z;
14 procedure p4([IN x copia] int x)
15 int t;
16 BEGIN
17 t = x-2;
18 z=4;
19 y=y;
20 p2(x,t);
21 write(x,y,z,t);
22 END
23
24 BEGIN
25 z=t;
26 x=z*2;
27 t=x;
28 y=z*1;
29 p4(y);
30 write(x,y,z,t);
31 END
32
33 BEGIN
34 x=1;
35 y=1;
36 z=1;
37 t=2;
38 p3(z,y);
39 write(x,y,z,t);
40 END

```



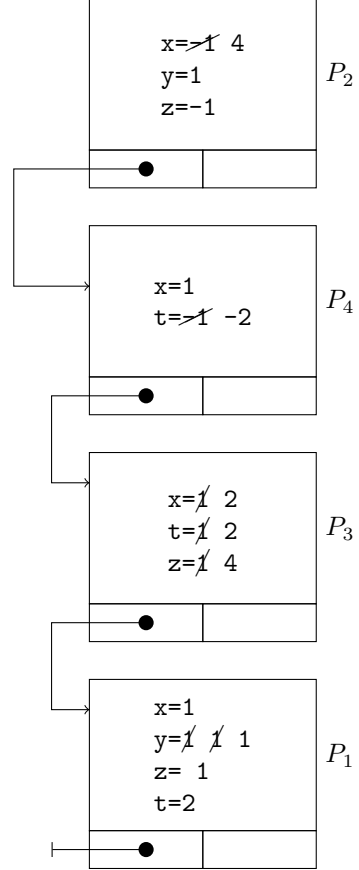
Determinare l'output e mostrare gli stack di attivazione, trannei nei casi di errore, nei quali bisogna invece indicare l'istruzione che causa l'errore, nei seguenti casi:

- Scoping dinamico, MODE = IN per copia;
- Scoping dinamico, MODE = IN per riferimento;
- Scoping statico, MODE = IN per copia;
- Scoping statico, MODE = IN OUT per riferimento;
- Scoping dinamico, MODE = IN OUT per copia;

Ciascuna procedura è rappresentata mediante il suo stack di attivazione. Per indicare l'ambiente non locale di ciascun record di attivazione si usa una freccia che, partendo dal record della procedura chiamata, a seconda del tipo di propagazione arriverà nel blocco della procedura chiamante nel caso di scoping dinamico o nel blocco di definizione della procedura nel caso di scoping statico. All'interno di ciascun record si descrivono le variabili locali definite nella procedura. Nel caso di passaggio per copia si mostra il valore della variabile locale mentre, nel caso di passaggio per riferimento si usa indicare con una freccia il legame¹.

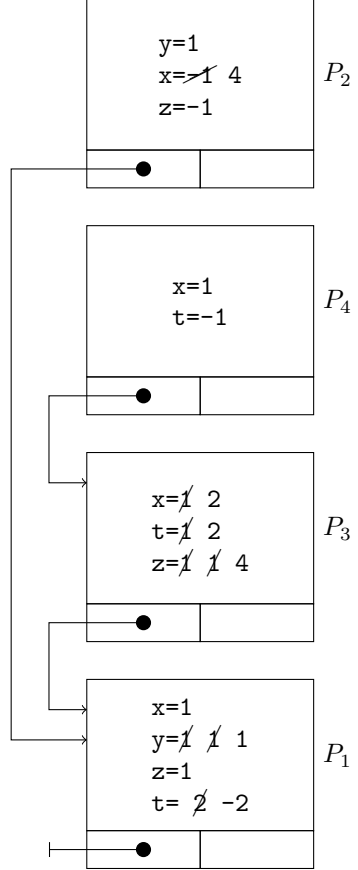
¹Si ricordi che in questo caso ogni assegnamento modificherà il valore della variabile originale mentre nel caso di passaggio per copia, ogni modifica è isolata nel blocco corrente.

Scope dinamico, MODE= IN per copia Si ha:



Output:
p2: 4 1 -1 -2
p4: 1 1 4 -2
p3: 2 1 4 2
p1: 1 1 1 2

Scope statico, MODE= IN per copia Si ha:

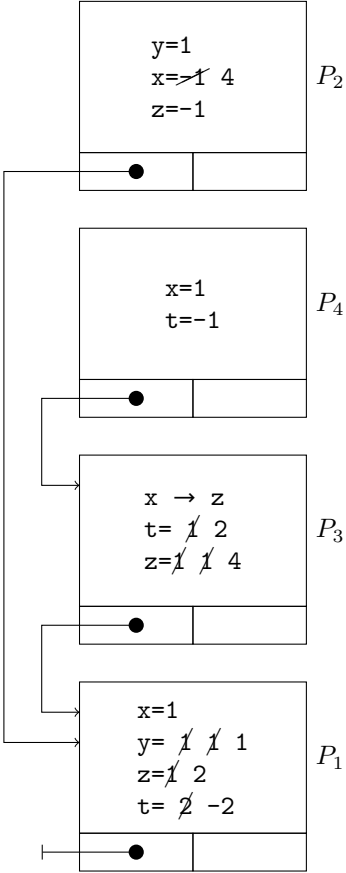


Output:
p2: 4 1 -1 -2
p4: 1 1 4 -1
p3: 2 1 4 2
p1: 1 1 1 -2

Scope dinamico, MODE= IN per riferimento Nel caso del passaggio dei parametri IN per riferimento bisogna stare attenti nel caso in cui c'è un tentativo di effettuare una scrittura sul parametro. Infatti, in questi casi, il compilatore restituirebbe un errore e il calcolo dell'output sarebbe impossibile. In questi casi² quindi, basta controllare se c'è una istruzione che tenta di modificare una variabile ricevuta con modalità IN per riferimento. In questo caso, è presente l'istruzione `x=z*2` che causa errore. Non va indicato quindi l'output e i vari record di attivazione.

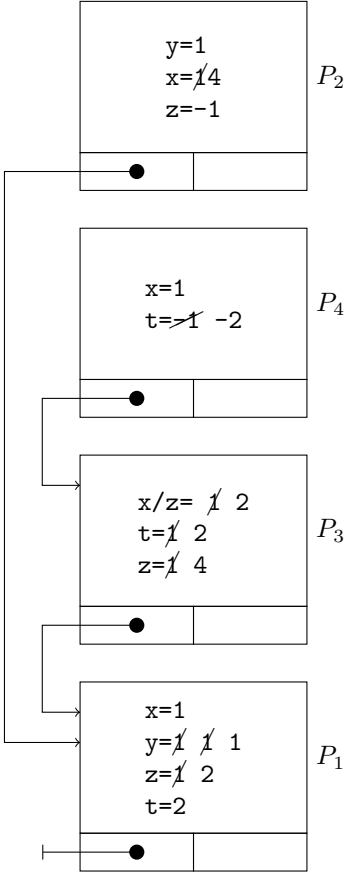
²Il 99,9% delle volte, negli esercizi sul passaggio di parametri, il programma della traccia è scritto in modo da dare errore nel caso di passaggio con modalità IN per riferimento.

Scope statico, MODE=IN/OUT per riferimento
 Per indicare che un parametro formale contiene il riferimento di un parametro attuale si usa il simbolo $\rightarrow$. In questa tipologia di esercizi bisogna stare attenti a non confondere lo scoping delle variabili non locali con il riferimento delle variabili passate con modalità IN/OUT per riferimento. Si ha quindi:



Output:
 p2: 4 1 -1 -2
 p4: 1 1 4 -1
 p3: 2 1 4 2
 p1: 1 1 2 -2

Scope dinamico, MODE=IN/OUT per copia
 Quando un parametro è di tipo IN/OUT per copia vuol dire che il suo valore finale deve essere copiato nel parametro attuale della procedura chiamante. Per ricordare la variabile dove copiare il valore si userà la notazione “ x/y ” per dire che il valore assegnato alla variabile x dovrà essere copiato nella variabile z . Si ha quindi:



Output:
 p2: 4 1 -1 -2
 p4: 1 1 4 -2
 p3: 2 1 4 2
 p1: 1 1 2 2

Esercizio

Si consideri il seguente programma:

```
1  program p1
2  int a; int b; int c; int d;
3  procedure p2([IN OUT x copia] int a)
4  int c; int d;
5  BEGIN
6  if b>3 then c=b else c=a+2;
7  d=4;
8  a=c-4;
9  if a>10 then b=c else b=d-3;
10 write(a,b,c,d);
11 END
12
13 procedure p3([MODE] int b, [IN x copia] int c)
14 int d;
15 procedure p4([IN x copia] int d)
16 int a; int c;
17 BEGIN
18 a=b*4;
19 c=3;
20 d=b-4;
21 b=d;
22 p2(b);
23 write(a,b,c,d);
24 END
25
26 BEGIN
27 d=a;
28 b=b+2;
29 c=a*3;
30 a=1;
31 p4(b);
32 write(a,b,c,d);
33 END
34
35 BEGIN
36 a=0;
37 b=3;
38 c=0;
39 d=2;
40 p3(d,c);
41 write(a,b,c,d);
42 END
```



Determinare l'output e mostrare gli stack di attivazione, tranne nei casi di errore, nei quali bisogna invece indicare l'istruzione che causa l'errore, nei seguenti casi:

- Scoping statico, MODE = OUT per riferimento

Scope statico, MODE=OUT per riferimento In questo caso l'assegnamento $b=b+2$ nel blocco P_3 risulta illegale in quanto si sta provando a leggere il valore di una variabile di output.

Esercizio

Si consideri il seguente programma:

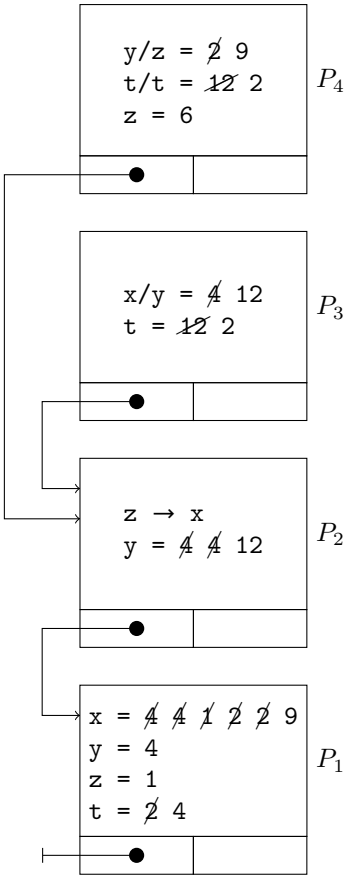
```
1  program p1
2  int x; int y; int z; int t;
3  procedure p2([MODE] int z)
4  int y;
5  procedure p3([IN OUT x copia] int x)
6  int t;
7  BEGIN
8  t=y*3;
9  if y=50 then x=z*1 else x=t;
10 y=y;
11 z=2;
12 p4(z, t);
13 write(x,y,z,t);
14 END
15
16 procedure p4([IN OUT x copia] int y,[IN OUT x copia] int t)
17 int z;
18 BEGIN
19 z=x*3;
20 y=z+3;
21 t=x;
22 if y>2 then x=2 else x=1;
23 write(x,y,z,t);
24 END
25
26 BEGIN
27 y=x*1;
28 z=4;
29 t=x;
30 x=1;
31 p3(y);
32 write(x,y,z,t);
33 END
34
35 BEGIN
36 x=4;
37 y=4;
38 z=1;
39 t=2;
40 p2(x);
41 write(x,y,z,t);
42 END
```



Determinare l'output e mostrare gli stack di attivazione, tranne nei casi di errore, nei quali bisogna invece indicare l'istruzione che causa l'errore, nei seguenti casi:

- Scoping statico, MODE = OUT per riferimento;
- Scoping dinamico, MODE = IN per riferimento;

Scope statico, MODE=OUT per riferimento Nel caso di scope statico l'ambiente non locale è dato dal blocco di definizione sintattica della procedure. In questa tipologia di passaggio dei parametri bisogna stare attenti ad eventuali tentativi di effettuare una lettura dei parametri formali di output; la loro scrittura invece è lecita ed, essendo il passaggio fatto per riferimento, ogni assegnamento viene fatto direttamente sul parametro attuale. Si ha quindi:



Si ha quindi come output:

p4: 9 2 6 2
p3: 12 4 9 2
p2: 9 12 9 4
p1: 9 4 1 4

Scope dinamico, MODE=IN per riferimento Questa tipologia di esercizio prevede che l'ambiente locale sia dato dall'ambiente locale del blocco chiamante. Per quanto riguarda il passaggio dei parametri, bisogna stare attenti affinché non ci siano tentativi di effettuare una scrittura sui parametri di input (la lettura è lecita). Non a caso, nella procedura p_2 notiamo l'istruzione $z=4$ dove z è un parametro di input che contiene l'indirizzo della variabile x presente nella procedura p_1 . Per questo motivo il compilatore restituisce errore, non viene costruito nessuno stack di attivazione e conseguentemente nessun output.

Esercizio

Si consideri il seguente programma:

```
1  program p1
2  int x; int y; int z; int t;
3  procedure p2([IN x copia] int y)
4  int z;
5  BEGIN
6  z=3;
7  y=2;
8  t=y+3;
9  x=x;
10 p3(t);
11 write(x,y,z,t);
12 END
13
14 procedure p3([IN x copia] int z)
15 int x; int t;
16 procedure p4([MODE] int x,[IN OUT x copia] int t)
17 int z;
18 BEGIN
19 z=x;
20 x=z-3;
21 t=4;
22 y=2;
23 write(x,y,z,t);
24 END
25
26 BEGIN
27 x=y;
28 t=z;
29 z=4;
30 y=x;
31 p4(y, z);
32 write(x,y,z,t);
33 END
34
35 BEGIN
36 x=3;
37 y=2;
38 z=4;
39 t=3;
40 p2(y);
41 write(x,y,z,t);
42 END
```

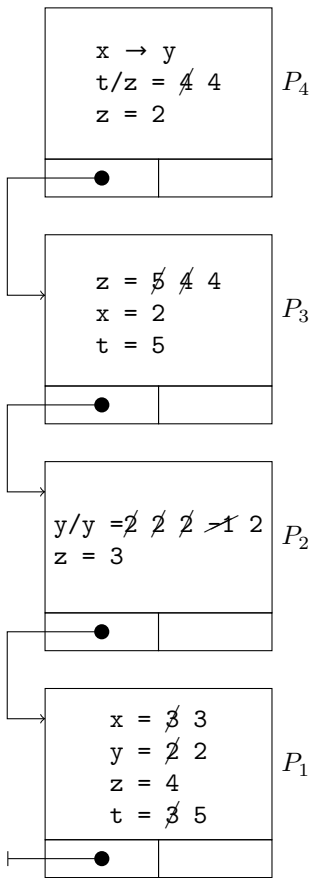


Determinare l'output e mostrare gli stack di attivazione, tranne nei casi di errore, nei quali bisogna invece indicare l'istruzione che causa l'errore, nei seguenti casi:

- Scoping dinamico, MODE = OUT per riferimento
- Scoping statico, MODE = IN OUT per riferimento

Scope dinamico, MODE=OUT per riferimento Si ha errore di compilazione dato dall'istruzione **z=x** nella procedura **p4** in cui si tenta di eseguire una lettura sul parametro di output **x** al quale è stato passato il riferimento della variabile **y** del blocco **p3**.

Scope statico, MODE=IN/OUT per riferimento In questa tipologia di esercizi, il passaggio per riferimento su parametri di IN/OUT non genera errore in alcun caso. Si ottiene quindi il seguente stack di attivazione:



L'output è:

p4: 2 2 2 4
p3: 2 2 4 5
p2: 3 2 3 5
p1: 3 2 4 5

Si consideri il seguente programma:

```
1  program p1
2  int x; int y; int z; int t;
3  procedure p2([IN OUT x copia] int x)
4  int y;
5  procedure p3([MODE] int z)
6  int y; int t;
7  BEGIN
8  y=x+4;
9  if z>20 then t=z+1 else t=3;
10 z=x*4;
11 if t<3 then x=z else x=2;
12 p4(z);
13 write(x,y,z,t);
14 END
15
16 procedure p4([IN OUT x copia] int x)
17 int y; int t;
18 BEGIN
19 y=3;
20 if x=4 then t=x*1 else t=3;
21 x=z*4;
22 z=4;
23 write(x,y,z,t);
24 END
25
26 BEGIN
27 y=3;
28 x=y;
29 t=t;
30 z=y;
31 p3(y);
32 write(x,y,z,t);
33 END
34
35 BEGIN
36 x=4;
37 y=4;
38 z=2;
39 t=2;
40 p2(t);
41 write(x,y,z,t);
42 END
```

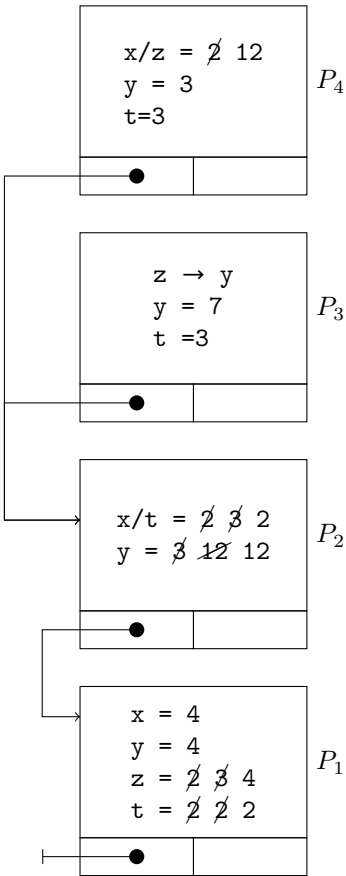


Determinare l'output e mostrare gli stack di attivazione, tranne nei casi di errore, nei quali bisogna invece indicare l'istruzione che causa l'errore, nei seguenti casi:

- Scoping dinamico, MODE = OUT per riferimento
- Scoping statico, MODE = IN OUT per riferimento

Scope dinamico, MODE=OUT per riferimeno In questo caso c'è un errore di compilazione dato da un tentativo di lettura di un parametro di output nella procedura p3: `if z>20 ....`

Scope statico, MODE=IN OUT per riferimento Nel caso dei passaggi dei parametri IN/OUT per riferimento non è possibile che il compilatore rilevi errori. Si ottiene quindi il seguente stack di attivazione:



Che restituisce il seguente output:

```
12 3 4 3
2 7 12 3
2 12 4 2
4 4 4 2
```


Esercizio

Si consideri il seguente programma:

```
1  program p1
2  int q; int r; int s; int t;
3  procedure p2([IN x copia] int q,[MODE] int s)
4  int r;
5  procedure p3([IN OUT x copia] int t)
6  int s; int r;
7  BEGIN
8  s=1;
9  r=3;
10 t=t;
11 q=4;
12 write(q,r,s,t);
13 END
14
15 BEGIN
16 if s=7 then r=3 else r=3;
17 q=t;
18 s=2;
19 t=1;
20 p3(t);
21 write(q,r,s,t);
22 END
23
24 procedure p4([IN OUT x rif] int q)
25 int t;
26 BEGIN
27 t=q+2;
28 q=3;
29 s=r;
30 r=r+1;
31 p2(s, t);
32 write(q,r,s,t);
33 END
34
35 BEGIN
36 q=4;
37 r=1;
38 s=2;
39 t=2;
40 p4(t);
41 write(q,r,s,t);
42 END
```

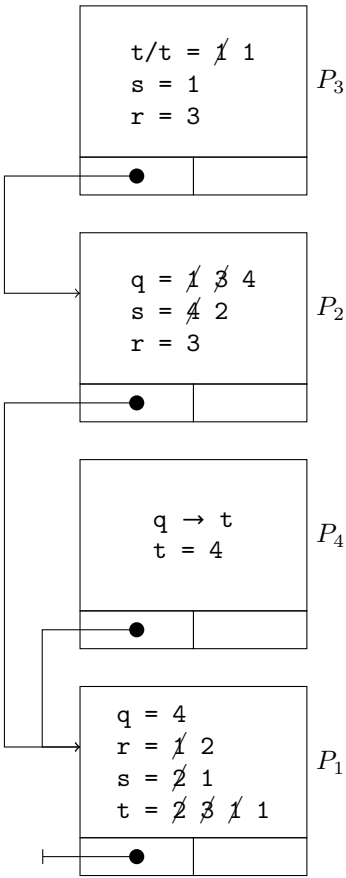


Determinare l'output e mostrare gli stack di attivazione, tranne nei casi di errore, nei quali bisogna invece indicare l'istruzione che causa l'errore, nei seguenti casi:

- Scoping dinamico, MODE = OUT per riferimento
- Scoping statico, MODE = IN per copia

Scope dinamico, MODE = OUT per riferimento Errore di compilazione nel blocco p2 dato dall'istruzione `if s=7...` che tenta di effettuare una lettura su un parametro di output.

Scope statico, MODE = IN per copia In questa tipologia di passaggio di parametro non possono avvenire errori in fase di compilazione. Si procede quindi con la descrizione dello stack di attivazione:



Ottenendo come output:

```
4 3 1 1
4 3 2 1
1 2 1 4
4 2 1 1
```

Esercizio

Si consideri il seguente programma:

```
1  program p1
2  int a; int b; int c; int d;
3  procedure p2([MODE] int c,[IN OUT x rif] int d)
4  int b;
5  procedure p3([IN x copia] int d)
6  int a; int b;
7  BEGIN
8  a=d-4;
9  b=2;
10 if c>1 then d=d*4 else d=1;
11 c=c*4;
12 write(a,b,c,d);
13 END
14
15 BEGIN
16 b=1;
17 c=4;
18 d=b+1;
19 a=1;
20 p3(c);
21 write(a,b,c,d);
22 END
23
24 procedure p4([IN OUT x rif] int a,[IN OUT x rif] int d)
25 int c;
26 BEGIN
27 if a>2 then c=b else c=1;
28 a=d-3;
29 d=a+1;
30 b=b*1;
31 p2(a, d);
32 write(a,b,c,d);
33 END
34
35 BEGIN
36 a=3;
37 b=2;
38 c=0;
39 d=3;
40 p4(a, c);
41 write(a,b,c,d);
42 END
```

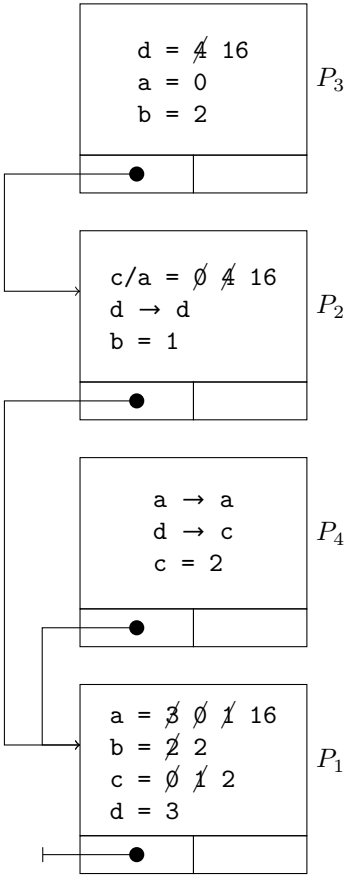


Determinare l'output e mostrare gli stack di attivazione, tranne nei casi di errore, nei quali bisogna invece indicare l'istruzione che causa l'errore, nei seguenti casi:

- Scoping dinamico, MODE = IN per riferimento
- Scoping statico, MODE = OUT per copia

Scope dinamico, MODE=IN per riferimento In questo caso si ha errore di compilazione nel blocco p2 dato dall'istruzione c=4 che tenta di effettuare una lettura su un parametro di input.

Scope statico, MODE = OUT per copia Nel caso del passaggio di parametri in modalità OUT per copia bisogna stare attenti ai tentativi di lettura delle variabili di output. Essendo la modalità di passaggio per copia, all'uscita della procedura, si dovrà aggiornare il valore del parametro attuale con quello del parametro formale. Come nel caso dei parametri IN/OUT per copia possiamo utilizzare la notazione “x/y=...” per evidenziare il fatto che la variabile di output x, all'uscita della procedura, dovrà copiare il suo valore nella variabile y della procedura chiamante. Si ottiene quindi il seguente stack di attivazione:



Si ottiene quindi il seguente output:

```
0 2 16 16
1 1 16 2
16 2 2 2
16 2 2 3
```

Esercizio

Si consideri il seguente programma:

```
1  program p1
2  int a; int b; int c; int d;
3  procedure p2([IN x copia] int b)
4  int c;
5  procedure p3([IN OUT x copia] int a)
6  int c;
7  BEGIN
8  c=d-1;
9  a=1;
10 d=d-3;
11 b=2;
12 p4(a);
13 write(a,b,c,d);
14 END
15
16 procedure p4([MODE] int b)
17 int a;
18 BEGIN
19 a=b;
20 b=2;
21 d=4;
22 c=3;
23 write(a,b,c,d);
24 END
25
26 BEGIN
27 c=d*2;
28 b=d;
29 d=1;
30 a=3;
31 p3(c);
32 write(a,b,c,d);
33 END
34
35 BEGIN
36 a=4;
37 b=3;
38 c=4;
39 d=0;
40 p2(a);
41 write(a,b,c,d);
42 END
```

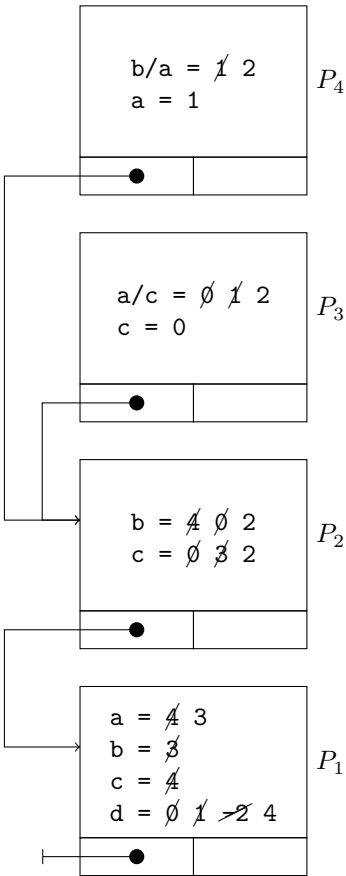


Determinare l'output e mostrare gli stack di attivazione, tranne nei casi di errore, nei quali bisogna invece indicare l'istruzione che causa l'errore, nei seguenti casi:

- Scoping dinamico, MODE = OUT per riferimento
- Scoping statico, MODE = IN OUT per riferimento

Scope dinamico, MODE = OUT per riferimento In questo caso si ha errore di compilazione nel blocco p4 dato dall'istruzione **a=b** che tenta di effettuare una lettura su un parametro di output.

Scope statico, MODE = IN/OUT per riferimento In questa tipologia di passaggio di parametri non possono avvenire errori in fase di compilazione. Si procede quindi con la descrizione dello stack di attivazione:



Si ottiene quindi l'output:

```
1 2 3 4
2 2 0 4
3 2 2 4
3 3 4 4
```

Esercizio

Si consideri il seguente programma:

```
1  program p1
2  int a; int b; int c; int d;
3  procedure p2([MODE] int c)
4  int d; int b;
5  procedure p3([IN OUT x rif] int d,[IN OUT x copia] int b)
6  int c;
7  BEGIN
8  c=d;
9  d=1;
10 b=a+1;
11 a=b;
12 write(a,b,c,d);
13 END
14
15 BEGIN
16 d=3;
17 b=2;
18 c=a;
19 a=b*1;
20 p3(c, d);
21 write(a,b,c,d);
22 END
23
24 procedure p4([IN OUT x rif] int a,[IN OUT x copia] int d)
25 int b;
26 BEGIN
27 b=c*2;
28 a=a*3;
29 d=d+3;
30 c=1;
31 p2(d);
32 write(a,b,c,d);
33 END
34
35 BEGIN
36 a=0;
37 b=2;
38 c=4;
39 d=1;
40 p4(b, a);
41 write(a,b,c,d);
42 END
```

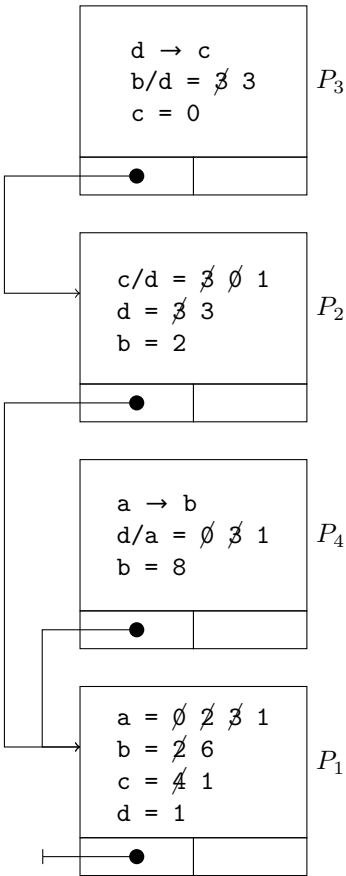


Determinare l'output e mostrare gli stack di attivazione, tranne nei casi di errore, nei quali bisogna invece indicare l'istruzione che causa l'errore, nei seguenti casi:

- Scoping dinamico, MODE = IN per riferimento
- Scoping statico, MODE = OUT per copia

Scope dinamico, MODE = IN per riferimento Errore di compilazione dato dall'istruzione **c=a** nel blocco **p2** che tenta di effettuare una scrittura su un parametro di input.

Scope statico, MODE = OUT per copia Nessun errore in fase di compilazione. Si procede quindi con la descrizione dello stack di attivazione:



Si ottiene quindi come output:

3	3	0	1
3	2	1	3
6	8	1	1
1	6	1	1

Esercizio

Si consideri il seguente programma:

```
1  program p1
2  int q; int r; int s; int t;
3  procedure p2([IN OUT x copia] int q)
4  int s;
5  BEGIN
6  s=1;
7  q=2;
8  t=q+2;
9  if r=9 then r=s+4 else r=r;
10 p3(s, q);
11 write(q,r,s,t);
12 END
13
14 procedure p3([MODE] int t,[IN x copia] int q)
15 int s;
16 procedure p4([IN OUT x copia] int s)
17 int q; int r;
18 BEGIN
19 q=t+3;
20 r=t-2;
21 s=s;
22 t=q-2;
23 write(q,r,s,t);
24 END
25
26 BEGIN
27 s=3;
28 t=s-3;
29 q=2;
30 r=q*1;
31 p4(r);
32 write(q,r,s,t);
33 END
34
35 BEGIN
36 q=1;
37 r=2;
38 s=4;
39 t=1;
40 p2(r);
41 write(q,r,s,t);
42 END
```

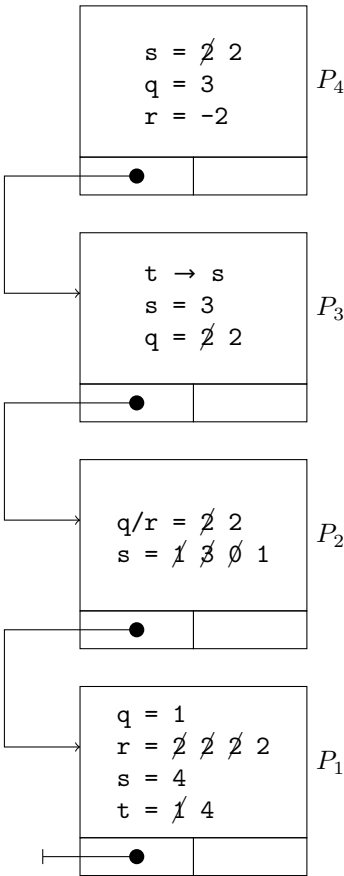


Determinare l'output e mostrare gli stack di attivazione, tranne nei casi di errore, nei quali bisogna invece indicare l'istruzione che causa l'errore, nei seguenti casi:

- Scoping dinamico, MODE = IN per riferimento
- Scoping statico, MODE = OUT per riferimento

Scope dinamico, MODE = IN per riferimento In questo caso si ha errore in fase di compilazione dato dall'istruzione `t=s-3` nel blocco `p3` che tenta di effettuare una scrittura su un parametro di input.

Scope statico, MODE = OUT per riferimento Nessun errore in fase di compilazione. Si procede quindi con la descrizione dello stack di attivazione:



Output:

```
3 -2 2 1
2 2 3 1
2 2 1 4
1 2 4 4
```

Esercizio

Si consideri il seguente programma:

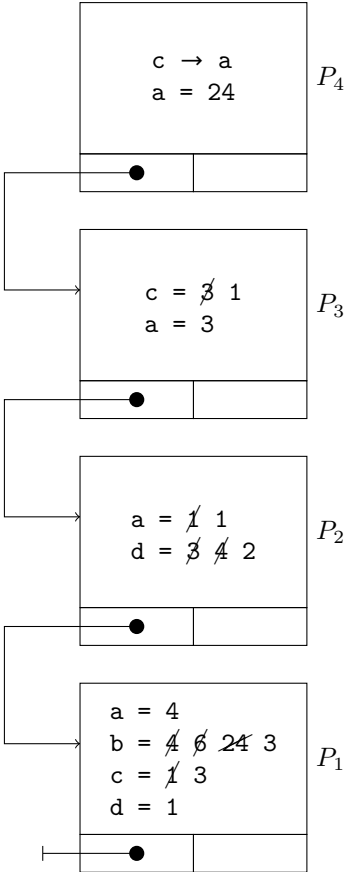
```
1  program p1
2  int a; int b; int c; int d;
3  procedure p2([IN x copia] int a)
4  int d;
5  procedure p3([IN x copia] int c)
6  int a;
7  procedure p4([MODE] int c)
8  int a;
9  BEGIN
10 a=b;
11 c=2;
12 b=c+1;
13 d=c;
14 write(a,b,c,d);
15 END
16
17 BEGIN
18 a=d;
19 c=d-2;
20 d=4;
21 b=b*4;
22 p4(a);
23 write(a,b,c,d);
24 END
25
26 BEGIN
27 d=3;
28 a=1;
29 c=3;
30 b=d*2;
31 p3(d);
32 write(a,b,c,d);
33 END
34
35 BEGIN
36 a=4;
37 b=4;
38 c=1;
39 d=1;
40 p2(c);
41 write(a,b,c,d);
42 END
```



Determinare l'output e mostrare gli stack di attivazione, tranne nei casi di errore, nei quali bisogna invece indicare l'istruzione che causa l'errore, nei seguenti casi:

- Scoping dinamico, MODE = IN OUT per riferimento
- Scoping statico, MODE = IN per riferimento

Scope dinamico, MODE = IN OUT per riferimento Nessun errore di compilazione. Si procede quindi con la descrizione dello stack di attivazione:



Scope statico, MODE = IN per riferimento Si ha errore di compilazione nel blocco p4 dato dall'istruzione $c=2$ che tenta di effettuare una scrittura su un parametro di input.

Tabella 7.1: Errori di compilazione causati dal passaggio dei parametri

Modalità		Errore in compilazione
IN	per copia	Mai
	per riferimento	Se c'è una scrittura sul parametro
OUT	per copia	Se c'è una lettura del valore del parametro
	per riferimento	Se c'è una lettura del valore del parametro
IN/OUT	per copia	Mai
	per riferimento	Mai



Esercizio

In questa successione di enunciati, quale linea non compila?

```
1 byte b = 5;  
2 char c = '5';  
3 short s = 55;  
4 int i = 555;  
5 float f = 555.5f;  
6 b = s;  
7 i = c;  
8 if (f>b)  
9 f = i;
```



Risposta corretta: 6

Soluzione Non è possibile effettuare una conversione implicita quando il tipo della variabile di destinazione è più piccolo della sorgente. In questo caso particolare si sta tentando di salvare il valore di `s` che è di tipo `short` all'interno di `b` di tipo `byte`. Per non incorrere in errori a tempo di compilazione è necessario effettuare un cast esplicito.

Esercizio

Il codice seguente viene compilato correttamente?

```
1 byte b = 2;  
2 byte b1 = 3;  
3 b = b * b1;
```



- A. Si
- B. No

Risposta corretta: B

Soluzione Non è possibile effettuare una conversione implicita quando il tipo della variabile di destinazione è più piccolo della sorgente. In questo caso si sta tentando di salvare il valore dell'espressione `b * b1` che viene automaticamente promosso ad `int` all'interno di una variabile di tipo `byte`. Anche in questo caso sarebbe necessario effettuare un cast esplicito.

Esercizio

Nel codice seguente, quali sono i possibili tipi per la variabile `result`?

```
1 byte b = 11;  
2 short s = 13;  
3 result = b * ++s;
```



- A. byte, short, int, long, float, double
- B. boolean, byte, short, char, int, long, float, double
- C. byte, short, char, int, long, float, double
- D. byte, short, char
- E. int, long, float, double

Risposta corretta: E

Soluzione Quando si effettuano operazioni aritmetiche su tipi più piccoli di `int` (come `byte` e `short`), Java esegue la promozione dei tipi in modo da eseguire l'operazione utilizzando almeno il tipo `int`.

Esercizio

```
1 class Cruncher {
2     void crunch ( int i ) {
3         System.out.println(" int version ");
4     }
5     void crunch ( String s ) {
6         System.out.println(" String version ");
7     }
8
9     public static void main ( String args []) {
10         Cruncher crun = new Cruncher ();
11         char ch = 'p ';
12         crun.crunch ( ch );
13     }
14 }
```



Quali delle seguenti affermazioni è vera?

- A. La linea 5 non compila, poiché i metodi void non possono essere sovrapposti.
- B. La linea 12 non compila, poiché nessuna versione di `crunch()` ha un argomento `char`.
- C. Il codice compila correttamente, ma viene lanciata una eccezione alla linea 12.
- D. Il codice compila correttamente e viene prodotto il seguente output: `int version`
- E. Il codice compila correttamente e viene prodotto il seguente output: `String version`

Risposta corretta: D

Soluzione Poiché non esiste una firma esatta che accetti un parametro di tipo `char`, Java esegue l'autoboxing del `char` a `int` per cercare di soddisfare la firma più vicina, che è il metodo che accetta un parametro di tipo `int`. Di conseguenza, verrà chiamato il metodo `crunch(int i)`, e verrà stampato `"int version"`.

Esercizio

Un tipo di dato con segno ha un ugual numero di valori positivi e negativi.

- A. Vero
- B. Falso

Risposta corretta: B

Soluzione Falso in quanto, nella rappresentazione complemento a due, il numero 0 ha una doppia rappresentazione.

Esercizio

Scegliere gli identificatori legali tra questi:

- A. `StringaLunghissimaSenzaSignificato`
- B. `$int`
- C. `bytes`
- D. `$1`
- E. `finals`

Risposta corretta: tutti

Esercizio

Quali delle seguenti signature sono valide per il metodo `main`?

- A. `public static void main()`
- B. `public static void main(String arg[])`
- C. `public void main(String[] arg)`
- D. `public static void main(String[] args)`
- E. `public static int main(String[] arg)`

Risposta corretta: B,D

Esercizio

Se in un file sorgente sono presenti tutti e tre gli elementi top-level, in quale ordine devono apparire?

- A. import, package, class
- B. class, import, package
- C. package per primo, l'ordine degli altri non importa
- D. package, import, class
- E. import per primo, l'ordine degli altri non importa

Risposta corretta: D

Esercizio

Si consideri la seguente linea di codice:

```
1 int [] x = new int [25];
```



Dopo l'esecuzione, quali delle seguenti affermazioni sono vere?

- A. x[0] è 0
- B. x è indefinito
- C. x è 0
- D. x[0] è null
- E. x.length è 25

Risposta corretta: A,E

Esercizio

Qual è l'output della seguente applicazione?

```
1 class D6 {  
2     public static void main ( String args []) {  
3         Scatola s = new Scatola ();  
4         s.interno = 100;  
5         s.aumenta(s);  
6         System.out.println ( s.interno );  
7     }  
8 }  
9 class Scatola {  
10     public int interno ;  
11     public void aumenta ( Scatola scatola ) {  
12         scatola.interno ++;  
13     }  
14 }
```



- A. 0
- B. 1
- C. 100
- D. 101

Risposta corretta: D

Esercizio

Qual è l'output della seguente applicazione?

```
1 class D7 {  
2     public static void main ( String args []) {  
3         double d = 12.3;  
4         Decremento dec = new Decremento ();  
5         dec.decrementa( d );  
6         System.out.println( d );  
7     }  
8 }  
9 class Decremento {  
10     public void decrementa ( double dec ) {  
11         dec = dec - 1.0;  
12     }  
13 }
```

- A. 0.0
- B. -1.0
- C. 12.3
- D. 11.3

Risposta corretta: C

Esercizio

Come si può forzare la garbage collection di un oggetto?

- A. Il garbage collector non può essere forzato.
- B. Con una chiamata a `System.gc()`.
- C. Con una chiamata a `System.gc()`, passando il riferimento all'oggetto.
- D. Con una chiamata a `Runtime.gc()`.
- E. Ponendo tutti i riferimenti a quell'oggetto a `null`.

Risposta corretta: A

Soluzione Il Garbage Collector in Java libera la memoria automaticamente eliminando gli oggetti non referenziati. Attiva la pulizia della memoria in situazioni come l'allocazione di nuovi oggetti o quando il sistema operativo richiede risorse. Non può essere forzato in modo deterministico, poiché agisce in modo asincrono per evitare problemi di sincronizzazione e per ottimizzare le prestazioni. L'utilizzo di `System.gc()` può suggerire al sistema di eseguire la raccolta dei rifiuti, ma senza garanzia di esecuzione immediata.

Esercizio

Qual è il range di valori per una variabile di tipo `short`?

- A. Dipende dall'hardware che ospita la JVM
- B. $0 \dots 2^{16} - 1$
- C. $0 \dots 2^{32} - 1$
- D. $-2^{15} \dots 2^{15} - 1$
- E. $-2^{31} \dots 2^{31} - 1$

Risposta corretta: D

Esercizio

Qual è il range di valori per una variabile di tipo `byte`?

- A. Dipende dall'hardware che ospita la JVM
- B. $0 \dots 2^8 - 1$
- C. $0 \dots 2^{16} - 1$
- D. $-2^7 \dots 2^7 - 1$
- E. $-2^{15} \dots 2^{15} - 1$

Risposta corretta: D

Esercizio

Quali sono i valori di x, a, b dopo l'esecuzione del codice:

```
1 int x , a = 6 , b = 7;  
2 x = (a++) + (b++);
```

- A. x=15, a=7, b=8
- B. x=15, a=6, b=7
- C. x=13, a=7, b=8
- D. x=13, a=6, b=7

Risposta corretta: C

Esercizio

Quali delle seguenti espressioni sono legali?

- A. `int x=6; x=!x;`
- B. `int x=6; if (!(x>3)) {}`
- C. `int x=6; x= ~x;`

Risposta corretta: B,C

Esercizio

Quali delle seguenti espressioni risultano in un valore positivo in x?

- A. `int x=-1; x = x >>> 5;`
- B. `int x=-1; x = x >>> 32;`
- C. `byte x=-1; x = x >>> 5;`
- D. `int x=-1; x = x >> 5;`

Risposta corretta: A

Esercizio

Quali delle seguenti espressioni sono legali?

- A. `String x="Ciao"; int y=7; x += y;`
- B. `String x="Ciao"; int y=7; if (x == y) {}`
- C. `String x="Ciao"; int y=7; x = x + y;`
- D. `String x="Ciao"; int y=7; y = y + x;`
- E. `String x=null;int y = (x!=null) && (x.length()>0) ? x.length() : 0;`

Risposta corretta: A,C,E

Esercizio

Qual è il risultato dell'esecuzione del seguente codice?

```
1 public class Xor {  
2     public static void main ( String args []) {  
3         byte b = 10; // 00001010 binario  
4         byte c = 15; // 00001111 binario  
5         b = ( byte )( b^c );  
6         System.out.println ( " b vale " + b );  
7     }  
8 }
```

- A. b vale 10
- B. b vale 5
- C. b vale 250
- D. b vale 245

Risposta corretta: B

Esercizio

Qual è il risultato della compilazione ed esecuzione del seguente codice?

```
1 public class Condizionale {  
2     public static void main ( String args []) {  
3         int x = 4;  
4         System.out.println ( " Il valore e ' " + (( x > 4) ? 99.99 : 9));  
5     }  
6 }
```



- A. Il valore è 99.99
- B. Il valore è 9
- C. Il valore è 9.0
- D. Un errore di compilazione alla linea 5.

Risposta corretta: C

Esercizio

Qual è l'output del seguente frammento di codice?

```
1 int x =3; int y = -10;  
2 System . out . println ( y % x );
```



- A. 0
- B. 1
- C. -1
- D. -3

Risposta corretta: C

Esercizio

Qual è l'output del seguente frammento di codice?

```
1 int x =1;  
2 String [] nomi = {"Mario","Anna","Carlo"};  
3 nomi[--x] += ".";  
4 for( int i = 0; i < nomi.length; i++) {  
5     System.out.println(nomi[i]);  
6 }
```



- A. L'output include Mario. con un punto finale.
- B. L'output include Anna. con un punto finale.
- C. L'output include Carlo. con un punto finale.
- D. Nessun nome stampato ha il punto finale.
- E. Viene lanciata l'eccezione `ArrayIndexOutOfBoundsException`.

Risposta corretta: A

Esercizio

Quali linee fanno parte dell'output del seguente codice?

```
1  for ( int i =0; i <2; i ++ ) {  
2      for ( int j =0; j <3; j ++ ) {  
3          if ( i = j ) {  
4              continue ;  
5          }  
6          System . out . println ( " i =" + i + " j =" + j );  
7      }  
8  }
```



- A. i=0 j=0
- B. i=0 j=1
- C. i=0 j=2
- D. i=1 j=0
- E. i=1 j=1
- F. i=1 j=2

Risposta corretta: B,C,D,F

Esercizio

Quali tra queste sono costruzioni legali di loop?

A.

```
1  while ( int i <7 ) {  
2      i ++;  
3      System.out.println(" i =" +i);  
4  }
```



B.

```
1  int i =3;  
2  while (i) {  
3      System.out.println("i="+i);  
4  }
```



C.

```
1  int j =0;  
2  for ( int k = 0; j+k ≠ 10;j++,k++) {  
3      System.out.println ( " j =" + j + " k =" + k );  
4  }
```



D.

```
1  int j =0;  
2  do {  
3      System.out.println(" j =" + j ++);  
4      if ( j = 3 ) { continue loop ;}  
5  } while ( j < 10);
```



Risposta corretta: C

Soluzione La A non è legale perché la variabile **i** non è inizializzata, infatti il **while** non permette una dichiarazione interna, ed una dichiarazione non permette una condizione booleana; B non legale perché Java richiede un **boolean** come condizione; D non è legale poiché l'etichetta **loop** non è stata definita.

Esercizio

Qual è l'output di questo frammento di codice?

```
1  int x =0 , y =4 , z =5;
2  if ( x >2 ) {
3      if ( y <5 ) {
4          System.out.println ( " uno " );
5      } else {
6          System.out.println ( " due " );
7      }
8  } else if ( z >5 ) {
9      System.out.println ( " tre " );
10 } else {
11     System.out.println( " quattro " );
12 }
```



- A. uno
- B. due
- C. tre
- D. quattro

Risposta corretta: D

Esercizio

Dato il codice:

```
1  int j =2;
2  switch ( j ) {
3      case 2:
4          System.out.print("2");
5      case 2+1:
6          System.out.print("3");
7          break;
8      default :
9          System.out.print(j);
10         break;
11     }
12     System.out.println();
```



Quale dei seguenti enunciati è vero?

- A. Il codice è illegale a causa dell'espressione alla linea 5.
- B. I tipi accettabili per la variabile di controllo di uno switch sono `byte`, `short`, `int`, `long`.
- C. L'output è 2.
- D. L'output è 23.
- E. L'output è 232.

Risposta corretta: D

Soluzione Notare il fatto che manca la clausola `break` nel primo ramo dello `switch`. Il flusso di controllo, dopo aver eseguito il primo ramo andrà in quello successivo dove trova il `break`.

Esercizio

Quali delle seguenti dichiarazioni sono illegali?

- A. `default String s;`
- B. `transient int i = 41;`
- C. `public final static native int w();`
- D. `abstract double d;`
- E. `abstract final double cosenoIperbolico();`

Risposta corretta: B,C

Esercizio

Quali delle seguenti affermazioni è vera?

- A. Una classe **abstract** non può avere metodi **final**.
- B. Una classe **final** non può avere metodi **abstract**.

Risposta corretta: A

Esercizio

Qual è la minima modifica che rende il seguente codice compilabile?

```
1 final class Aaa {  
2     int xxx;  
3     void yyy() { xxx =1; }  
4 }  
5  
6 class Bbb extends Aaa {  
7     final Aaa fref = new Aaa ();  
8     final void yyy () {  
9         System.out.println ("In yyy()");  
10        fref.xxx = 12345;  
11    }  
12 }
```



- A. Alla linea 1, rimuovere **final**.
- B. Alla linea 7, rimuovere **final**.
- C. Rimuovere la linea 11
- D. Alle linee 1 e 7, rimuovere **final**
- E. Nessuna modifica è necessaria

Risposta corretta: A

Esercizio

Riguardo al codice seguente, quale affermazione è vera?

```
1 class Roba {  
2     static int x = 10;  
3     static { x += 5; }  
4     public static void main ( String [] args ) {  
5         System.out.println(" x =" + x );  
6     }  
7     static { x /= 5; }  
8 }
```



- A. Le linee 3 e 9 non sono compilate, poiché mancano i nomi di metodi e i tipi di ritorno.
- B. La linea 9 non è compilata, poiché si può avere solo un blocco top-level static.
- C. Il codice viene compilato e l'esecuzione produce **x=10**.
- D. Il codice viene compilato e l'esecuzione produce **x=15**.
- E. Il codice viene compilato e l'esecuzione produce **x=3**.

Risposta corretta: E

Esercizio

Rispetto al codice seguente, quale affermazione è vera?

```
1  class A {  
2      private static int x =100;  
3      public static void main (String [] args ) {  
4          A hs1 = new A ();  
5          hs1.x ++;  
6          A hs2 = new A ();  
7          hs2.x ++;  
8          hs1 = new A ();  
9          hs1.x ++;  
10         A.x ++;  
11         System.out.println ( " x = " + x );  
12     }  
13 }
```



- A. La linea 7 non compila, poiché è un riferimento **static** ad una variabile **private**.
- B. La linea 12 non compila, poiché è un riferimento **static** ad una variabile **private**.
- C. Il programma viene compilato e stampa $x = 102$.
- D. Il programma viene compilato e stampa $x = 103$.
- E. Il programma viene compilato e stampa $x = 104$.

Risposta corretta: E

Esercizio

Dato il codice seguente:

```
1  class SuperC {  
2      void unMetodo () { }  
3  }  
4  
5  class SubC extends SuperC {  
6      void unMetodo () { }  
7  }
```



- A. Quali modificatori di accesso possono essere legalmente dati ad **unMetodo** alla linea 2, lasciando il resto del codice inalterato?
- B. Quali modificatori di accesso possono essere legalmente dati ad **unMetodo** alla linea 6, lasciando il resto del codice inalterato?

Soluzione

- A. Alla riga 2, il metodo **unMetodo** nella classe **SuperC** può avere qualsiasi modificatore di accesso, poiché è un metodo di una classe base.
- B. Alla riga 6, il metodo **unMetodo** nella classe **SubC** può avere lo stesso modificatore di accesso o un modificatore meno restrittivo rispetto al metodo corrispondente nella classe base **SuperC**. Non può avere un modificatore più restrittivo (come **private**), poiché questo violerebbe la regola dell'override dei metodi che richiede che il metodo derivato abbia un accesso almeno uguale a quello del metodo base.

Esercizio

Quale dei seguenti enunciati è corretto?

- A. Solo i tipi primitivi sono convertiti automaticamente; per cambiare il tipo di un riferimento bisogna fare un cast.
- B. Solo i riferimenti sono convertiti automaticamente; per cambiare il tipo di un primitivo bisogna fare un cast.
- C. La promozione aritmetica di riferimenti richiede il cast esplicito.
- D. Sia i tipi primitivi, sia i riferimenti possono essere convertiti sia automaticamente sia attraverso un cast
- E. Il cast dei tipi numerici richiede un controllo in esecuzione.

Risposta corretta: D

Esercizio

Quale dei seguenti enunciati è vero?

- A. I riferimenti ad oggetti possono essere convertiti nelle assegnazioni e non nelle invocazioni di metodi.
- B. I riferimenti ad oggetti possono essere convertiti nelle invocazioni di metodi e non nelle assegnazioni.
- C. I riferimenti ad oggetti possono essere convertiti sia nelle assegnazioni, sia nelle invocazioni di metodi, ma le regole nei due casi sono diverse.
- D. I riferimenti ad oggetti possono essere convertiti sia nelle assegnazioni, sia nelle invocazioni di metodi, e le regole nei due casi sono identiche.
- E. I riferimenti ad oggetti non possono essere mai convertiti.

Risposta corretta: C

Soluzione Nel caso delle assegnazioni, la conversione avviene attraverso il concetto di “upcasting” (casting verso l’alto) senza la necessità di un’espressione esplicita. Nel caso delle invocazioni di metodi, la conversione richiede un casting esplicito se si sta chiamando un metodo che è specifico della classe del riferimento originale, ma non è definito nella classe del riferimento al quale si sta convertendo.

Esercizio

Quale linea del codice seguente non compila?

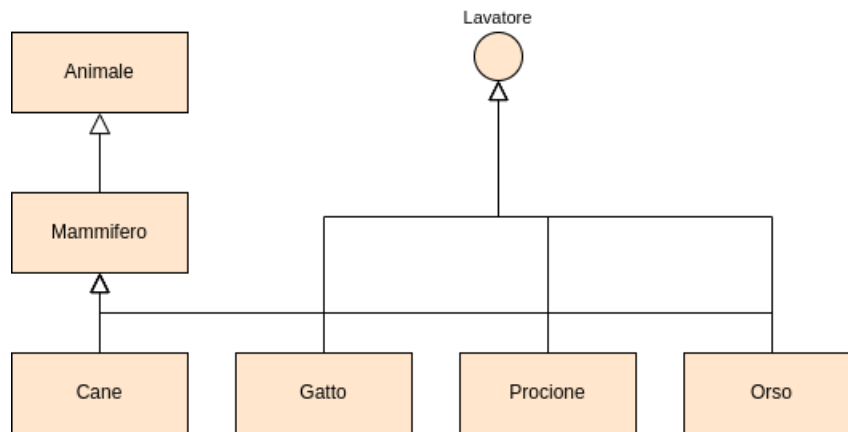
```
1 Object ob = new Object ();
2 String stringarr [] = new String [50];
3 Float floater = new Float (3.14 f );
4 ob = stringarr ;
5 ob = stringarr [5];
6 floater = ob ;
7 ob = floater ;
```



- A. La linea 5.
- B. La linea 6.
- C. La linea 7.
- D. La linea 8.

Risposta corretta: B

Soluzione Questa assegnazione genererà un errore di compilazione in Java perché si sta cercando di assegnare un riferimento a un oggetto di tipo `Object` (il tipo di `ob`) a una variabile di tipo `Float` (il tipo di `floater`) senza un casting esplicito. In Java, è necessario effettuare un casting esplicito quando si assegna un riferimento a un oggetto a una variabile di un tipo più specifico.



Si consideri il codice seguente:

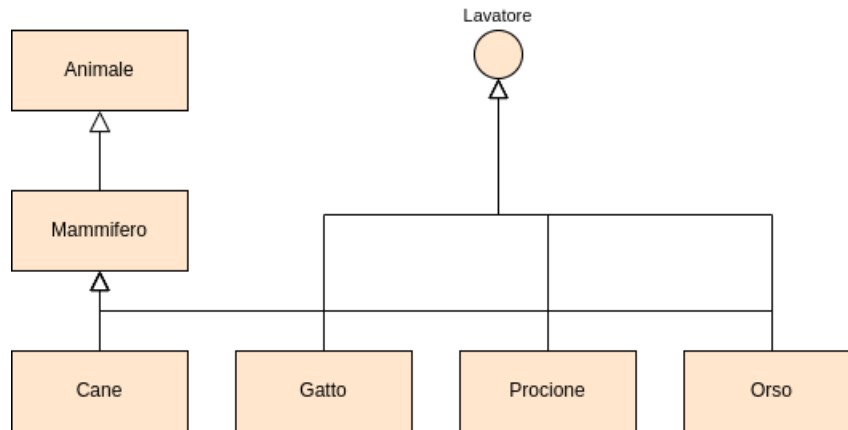
```
1 Cane billy, fido;
2 Animale anim;
3
4 billy = new Cane;
5 anim = billy;
6 fido = (Cane) anim;
```



Quale dei seguenti enunciati è vero?

- A. La linea 5 non compila.
- B. La linea 6 non compila.
- C. Il codice è compilato correttamente ma viene lanciata una eccezione in esecuzione.
- D. Il codice è compilato ed eseguito correttamente.
- E. Il codice è compilato ed eseguito correttamente, ma il cast alla linea 6 è superfluo e può essere eliminato.

Risposta corretta: D



Si consideri il codice seguente:

```
1 Gatto fufi;
2 Lavatore lala;
3 Orso bubu;
4
5 fufi = new Gatto();
6 lala = fufi;
7 bubu = (Orso) lala;
```

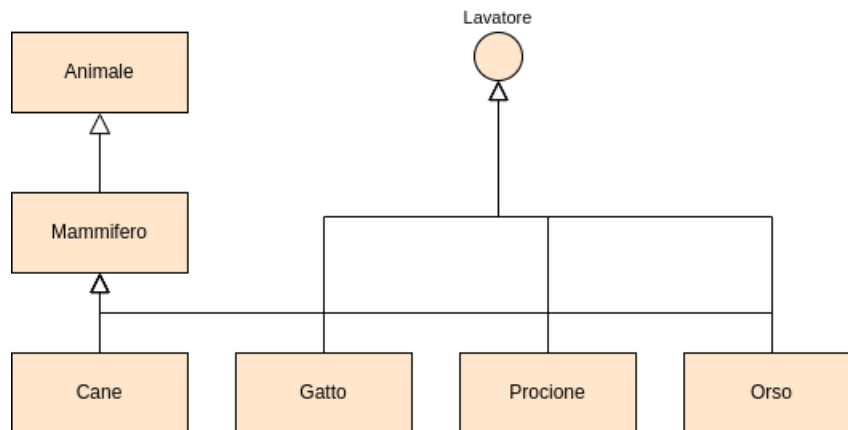


Quale dei seguenti enunciati è vero?

- A. La linea 6 non compila poiché è richiesto un cast esplicito per convertire un Gatto ad un Lavatore.
- B. La linea 7 non compila poiché non si può fare il cast da una interfaccia ad una classe.
- C. Il codice è compilato ed eseguito correttamente ma il cast alla linea 7 non è necessario.
- D. Il codice è compilato ma, alla linea 7, lancia una eccezione in esecuzione poiché la conversione a runtime di una interfaccia in una classe è proibita.
- E. Il codice è compilato ma, alla linea 7, lancia una eccezione poiché il tipo di lala in esecuzione non può essere convertito al tipo Orso.

Risposta corretta: C

Esercizio



Si consideri il codice seguente:

```
1 Procione rocco;
2 Orso bubu;
3 Lavatore lala;
4
5 rocco = new Procione();
6 lala = rocco;
7 bubu = lala;
```



Quale dei seguenti enunciati è vero?

- A. La linea 6 non compila: è richiesto un cast esplicito.
- B. La linea 7 non compila: è richiesto un cast esplicito.
- C. Il codice è compilato ed eseguito correttamente.
- D. Il codice è compilato ma viene lanciata una eccezione alla linea 7, poiché in esecuzione la conversione da una interfaccia ad una classe è proibita.
- E. Il codice è compilato ma viene lanciata una eccezione alla linea 7, poiché la classe dell'oggetto `lala` non può essere convertita al tipo `Orso`.

Risposta corretta: B

Esercizio

Data la stringa costruita mediante `s = new String("xyzyz")`, quali delle seguenti invocazioni di metodi modificherà la stringa?

- A. `s.append("aaa");`
- B. `s.trim();`
- C. `s.substring(3);`
- D. `s.replace('z', 'a');`
- E. `s.concat(s);`
- F. Nessuna delle precedenti

Risposta corretta: A,D,E

Esercizio

Qual è l'output del seguente brano di codice?

```
1 String s1 = " abc " + " def ";
2 String s2 = new String ( s1 );
3 if ( s1 == s2 )
4 System.out.print("A");
5 if ( s1.equals ( s2 ))
6 System.out.print("B");
```



- A. AB
- B. A
- C. B
- D. Nessun output

Risposta corretta: C

Esercizio

Quanti oggetti sono prodotti nel seguente frammento di codice?

```
1 StringBuffer sbuf = new StringBuffer ( " abcde ");
2 sbuf.insert (3,"xyz");
```



- A. 1
- B. 2
- C. 3
- D. 4
- E. 5

Risposta corretta: A

Esercizio

Dato il seguente codice;

```
1 try {
2     // codice rischioso ; hp :
3     // Exception
4     // + - - - EccA
5     //     + - - - EccB
6     //     + - - - EccC
7     System.out.print (1);
8 }
9 catch ( EccB e ) {
10     System.out.println (2);
11 }
12 catch ( EccC e ) {
13     System.out.println (3);
14 }
15 catch ( Exception e ) {
16     System.out.println (4);
17 }
18 finally {
19     System.out.println (5);
20 }
21 System.out.println (6);
```



Quali righe saranno presenti nell'output, nel caso in cui, alla riga 2 venga lanciata una eccezione di tipo EccB?

- A. 1
- B. 2
- C. 3
- D. 4
- E. 5
- F. 6

Risposta corretta: B,E,F

Esercizio

Dato il seguente codice:

```
1 try {
2     // codice rischioso ; hp :
3     // Exception
4     // + - - - EccA
5     // + - - - EccB
6     // + - - - EccC
7     System.out.print (1);
8 }
9 catch ( EccB e ) {
10    System.out.println (2);
11 }
12 catch ( EccC e ) {
13    System.out.println (3);
14 }
15 catch ( Exception e ) {
16    System.out.println (4);
17 }
18 finally {
19    System.out.println (5);
20 }
21 System.out.println (6);
```



Quali righe saranno presenti nell'output, nel caso in cui, alla riga 2 non venga lanciata alcuna eccezione?

- A. 1
- B. 2
- C. 3
- D. 4
- E. 5
- F. 6

Risposta corretta: A,E,F

Esercizio

Dato il seguente codice:

```
1 try {
2     // codice rischioso ; hp :
3     // Exception
4     // + - - - EccA
5     //     + - - - EccB
6     //     + - - - EccC
7     System.out.print (1);
8 }
9 catch ( EccB e ) {
10    System.out.println (2);
11 }
12 catch ( EccC e ) {
13    System.out.println (3);
14 }
15 catch ( Exception e ) {
16    System.out.println (4);
17 }
18 finally {
19    System.out.println (5);
20 }
21 System.out.println (6);
```



Quali righe saranno presenti nell'output, nel caso in cui, alla riga 2 venga lanciata una eccezione di tipo EccC?

- A. 1
- B. 2
- C. 3
- D. 4
- E. 5
- F. 6

Risposta corretta: C,E,F

Esercizio

Dato il seguente codice:

```
1 try {
2     // codice rischioso ; hp :
3     // Exception
4     // + - - - EccA
5     // + - - - EccB
6     // + - - - EccC
7     System.out.print (1);
8 }
9 catch ( EccB e ) {
10    System.out.println (2);
11 }
12 catch ( EccC e ) {
13    System.out.println (3);
14 }
15 catch ( Exception e ) {
16    System.out.println (4);
17 }
18 finally {
19    System.out.println (5);
20 }
21 System.out.println (6);
```



Quali righe saranno presenti nell'output, nel caso in cui, alla riga 2 venga lanciata una eccezione di tipo EccA?

- A. 1
- B. 2
- C. 3
- D. 4
- E. 5
- F. 6

Risposta corretta: D,E,F

Esercizio

Dato il seguente codice:

```
1 try {
2     // codice rischioso ; hp :
3     // Exception
4     // + - - - EccA
5     //     + - - - EccB
6     //     + - - - EccC
7     System.out.print (1);
8 }
9 catch ( EccB e ) {
10    System.out.println (2);
11 }
12 catch ( EccC e ) {
13    System.out.println (3);
14 }
15 catch ( Exception e ) {
16    System.out.println (4);
17 }
18 finally {
19    System.out.println (5);
20 }
21 System.out.println (6);
```



Quali righe saranno presenti nell'output, nel caso in cui, alla riga 2 venga lanciata una eccezione di tipo **Exception**?

- A. 1
- B. 2
- C. 3
- D. 4
- E. 5
- F. 6

Risposta corretta: D,E,F

Esercizio

Dato il seguente codice:

```
1 try {
2     // codice rischioso ; hp :
3     // Exception
4     // + - - - EccA
5     //     + - - - EccB
6     //     + - - - EccC
7     System.out.print (1);
8 }
9 catch ( EccB e ) {
10    System.out.println (2);
11 }
12 catch ( EccC e ) {
13    System.out.println (3);
14 }
15 catch ( Exception e ) {
16    System.out.println (4);
17 }
18 finally {
19    System.out.println (5);
20 }
21 System.out.println (6);
```



Quali righe saranno presenti nell'output, nel caso in cui, alla riga 2 venga lanciata una eccezione di tipo `RuntimeException`?

- A. 1
- B. 2
- C. 3
- D. 4
- E. 5
- F. 6

Risposta corretta: D,E,F

Esercizio

Dato il seguente codice:

```
1 try {
2     // codice rischioso ; hp :
3     // Exception
4     // + - - - EccA
5     // + - - - EccB
6     // + - - - EccC
7     System.out.print (1);
8 }
9 catch ( EccB e ) {
10    System.out.println (2);
11 }
12 catch ( EccC e ) {
13    System.out.println (3);
14 }
15 catch ( Exception e ) {
16    System.out.println (4);
17 }
18 finally {
19    System.out.println (5);
20 }
21 System.out.println (6);
```



Quali righe saranno presenti nell'output, nel caso in cui, alla riga 2 venga lanciata una eccezione di tipo **Error**?

- A. 1
- B. 2
- C. 3
- D. 4
- E. 5
- F. 6

Risposta corretta: E,F

Esercizio

```
1 public class M {
2     public static void main ( String [] args ) {
3         int k =0;
4         try {
5             int i =5/ k ;
6         }
7         catch ( ArithmeticException e){
8             System.out.print (1);
9         }
10        catch ( RuntimeException e ){
11            System.out.print(2);
12            return;
13        }
14        catch ( Exception e ) {
15            System.out.print (3);
16        }
17        finally {
18            System.out.print (4);
19        }
20        System.out.print (5);
21    }
22 }
```



Qual è l'output del programma precedente?

- A. 5
- B. 14
- C. 124
- D. 145
- E. 1245
- F. 35

Risposta corretta: D

Esercizio

```
1 public class Eccezioni {
2     public static void main ( String [] args ) {
3         try {
4             if (args.length == 0) return ;
5             System.out.println (args[0]);
6         }
7         finally {
8             System.out.println ("Fine");
9         }
10    }
11 }
```



Quali affermazioni riguardanti il precedente programma sono vere?

- A. Se eseguito senza argomenti, il programma non produce output.
- B. Se eseguito senza argomenti, il programma stampa Fine.
- C. Il programma lancia un `ArrayIndexOutOfBoundsException`.
- D. Se eseguito con un argomento, il programma stampa solo l'argomento dato.
- E. Se eseguito con un argomento, il programma stampa l'argomento dato seguito da Fine.

Risposta corretta: B,E

Esercizio

Qual è l'output del seguente programma?

```
1 public class MyClass
2 public static void main (String [] args){
3     RuntimeException re = null;
4     throw re;
5 }
```



- A. Il codice non viene compilato, poiché il `main` non dichiara che lancia una `RuntimeException`.
- B. Il codice non viene compilato, poiché non può rilanciare `re`.
- C. Il programma viene compilato e lancia `java.lang.RuntimeException` in esecuzione.
- D. Il programma viene compilato e lancia `java.lang.NullPointerException` in esecuzione.
- E. Il programma viene compilato, eseguito e termina senza produrre alcun output.

Risposta corretta: D

Esercizio

Quali di queste affermazioni sono vere?

- A. Se una eccezione non è catturata in un metodo, il metodo termina e viene ripresa la successiva normale esecuzione.
- B. Un metodo sovrapposto in una sottoclasse deve dichiarare che lancia lo stesso tipo di eccezione del metodo che sovrappone.
- C. Il `main` può dichiarare che lancia eccezioni checked.
- D. Un metodo che dichiara di lanciare un certo tipo di eccezione, può lanciare una istanza di una qualunque sottoclasse di quel tipo.
- E. Il blocco `finally` è eseguito se e solo se viene lanciata una eccezione all'interno del corrispondente blocco `try`.

Risposta corretta: A

Esercizio

Quali, tra i seguenti enunciati, sono veri?

- A. Una classe interna può essere marcata `private`.
- B. Una classe interna può essere marcata `static`.
- C. Una classe interna definita in un metodo deve sempre essere anonima.
- D. Una classe interna definita in un metodo può accedere tutte le variabili locali di quel metodo.
- E. La costruzione di una classe interna può richiedere una istanza della classe esterna.

Risposta corretta: A,B,D,E

Esercizio

Si consideri la seguente definizione:

```
1 public class Outer {
2     public int a = 1;
3     private int b = 2;
4     public void method ( final int c ) {
5         int d = 3;
6         class Inner {
7             private void iMethod ( int e ) {
8
9             }
10        }
11    }
12 }
```



Quali variabili possono essere correttamente utilizzate alla linea 8?

- A. a
- B. b
- C. c
- D. d
- E. e

Risposta corretta: A

Esercizio

Data la seguente classe:

```
1 public class C{  
2     public void f (int i){}  
3  
4 }
```



Quale dei seguenti metodi può essere legalmente aggiunto alla linea 3?

- A. `private void f (float i)`
- B. `public int f (int j)`
- C. `private void f (int j)`
- D. `public static void f (int j)`
- E. Nessuna delle precedenti

Risposta corretta: A

Soluzione La risposta corretta è la 1 in quanto il dynamic method dispatch prende in considerazione il tipo e il numero di argomenti di un metodo per risolvere il sovraccaricamento. I metodi elencati ai numeri 2, 3 e 4 hanno però lo stesso numero e lo stesso tipo del metodo sovrascritto rendendo così impossibile risolvere il sovraccaricamento. Bisogna ricordare inoltre che il tipo di ritorno di un metodo non rappresenta un fattore determinante nella risoluzione del sovraccaricamento.

Esercizio

Qual è il risultato della compilazione ed esecuzione del seguente programma?

```
1 public class Vehicle {  
2     String producer, model;  
3     Vehicle(String p, String m) {  
4         producer = p;  
5         model = m;  
6     }  
7  
8     public String id() {  
9         return producer + " ";  
10    }  
11 }  
12  
13 class Car extends Vehicle {  
14     Car(){}  
15     Car(String p, String m) {  
16         super(p,m);  
17     }  
18  
19     public String id() {  
20         return producer + " " + model + " ";  
21     }  
22 }  
23  
24 class Main {  
25     public static void main(String[] args) {  
26         Vehicle v = new Vehicle("Fiat","500");  
27         Vehicle c = new Car("Opel","Corsa");  
28         Car d = new Car("Citroen","C3");  
29         System.out.print(v.id());  
30         System.out.print(c.id());  
31         System.out.print(d.id());  
32     }  
33 }
```



- A. Fiat Opel Citroen C3
- B. Fiat Opel Corsa Citroen C3
- C. Citroen Citroen Citroen C3
- D. Errore a tempo di compilazione.
- E. Errore a tempo di esecuzione.

Risposta corretta: D

Soluzione Il programma restituisce un errore a tempo di compilazione dato dalla presenza del costruttore vuoto nella classe **Car**. Non essendo presente alcun costruttore vuoto nella classe **Vehicle** non è possibile richiamarlo all'interno del costruttore di **Car**. Nella sezione 4.10 viene spiegato il meccanismo che porta alla costruzione di un oggetto.

Esercizio

Dato il seguente codice:

```
1 public interface I {  
2     int n = 0;  
3 }
```



Quale delle seguenti linee può essere equivalentemente sostituita alla linea 2?

- A. `final int n = 0;`
- B. `abstract int n = 0;`
- C. `protected int n = 0;`
- D. `public int n;`
- E. Nessuna delle precedenti

Risposta corretta: A

Soluzione Attraverso la dichiarazione di una interfaccia si definisce uno “standard” che qualsiasi classe che implementi tale interfaccia dovrà seguire. In Java, tutti i membri di un’interfaccia sono implicitamente pubblici, statici e finali. Ciò significa che la costante **n** è implicitamente pubblica, quindi può essere accessibile da qualsiasi classe che implementa questa interfaccia. È statica, il che significa che può essere accessibile senza creare un’istanza dell’interfaccia, e finale, il che significa che il suo valore non può essere modificato dopo l’inizializzazione.

Esercizi

Dire quale delle seguenti affermazioni è vera:

- A. Una classe interna non può essere dichiarata **private**.
- B. Una classe interna non può essere dichiarata **final**.
- C. Una classe interna non può essere dichiarata **static**.
- D. Una istanza di una classe interna può essere creata solo all’interno della classe includente.
- E. Nessuna delle precedenti

Risposta corretta: E

Esercizio

Quanti oggetti *al massimo* possono essere deallocati al punto indicato?

```
1 class Star {  
2     String name = "";  
3     double massIdx = 0;  
4 }  
5 class Sun extends Star {  
6     String name = "Sun";  
7     double massIdx = 1.98910;  
8 }  
9 public static void main(String[] args){  
10     Star s1 = new Star();  
11     Sun s2 = new Sun();  
12     s1 = s2;  
13     s2 = null; /*QUI*/  
14 }
```



- A. 1
- B. 2
- C. 3
- D. 4
- E. Nessuna delle precedenti

Risposta corretta: A

Esercizio

Quale output si ottiene invocando il metodo q?

```
1  class D {
2      private Integer i1 = new Integer(0);
3      private Float f1 = new Float(1.5);
4      private Integer i4 = new Integer(20);
5      void q() {
6          m(i1, f1, new Integer(20));
7      }
8      void m(Integer i2, Object f2, Integer i3) {
9          if(i1 == i2) {
10             System.out.print(1);
11         } else {
12             System.out.print(0);
13         }
14         if(f2 == f1) {
15             System.out.print(1);
16         } else {
17             System.out.print(0);
18         }
19         if(i3 == i4) {
20             System.out.print(1);
21         } else {
22             System.out.print(0);
23         }
24     }
25 }
```



- A. 011
- B. 111
- C. 110
- D. 010
- E. 100

Risposta corretta: C

Esercizio

Qual è l'output di questo codice?

```
1 class MyExc1 extends Exception { }
2 class MyExc2 extends MyExc1 { }
3 class MyExc3 extends MyExc1 { }
4 public class C1 {
5     public static void main(String [] argv) {
6         try {
7             System.out.print(1);
8             m();
9         }
10        catch( MyExc2 g ) {
11            System.out.print(2);
12        }
13        catch( MyExc3 w ) {
14        }
15    }
16    static void m() {
17        try {
18            System.out.print(3);
19            throw( new MyExc3() );
20        }
21        catch( MyExc2 f ) {
22            System.out.print(4);
23        }
24        catch( MyExc3 v ) {
25            System.out.print(5);
26        }
27        catch( Exception i ) {
28        }
29        finally {
30            System.out.print(6);
31            throw( new MyExc1() );
32        }
33    }
34 }
```



- A. 135
- B. 1356Exception in thread "main" MyExc1
- C. Errore a tempo di compilazione
- D. 1356
- E. Nessuna delle precedenti

Risposta corretta: C

Soluzione Da notare che dal metodo `m` viene sollevata una eccezione di tipo `MyExc1` che non viene mai catturata nel blocco `try ...catch` nel `main`, da qui l'errore a compile-time.

Esercizio

Dire quale delle seguenti affermazioni è vera:

1. Due metodi non possono avere diverso nome ma lo stesso tipo di parametri
2. Quando due metodi hanno lo stesso nome si ha overloading
3. Non tutti i tipi di eccezioni estendono la classe `Object`
4. Un array non possiede dei membri
5. Due metodi possono differire per il tipo di ritorno

Risposta corretta: 5

Esercizi

Qual'è l'output di questo programma?

```
1 public class C {  
2     public static void main(String args[]){  
3         C[] c = new C[10];  
4         System.out.println(c[6]);  
5     }  
6 }
```



- A. Errore a tempo di compilazione per errata dichiarazione di un array.
- B. Viene lanciata una `NullPointerException`.
- C. Viene lanciata una `ArrayIndexOutOfBoundsException`.
- D. `null`
- E. Nessuna delle precedenti.

Risposta corretta: D

Esercizio

Qual è l'output di questo codice?

```
1 class MyExc1 extends Exception { }  
2 class MyExc2 extends MyExc1 { }  
3 class MyExc3 extends Exception { }  
4 public class A1 {  
5     public static void main(String [] argv)  
6     throws Exception {  
7         try {  
8             System.out.print(1);  
9             q();  
10        }  
11        catch( MyExc1 v ) {  
12            System.out.print(2);  
13        }  
14        catch( Exception d ) {  
15            System.out.print(3);  
16        }  
17        finally {  
18            System.out.print(4);  
19            throw( new MyExc2() );  
20        }  
21    }  
22    static void q()  
23    throws Exception {  
24        try {  
25            System.out.print(5);  
26            if (true) throw( new MyExc3() );  
27            else throw( new MyExc1() );  
28        }  
29        catch( MyExc1 z ) {  
30            System.out.print(6);  
31        }  
32        finally {  
33            System.out.print(7);  
34        }  
35    }  
36 }
```



- A. `1574Exception` in thread "main" `MyExc2`
- B. 157
- C. 157342
- D. Errore a tempo di compilazione
- E. Nessuna delle precedenti

Risposta corretta: E

Esercizio

Qual è l'output di questo codice?

```
1  class MyExc1 extends Exception { }
2  class MyExc2 extends Exception { }
3  class MyExc3 extends Exception { }
4  public class D1 {
5      public static void main(String [] argv)
6          throws Exception {
7          try {
8              System.out.print(1);
9              p();
10         }
11         catch( Exception i ) {
12             System.out.print(2);
13         }
14         catch( MyExc3 a ) {
15         }
16         finally {
17             System.out.print(3);
18             throw( new MyExc2() );
19         }
20     }
21     static void p()
22         throws Exception {
23         try {
24             System.out.print(4);
25             throw( new MyExc1() );
26         }
27         catch( MyExc2 f ) {
28         }
29         catch( MyExc3 b ) {
30         }
31         catch( MyExc1 f ) {
32             System.out.print(5);
33         }
34         finally {
35             System.out.print(6);
36             throw( new MyExc3() );
37         }
38     }
39 }
```



- A. 145
- B. 145632
- C. Errore a tempo di compilazione
- D. 145623Exception in thread "main" MyExc2
- E. Nessuna delle precedenti

Risposta corretta: C

Esercizio

Qual è il risultato della compilazione ed esecuzione del seguente programma?

```
1 public class Vehicle {
2     static String producer, model;
3 }
4 Vehicle(String p, String m) {
5     producer = p;
6     model = m;
7 }
8 public static String id() {
9     return producer + " ";
10 }
11
12 class Car extends Vehicle {
13     Car(String p, String m) {super(p,m);}
14
15     public static String id() {
16         return producer + " " +
17             model + " ";
18     }
19 }
20
21 class Main {
22     public static void main (String[] args){
23         Vehicle v = new Vehicle("Fiat","500");
24         Vehicle c = new Car("Opel","Corsa");
25         Car d = new Car("Citroen","C3");
26         System.out.print(v.id());
27         System.out.print(c.id());
28         System.out.print(d.id());
29     }
30 }
```



- A. Fiat Opel Citroen C3
- B. Fiat Opel Corsa Citroen C3
- C. Citroen Citroen Citroen C3
- D. Errore a tempo di compilazione.
- E. Errore a tempo di esecuzione.

Risposta corretta: C

Esercizio

Date le dichiarazioni:

```
1 Boolean m;
2 Error q;
3 Object u;
4 q = new Error();
5 u = new Error();
6 m = new Boolean(false);
```



indicare quali dei seguenti assegnamenti sono corretti a tempo di esecuzione:

- A. `q = (Error) u;`
- B. `q = (Error) m;`
- C. `m = (Boolean) q;`
- D. `m = (Boolean) u;`
- E. Nessuno dei precedenti

Risposta corretta: A

Esercizio

Date le dichiarazioni:

```
1 class Super extends Object { ... }
2 class C2 extends Super { ... }
3 class B1 extends Super { ... }
```



e le dichiarazioni di variabile:

```
1 Super n;
2 B1 r;
3 C2 z;
```



indicare quali dei seguenti assegnamenti sono corretti a tempo di compilazione:

- A. `r = n;`
- B. `n = r;`
- C. `z = r;`
- D. `r = z;`
- E. `z = n;`

Risposta corretta: B

Esercizio

Qual è il risultato della compilazione ed esecuzione del seguente programma?

```
1 public class Vehicle {
2     String producer, model;
3     Vehicle(String p, String m) {
4         producer = p;
5         model = m;
6     }
7     public String id() {
8         return producer + " ";
9     }
10 }
11
12 class Car extends Vehicle {
13     Car(String p, String m) {super(p,m);}
14
15     public String id() {
16         return producer + " " +
17         model + " ";
18     }
19 }
20
21 class Main {
22     public static void main(String[] args) {
23         Vehicle v = new Vehicle("Fiat","500");
24         Vehicle c = new Car("Opel","Corsa");
25         Car d = new Car("Citroen","C3");
26         System.out.print(v.id());
27         System.out.print(c.id());
28         System.out.print(d.id());
29     }
30 }
```



- A. Fiat Opel Citroen C3
- B. Fiat Opel Corsa Citroen C3
- C. Citroen Citroen Citroen C3
- D. Errore a tempo di compilazione.
- E. Errore a tempo di esecuzione.

Risposta corretta: B

Esercizio

Quale output si ottiene invocando il metodo `g`?

```
1 class X {
2     void f(X x) {
3         System.out.println("X");
4     }
5 }
6 class Y extends X {}
7 class Z extends Y {
8     void f(Z z) {
9         System.out.println("Z");
10    }
11    void g() {
12        Y y = new Y();
13        Object o = new Z();
14        f(y); f(o);
15    }
16 }
```

- A. X X
- B. X Z
- C. Z Z
- D. Errore a tempo di compilazione
- E. Errore a tempo di esecuzione

Risposta corretta: D

7.4

ESERCIZI LINGUAGGIO ML



Esercizio

Scrivere in ML la funzione `reduce`.

```
1 fun reduce f t [] = e
2 | reduce f t (x::y) = f(x, reduce f t y);
```

Esercizio

Scrivere in ML la funzione che calcola `n!`.

```
1 fun fatt x = if x = 0 then 1 else x*fatt(x-1);
```

Esercizio

Scrivere in ML la funzione `filter`.

```
1 fun filter f [] = []
2 | filter f (x::y) = if f(x) then x::(filter f y)
3 else filter f y;
```

Esercizio

Scrivere in ML la funzione `map`.

```
1 fun map f [] = []
2 | map f (x::y) = f(x)::(map f y);
```



Esercizio

Scrivere in Prolog un predicato che calcola la lunghezza di una lista.

```
1 list_lenght([],0).
2 list_lenght([_|Tail],N) :- list_length(Tail,N1), N is N1+1.
```



Esercizio

Scrivere in Prolog un predicato che calcola il minimo degli elementi di una lista.

Per definire un predicato `list_min_elem([X],X)` che restituisca il minimo degli elementi di una lista bisogna innanzitutto definire come trovare il minimo tra due elementi. Si ottiene quindi, dalla definizione di minimo:

$$\min(x,y) = \begin{cases} x & \text{se } x \leq y \\ y & \text{altrimenti} \end{cases}$$

Quindi:

```
1 min_of_two(X,Y,X) :- X ≤ Y.
2 min_of_two(X,Y,Y) :- Y < X.
```



Una volta aver definito tale predicato possiamo passare alla definizione del predicato `list_min_elem` il quale può essere definito in questo modo:

- Se la lista contiene un solo elemento allora esso sarà il minimo;
- Se la lista contiene più di un elemento allora possiamo scriverla in modo parziale come `[X,Y|Rest]`, ovvero come una lista che ha la variabile `X` come testa e una sottolista che ha `Y` in testa e una coda che chiamiamo `Rest`. Allora, per trovare tale minimo basterà applicare ricorsivamente il predicato sulla sottolista `[Y|Rest]` fino ad arrivare alla sottolista con un solo elemento, dopo di che basterà applicare il predicato `min_of_two` per aggiornare di volta in volta il valore minimo.

```
1 min_of_two(X,Y,X) :- X ≤ Y.
2 min_of_two(X,Y,Y) :- Y < X.
3 list_min_elem([X],X).
4 list_min_elem([X,Y|Rest],X) :-
5 list_min_elem([Y|Rest],MinRest),
6 min_of_two(X,MinRest,Min).
```



Esercizio

Scrivere in Prolog un predicato che divide tutti gli elementi di una lista di interi per la lunghezza della lista.

```
1 dividi_lista([], []).
2 dividi_lista([X|Resto], [Diviso|RestoDiviso]) :-
3 list_lenght([X|Resto], Lunghezza),
4 Diviso is X / Lunghezza,
5 dividi_lista(Resto, RestoDiviso).
```



Esercizio

Scrivere in Prolog un predicato che calcola il massimo degli elementi di una lista.

```
1 max_of_two(X,Y,X) :- X ≥ Y.
2 max_of_two(X,Y,Y) :- X < Y.
3 list_max_elem([X],X).
4 list_max_elem([X,Y|Rest],Max) :-
5 list_max_elem([Y|Rest],MaxRest),
6 max_of_two(X,MaxRest,Max).
```



Esercizio

Scrivere in Prolog un predicato che calcola la lista degli elementi pari di una lista.

Il predicato `lista_pari` restituisce una lista vuota se la lista non contiene elementi, altrimenti:

- La prima clausola aggiunge l'elemento corrente alla lista degli elementi pari se è un numero pari, e poi richiama ricorsivamente se stessa per il resto della lista.
- La seconda clausola gestisce il caso in cui l'elemento corrente non è pari, ignorandolo e proseguendo con il resto della lista.

```
1 lista_pari([], []).
2 lista_pari([X|Resto], [X|Pari]) :-
3   X mod 2 == 0,
4   lista_pari(Resto, Pari).
5 lista_pari(_|Resto, Pari) :- lista_pari(Resto, Pari).
```



Esercizio

Scrivere in Prolog un predicato che calcola la lista degli elementi positivi di una lista.

Possiamo definire un predicato `positivo` che verifica se un numero è positivo:

```
1 positivo(X) :- X>0.
```



Possiamo utilizzare tale predicato all'interno del predicato `lista_positivi` per ottenere la lista degli elementi positivi:

```
1 lista_positivi([], []).
2 lista_positivi([X|Resto], [X|Positivi]) :-
3   positivo(X),
4   lista_positivi(Resto, Positivi).
5 lista_positivi(_|Resto, Positivi) :-
6   lista_positivi(Resto, Positivi).
```



Esercizio

Scrivere in Prolog un predicato che calcola la somma degli elementi di una lista.

In questo esercizio si definisce il predicato `list_sum(List,Sum)` che restituisce la somma degli elementi della lista. Chiaramente:

- Se la lista è vuota allora la somma sarà pari a zero;
- Se la lista non è vuota allora è rappresentabile come `[Head|Tail]`; per trovare la somma possiamo applicare ricorsivamente il predicato alla coda fino a quando non si ottiene la sottolista vuota e aggiornare man mano il valore `Sum=Head+SumTemp`

Si ottiene così:

```
1 list_sum([],0).
2 list_sum([Head|Tail], Sum) :-
3   list_sum(Tail,SumTemp),
4   Sum is Head + SumTemp.
```



Esercizio

Scrivere in Prolog un predicato che calcola il prodotto degli elementi di una lista.

Il predicato `list_prof` si ottiene in modo analogo al predicato `list_sum` con l'unica differenza di imporre come caso base `list_prod([],1)`:

```
1 list_prod([],1).
2 list_prod([Head|Tail], Prod) :-
3   list_prod(Tail,ProdTemp),
4   Prod is Head * ProdTemp.
```



Esercizio

Scrivere in Prolog un predicato che calcola la lista degli elementi negativi di una lista.

```
1 negativo(X) :- X < 0.  
2  
3 lista_negativi([], []).  
4 lista_negativi([X|Resto], [X|Negativi]) :-  
5   negativo(X),  
6   lista_negativi(Resto, Negativi).  
7 lista_negativi(_|Resto, Negativi) :-  
8   lista_negativi(Resto, Negativi).
```

